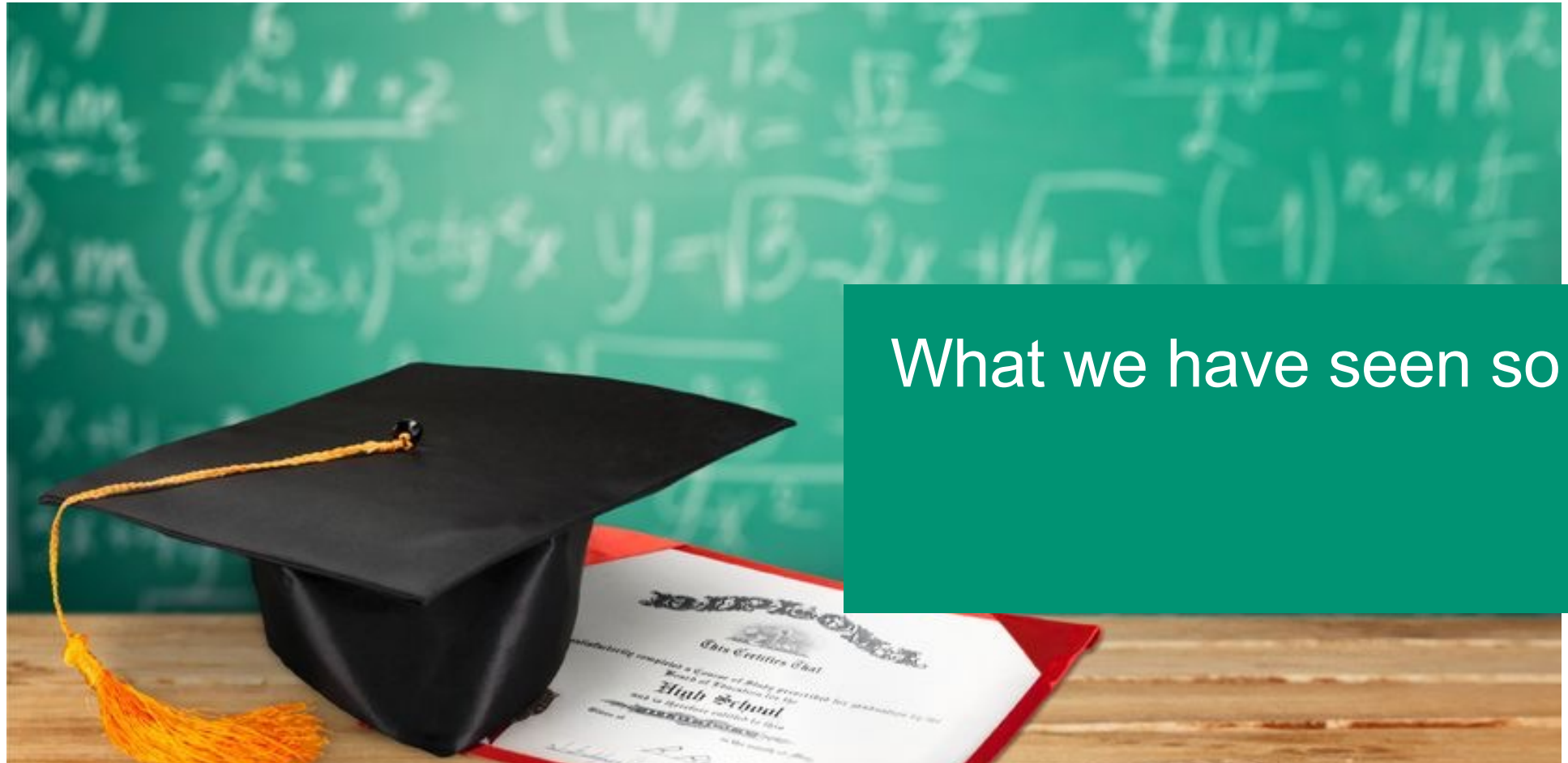


Information Retrieval

4. Tolerant retrieval

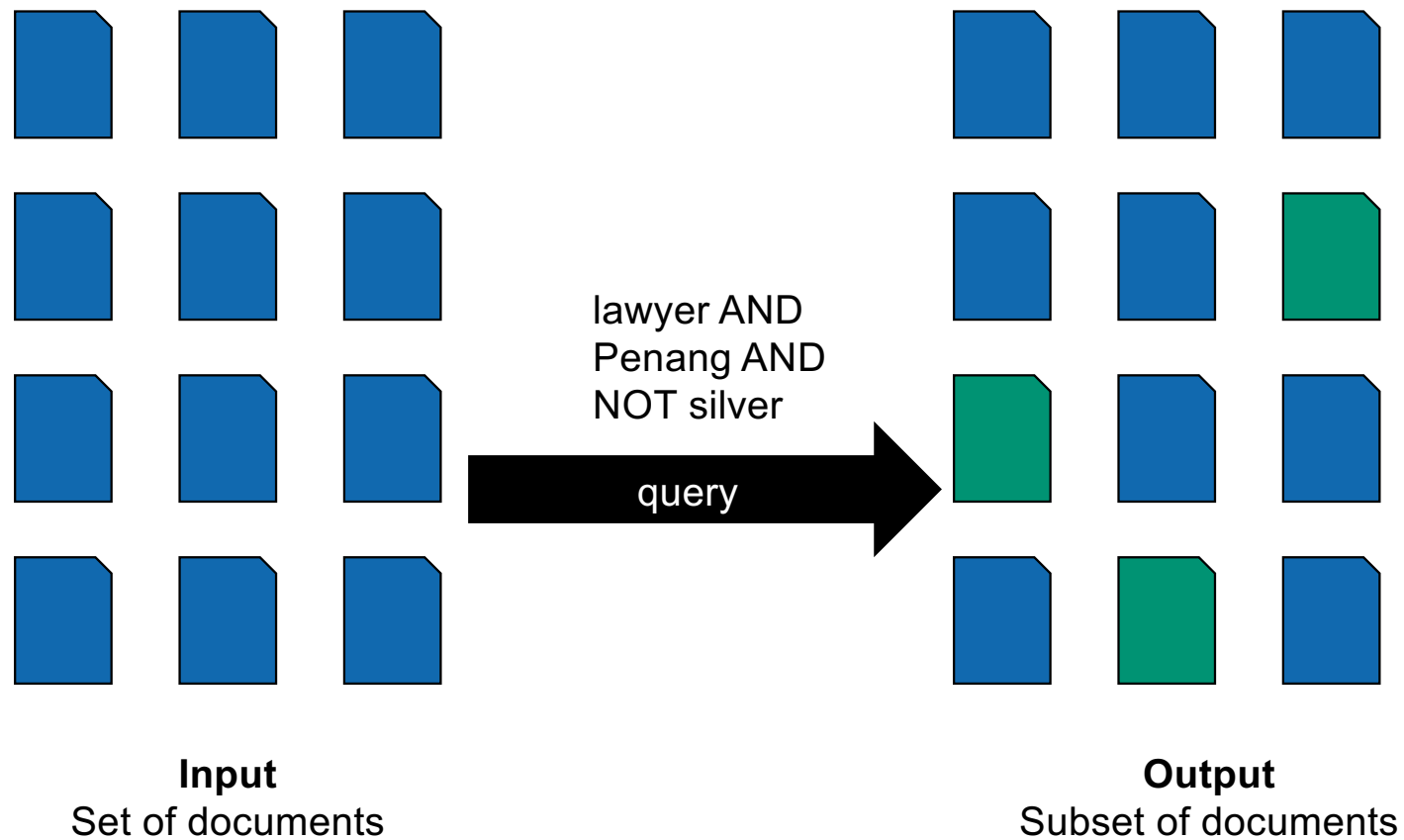
Ghislain Fourny
Spring 2022

4

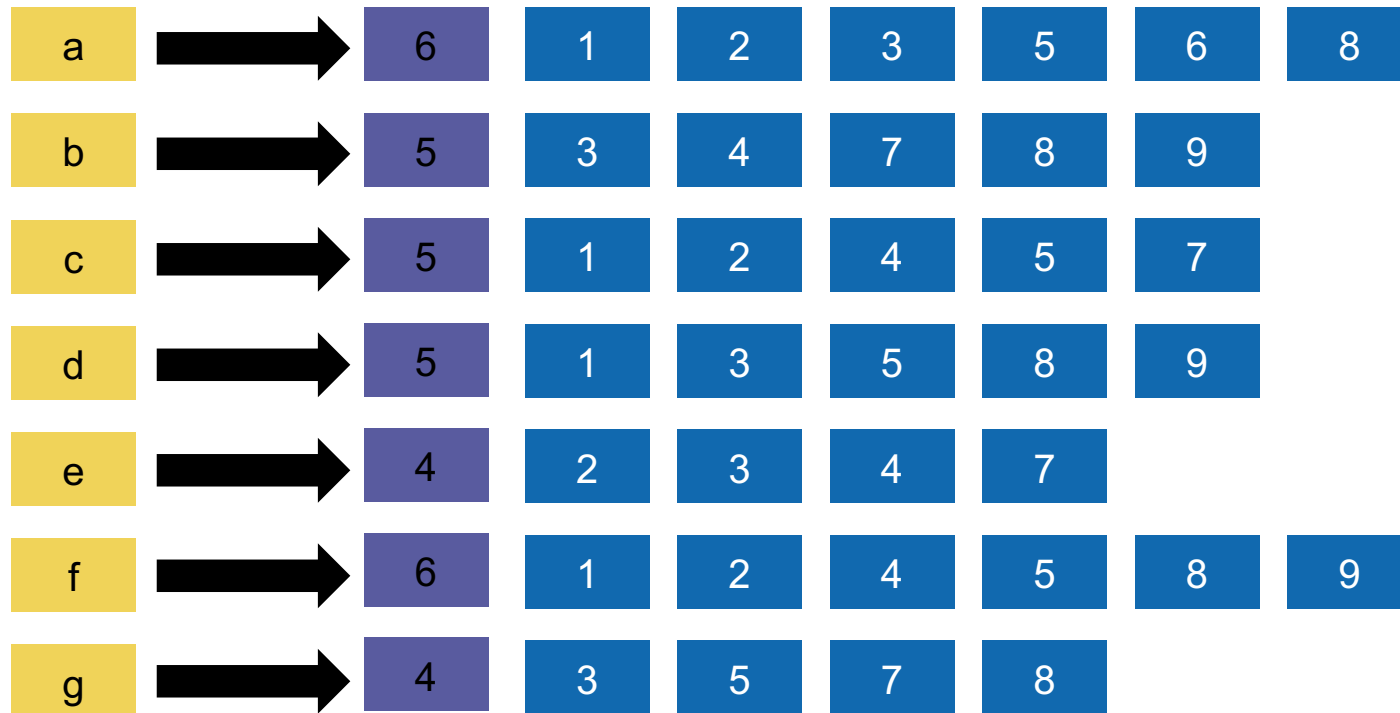


What we have seen so far

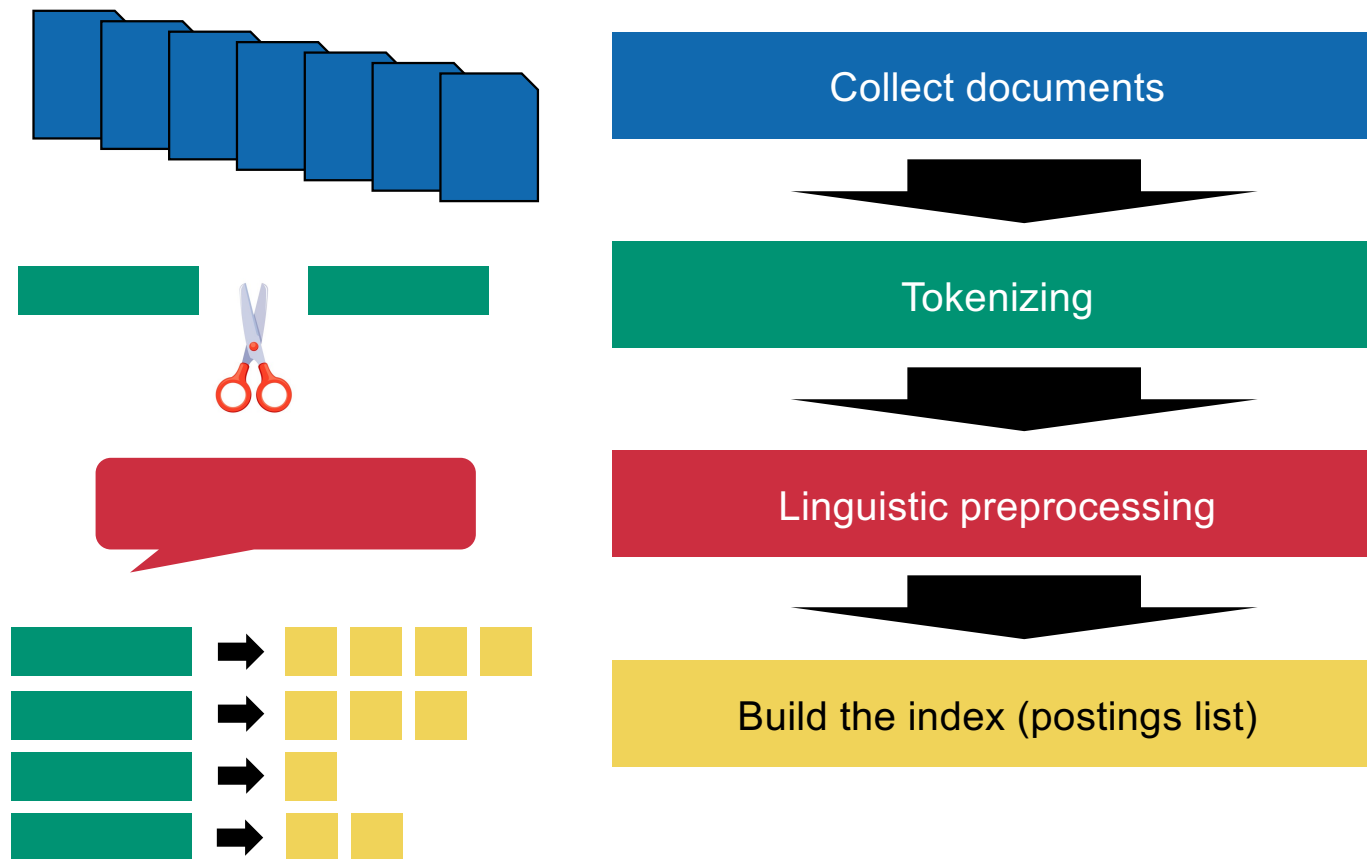
Boolean retrieval



Standard Inverted Index



Index construction in reality...

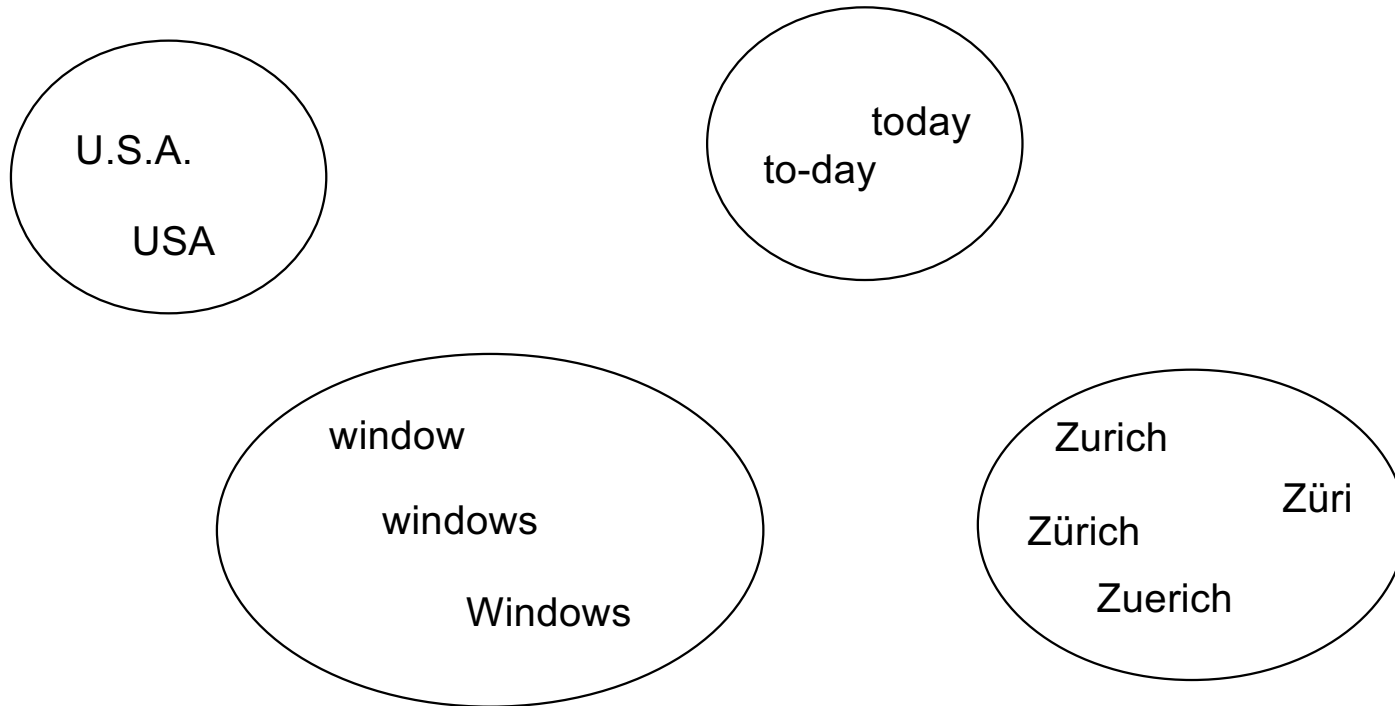


Stop words

a
an
and
are
as
at
be
by
for
from
has
he
in

is
it
its
of
on
that
the
to
was
were
will
with

Equivalence classes of terms (types)

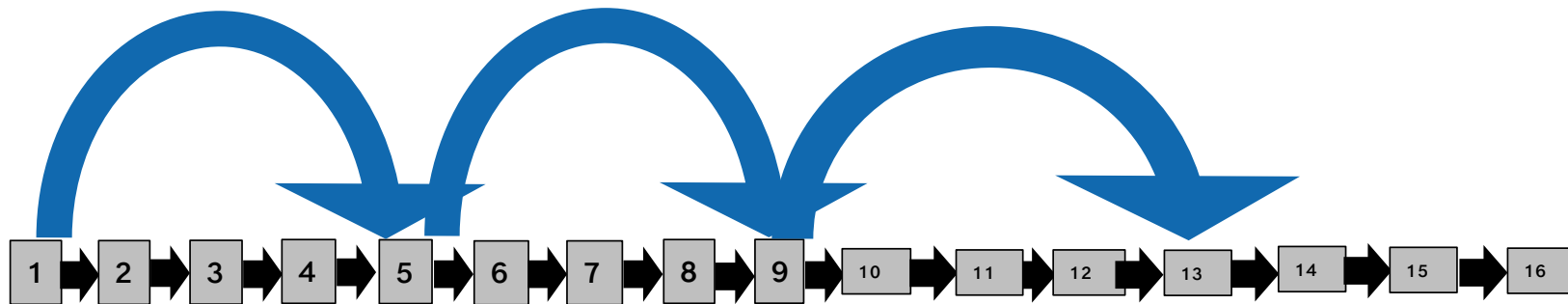


Porter Stemmer

(m>0) ENCI -> ENCE	valenci -> valence
(m>0) ANCI -> ANCE	hesitanci -> hesitance
(m>0) IZER -> IZE	digitizer -> digitize
(m>0) ABLI -> ABLE	conformabli -> conformable
(m>0) ALLI -> AL	radicalli -> radical
(m>0) ENTLI -> ENT	differentli -> different
(m>0) ELI -> E	vileli -> vile
(m>0) OUSLI -> OUS	analogousli -> analogous
(m>0) IZATION -> IZE	vietnamization -> vietnamize
(m>0) ATION -> ATE	predication -> predicate
(m>0) ATOR -> ATE	operator -> operate
(m>0) ALISM -> AL	feudalism -> feudal
(m>0) IVENESS -> IVE	decisiveness -> decisive
(m>0) FULNESS -> FUL	hopefulness -> hopeful
(m>0) OUSNESS -> OUS	callousness -> callous

<https://tartarus.org/martin/PorterStemmer/>

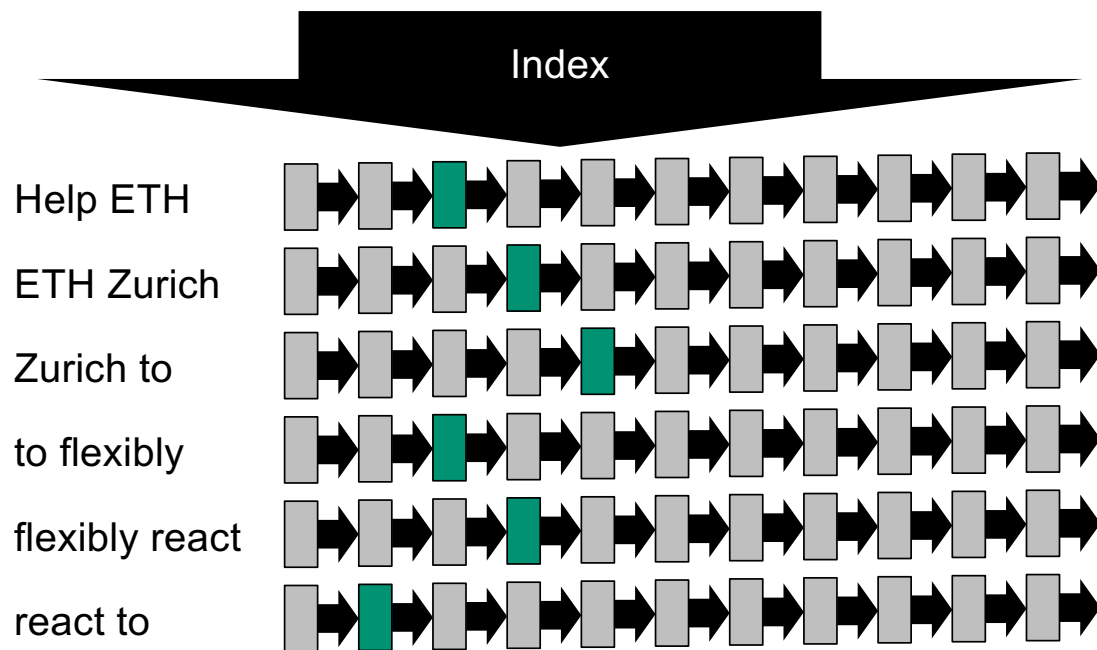
Skip pointers (performance improvement)



In practice $\sqrt{\text{Number of postings}}$

Bi-word indices (Phrase search feature)

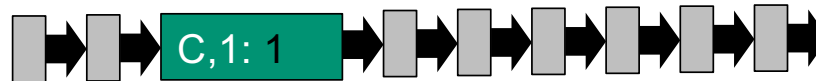
Help ETH Zurich to flexibly react to new challenges and to set new accents in the future.



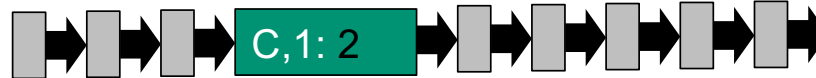
Positional index (phrase search feature)

"ETH Zurich"

Help



ETH



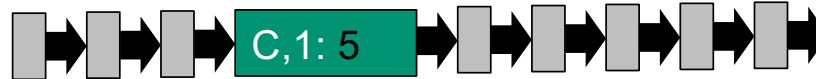
Zurich



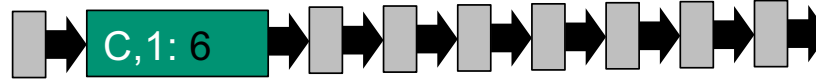
to



flexibly



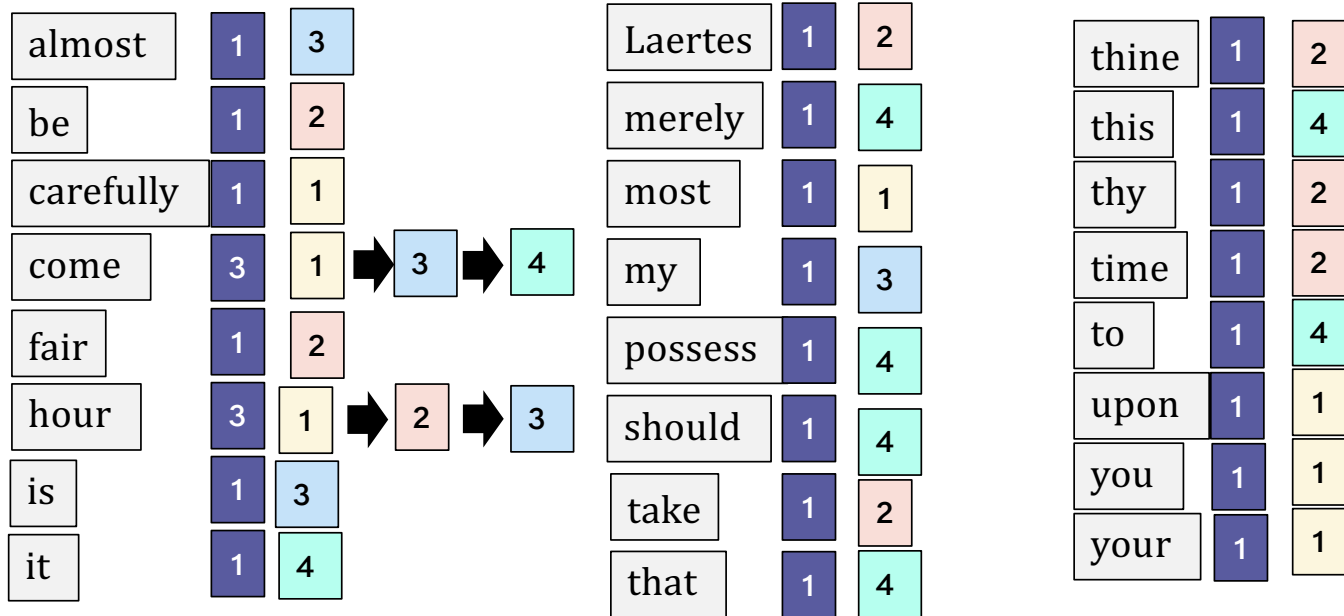
react



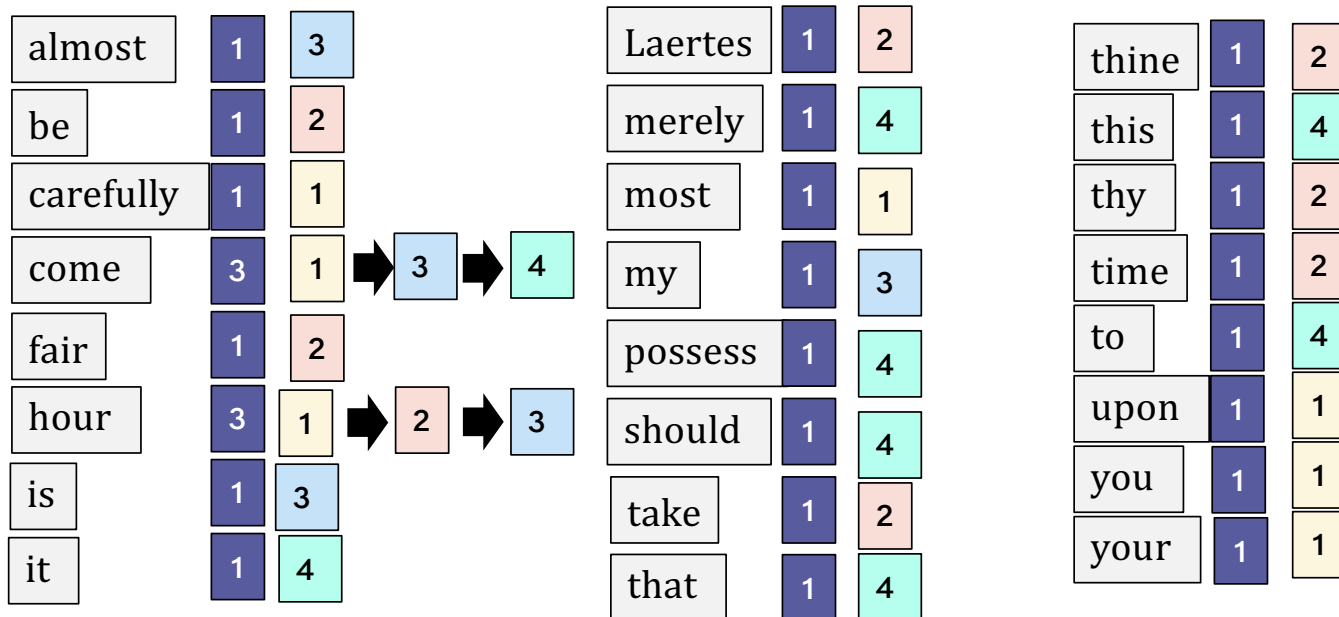


Search structures

Inverted index



Inverted index



But how do we lookup terms based on a query?

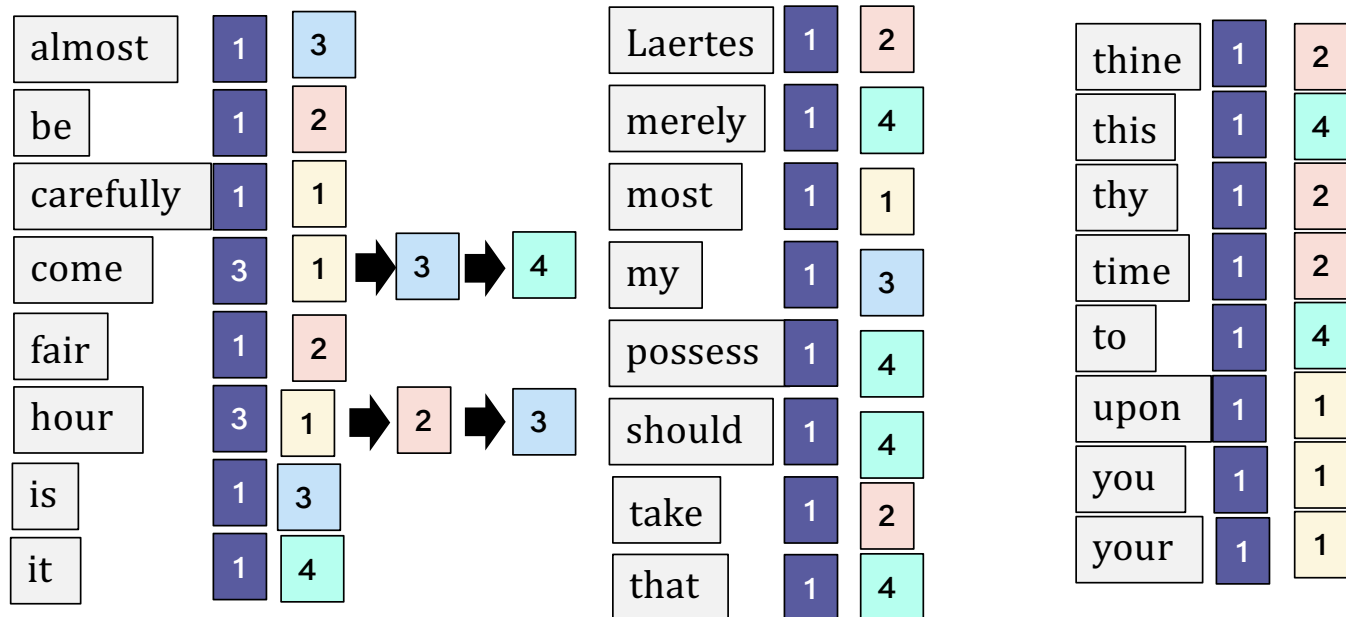
Looking up a term

Figure 1 displays a visualization of the word co-occurrence matrix for the sentence "almost be carefully come fair hour is it Laertes merely most my possess should take that thine this thy time to upon you your". The matrix is divided into three sections, each showing the co-occurrence counts for pairs of words in the sentence. The words are arranged in rows, and the counts are shown in colored boxes (blue, yellow, red, green) indicating different frequency ranges. Arrows indicate the sequence of words in the sentence.

Word 1	Word 2	Count
almost	be	1
almost	carefully	3
almost	come	1
almost	fair	1
almost	hour	3
almost	is	1
almost	it	1
be	carefully	2
be	come	1
be	fair	2
be	hour	1
be	is	3
be	it	4
carefully	come	1
carefully	fair	1
carefully	hour	1
carefully	is	3
carefully	it	4
come	fair	1
come	hour	3
come	is	1
come	it	4
fair	hour	2
fair	is	1
fair	it	4
hour	is	3
hour	it	4
is	it	4
Laertes	merely	1
Laertes	most	1
Laertes	my	1
Laertes	possess	1
Laertes	should	1
Laertes	take	1
Laertes	that	1
merely	most	4
merely	my	3
merely	possess	4
merely	should	4
merely	take	2
merely	that	4
most	my	1
most	possess	4
most	should	4
most	take	2
most	that	4
my	possess	3
my	should	4
my	take	2
my	that	4
possess	should	4
possess	take	2
possess	that	4
should	take	2
should	that	4
take	that	4
thine	this	1
thine	thy	1
thine	time	1
thine	to	1
thine	upon	1
thine	you	1
thine	your	1
this	thy	4
this	time	2
this	to	4
this	upon	1
this	you	1
this	your	1
thy	time	2
thy	to	4
thy	upon	1
thy	you	1
thy	your	1
time	to	4
time	upon	1
time	you	1
time	your	1
to	upon	1
to	you	1
to	your	1
upon	you	1
upon	your	1
you	your	1

1. Does it exist?

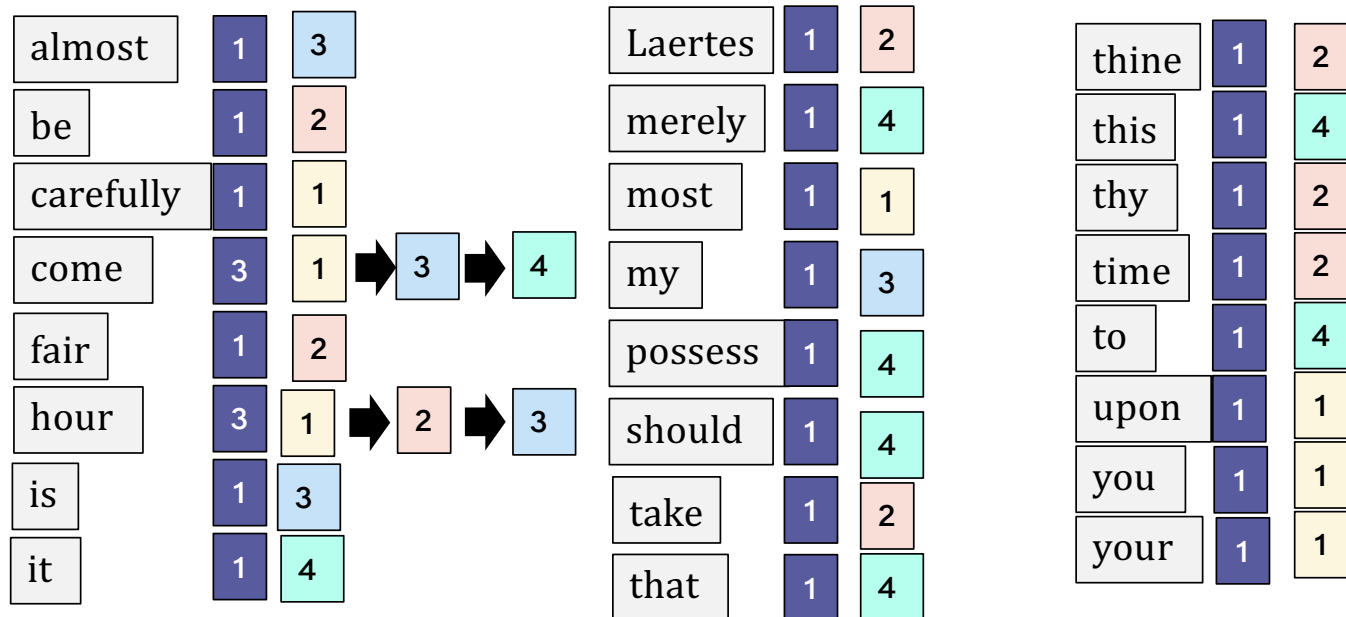
Looking up a term



1. Does it exist?

should

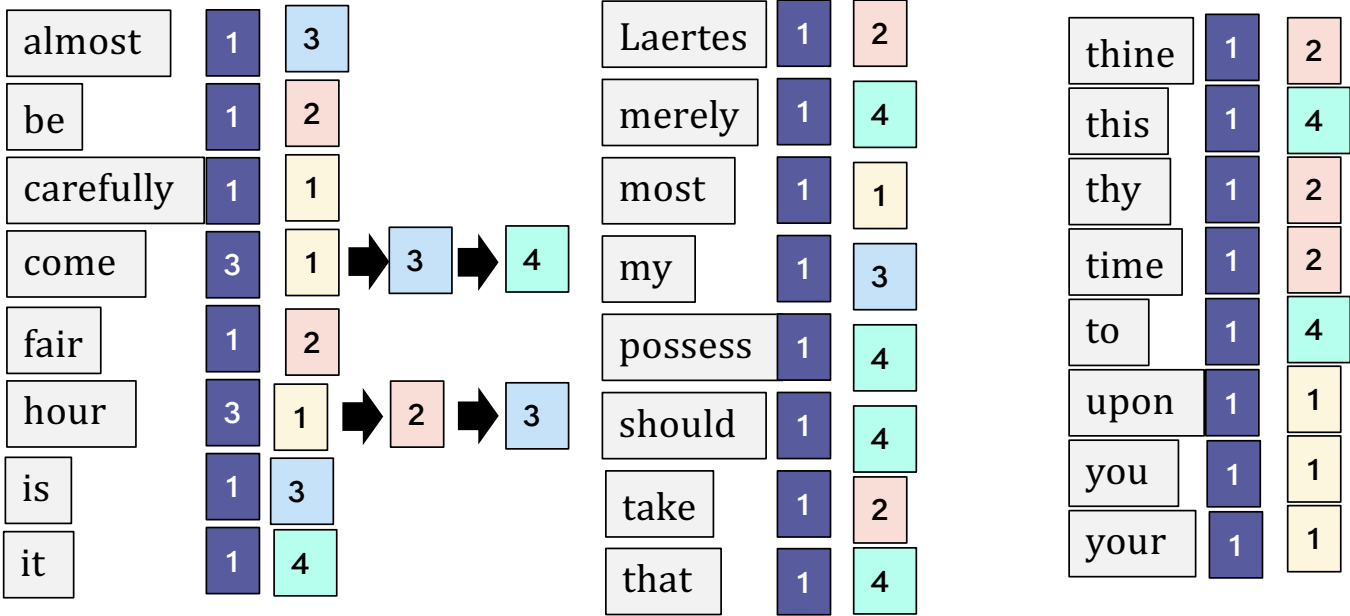
Looking up a term



1. Does it exist?

should

Looking up a term

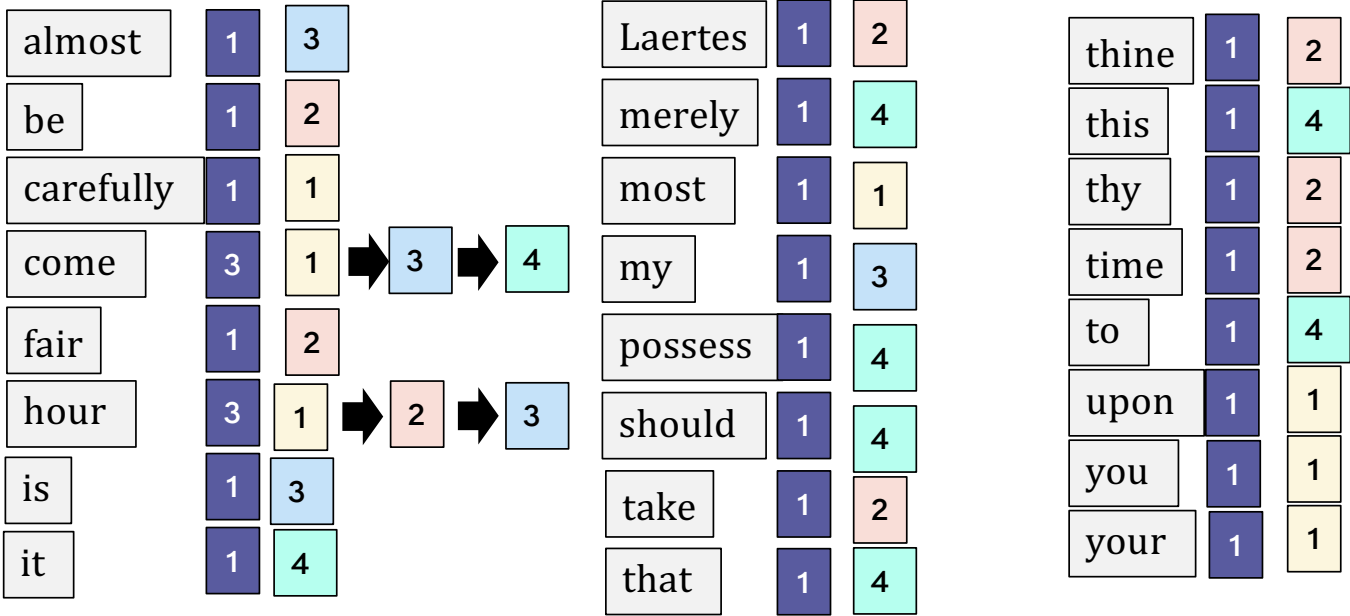


1. Does it exist?

should

could

Looking up a term

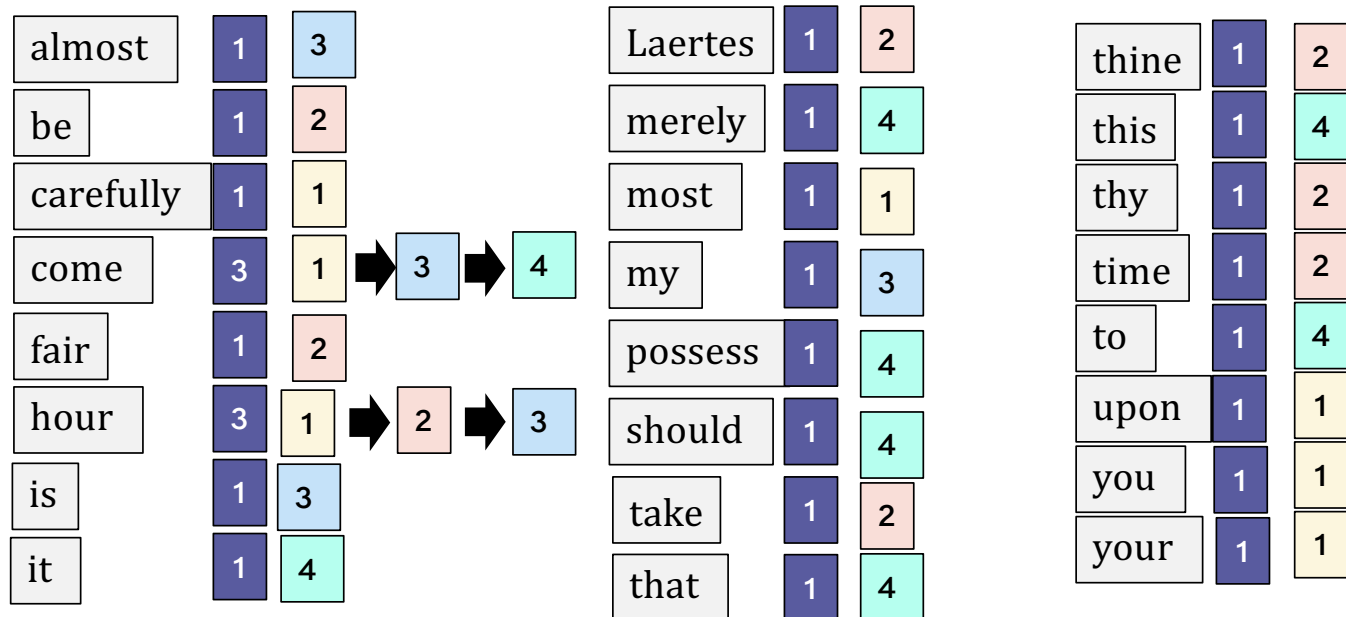


1. Does it exist?

should

could

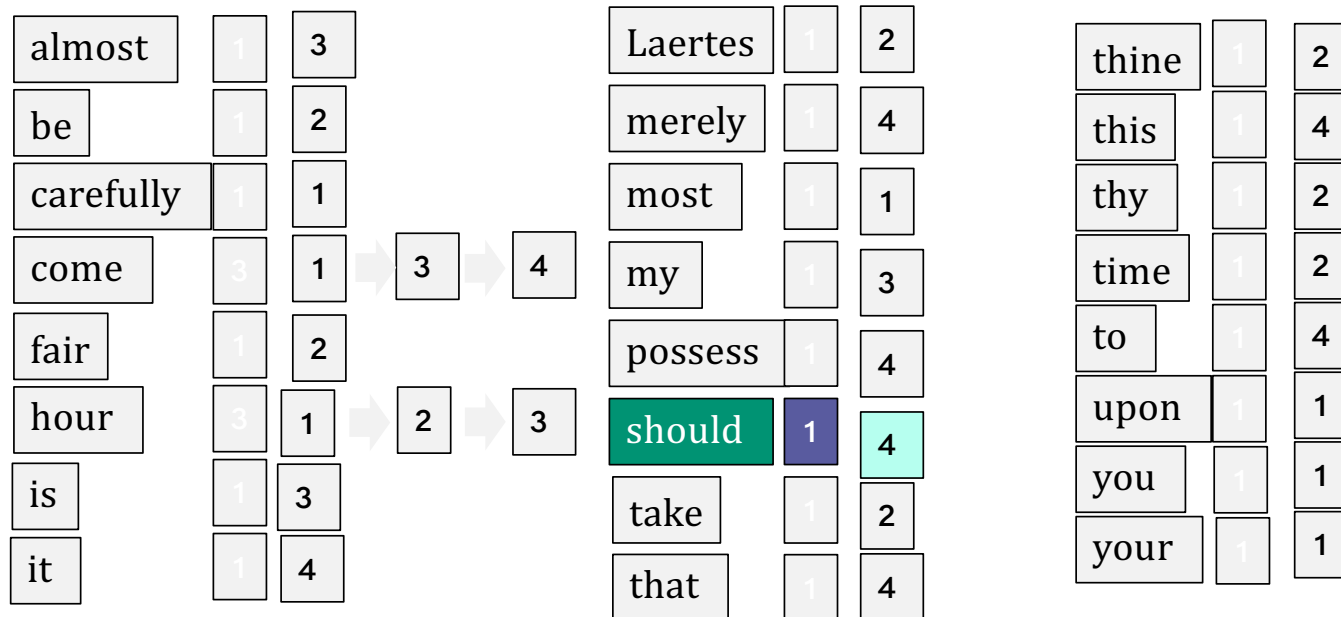
Looking up a term



2. Locate the postings list

should

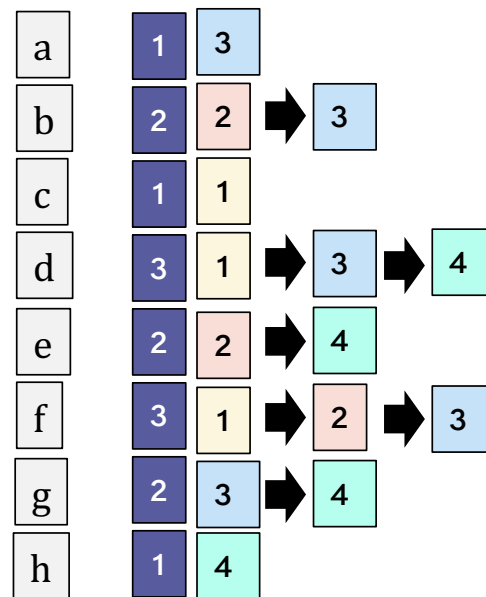
Looking up a term



2. Locate the postings list

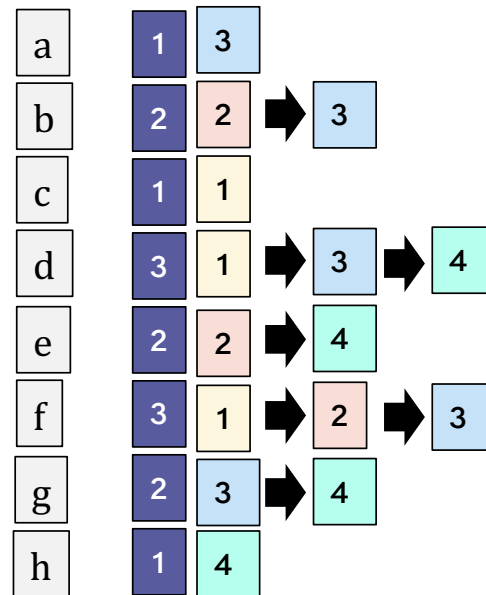
should

The missing part



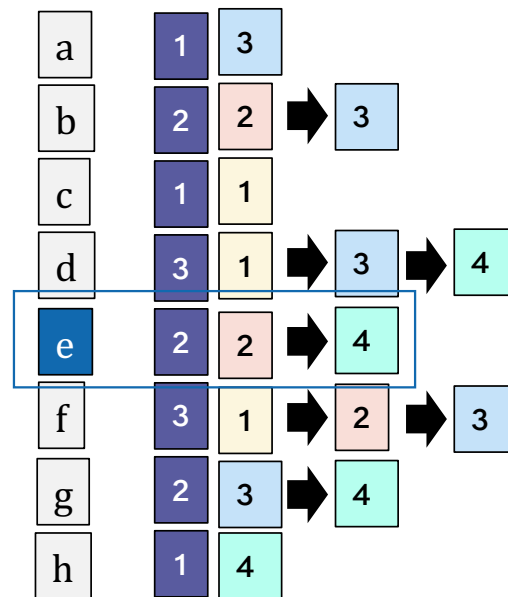
The missing part

e?

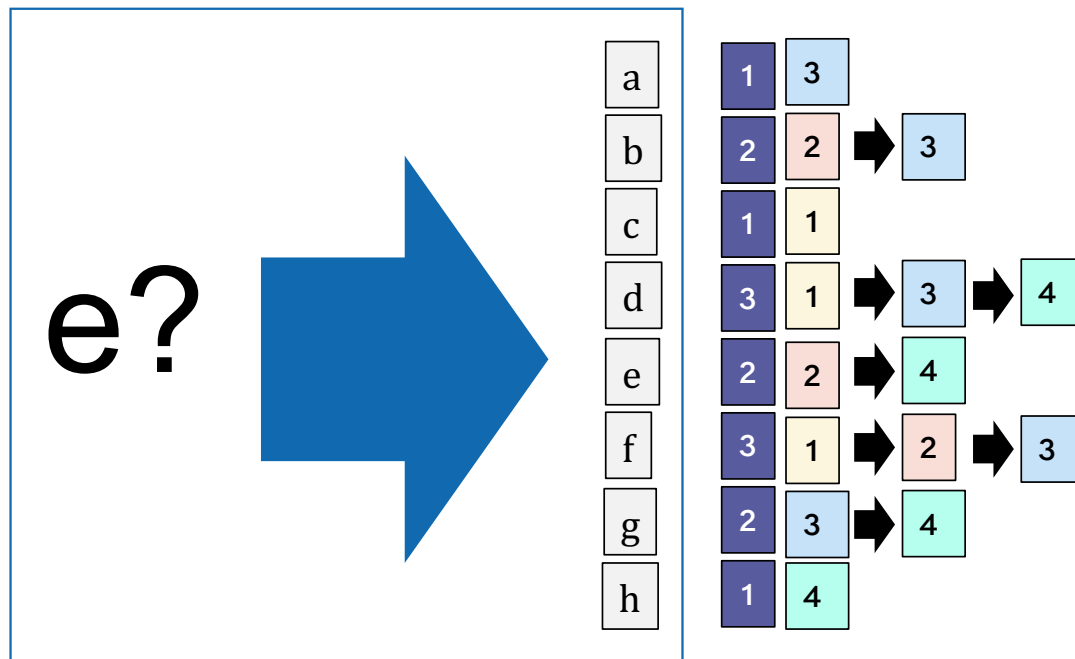


The missing part

e?



The missing part



Solution 1: hash table

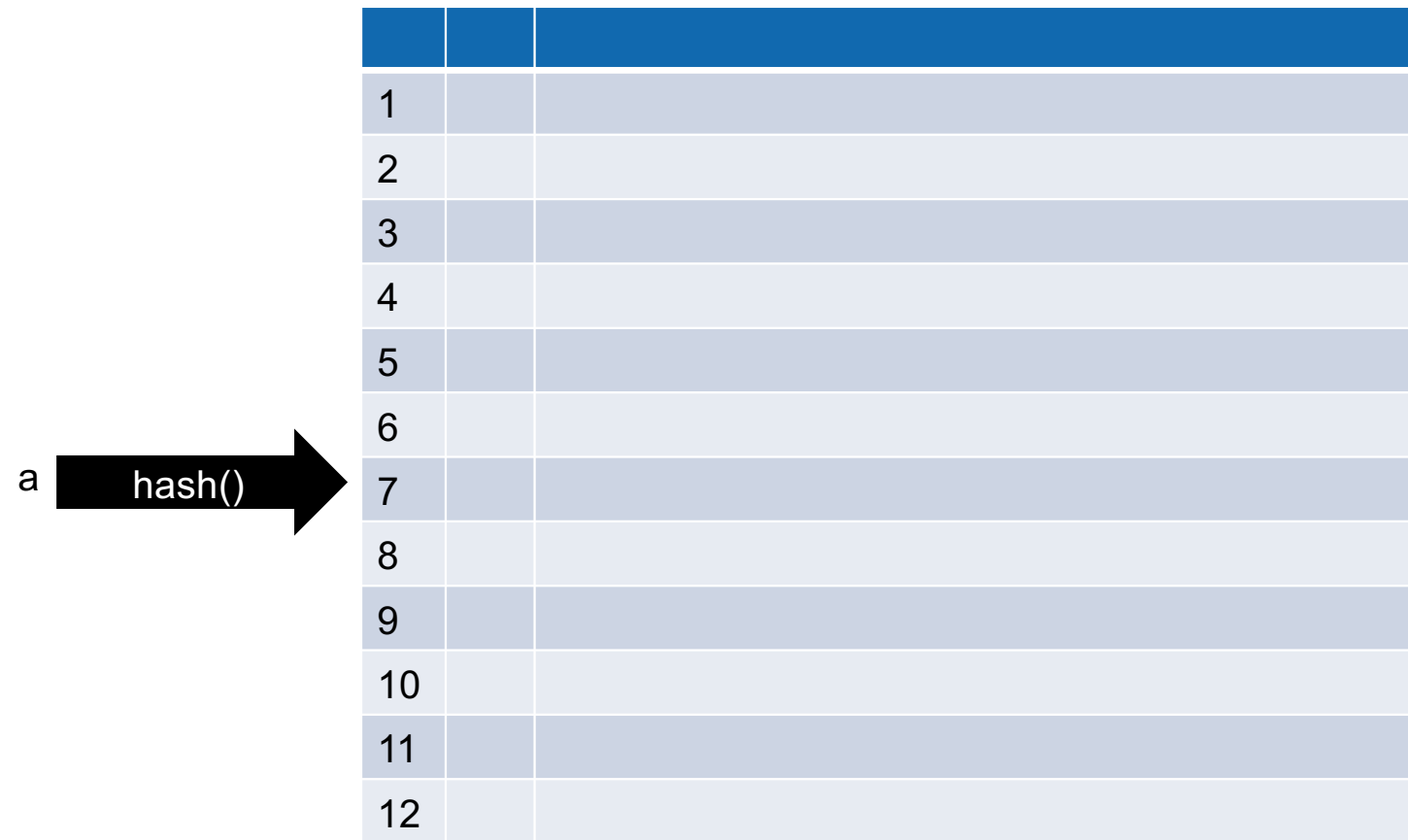
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		

Solution 1: hash table

a

1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		

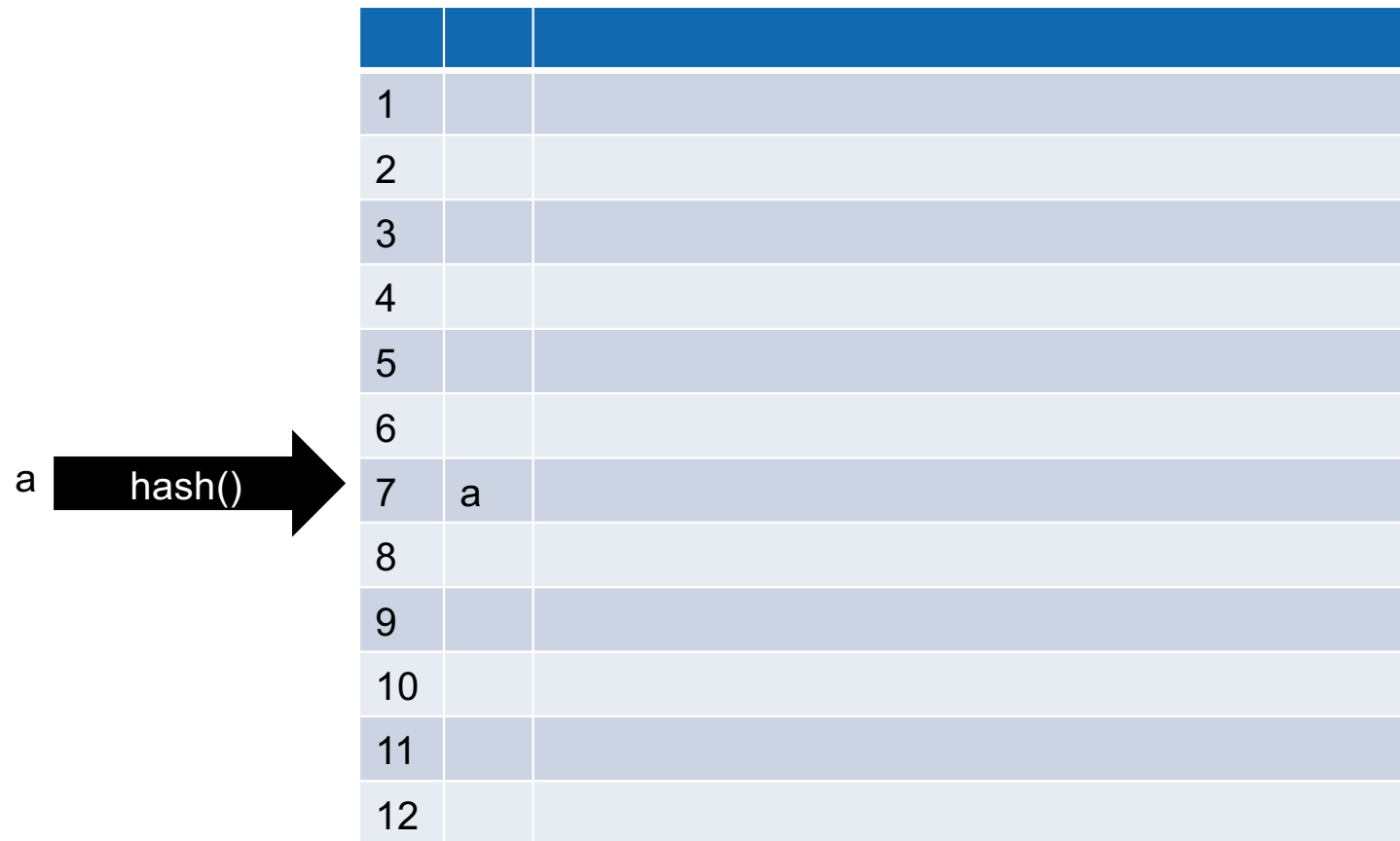
Solution 1: hash table



The diagram illustrates a hash table with 12 slots, indexed from 1 to 12. The slots are arranged in a grid with three columns. The first column contains the indices 1 through 12. The second column is empty. The third column is a solid blue bar. An arrow labeled 'a' and 'hash()' points to slot 7.

1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		

Solution 1: hash table



The diagram illustrates a hash table with 12 slots. An arrow labeled 'a' and 'hash()' points to slot 7, which contains the value 'a'.

1		
2		
3		
4		
5		
6		
7	a	
8		
9		
10		
11		
12		

Solution 1: hash table

1		
2	e	
3	h	
4		
5	b	
6		
7	a	
8	d	
9	c	
10	g	
11		
12	f	

Solution 1: hash table

1		
2	e	2 2 ➡ 4
3	h	1 4
4		
5	b	2 2 ➡ 3
6		
7	a	1 3
8	d	3 1 ➡ 3 ➡ 4
9	c	1 1
10	g	2 3 ➡ 4
11		
12	f	3 1 ➡ 2 ➡ 3

Solution 1: hash table

e?

1		
2	e	2 2 ➔ 4
3	h	1 4
4		
5	b	2 2 ➔ 3
6		
7	a	1 3
8	d	3 1 ➔ 3 ➔ 4
9	c	1 1
10	g	2 3 ➔ 4
11		
12	f	3 1 ➔ 2 ➔ 3

Solution 1: hash table

e? *hash()*

1		
2	e	2 → 2 → 4
3	h	1 → 4
4		
5	b	2 → 2 → 3
6		
7	a	1 → 3
8	d	3 → 1 → 3 → 4
9	c	1 → 1
10	g	2 → 3 → 4
11		
12	f	3 → 1 → 2 → 3

Solution 1: hash table

e?

hash()

1		
2	e	2 → 2 → 4
3	h	1 → 4
4		
5	b	2 → 2 → 3
6		
7	a	1 → 3
8	d	3 → 1 → 3 → 4
9	c	1 → 1
10	g	2 → 3 → 4
11		
12	f	3 → 1 → 2 → 3

Limitations of hash tables

Limitations of hash tables

a-c?

1		
2	e	2 2 ➔ 4
3	h	1 4
4		
5	b	2 2 ➔ 3
6		
7	a	1 3
8	d	3 1 ➔ 3 ➔ 4
9	c	1 1
10	g	2 3 ➔ 4
11		
12	f	3 1 ➔ 2 ➔ 3

Limitations of hash tables

a-c?

1		
2	e	2 2 ➔ 4
3	h	1 4
4		
5	b	2 2 ➔ 3
6		
7	a	1 3
8	d	3 1 ➔ 3 ➔ 4
9	c	1 1
10	g	2 3 ➔ 4
11		
12	f	3 1 ➔ 2 ➔ 3

Limitations of hash tables

Limitations of hash tables

No support for range queries

Limitations of hash tables

No support for range queries

Hash function not perfect in real life

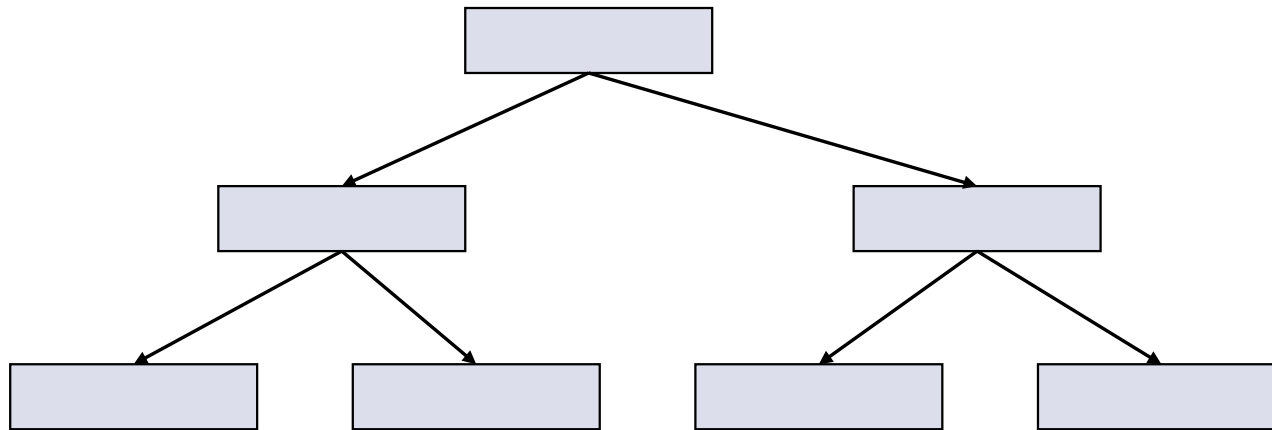
Limitations of hash tables

No support for range queries

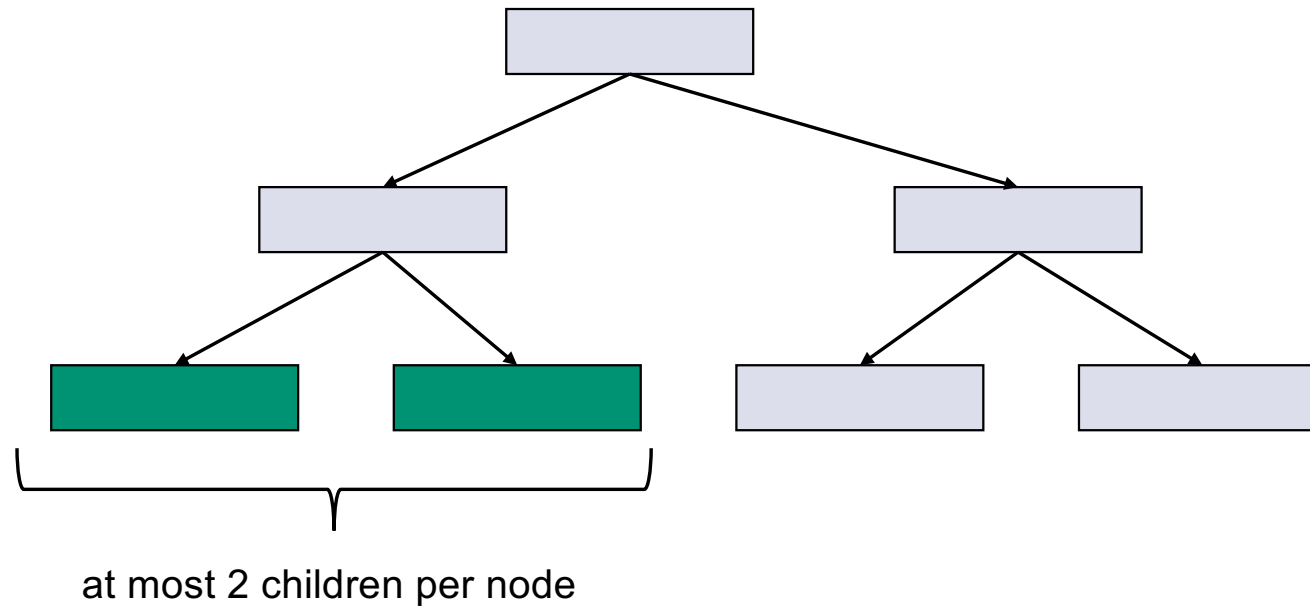
Hash function not perfect in real life

Space requirements for collision avoidance

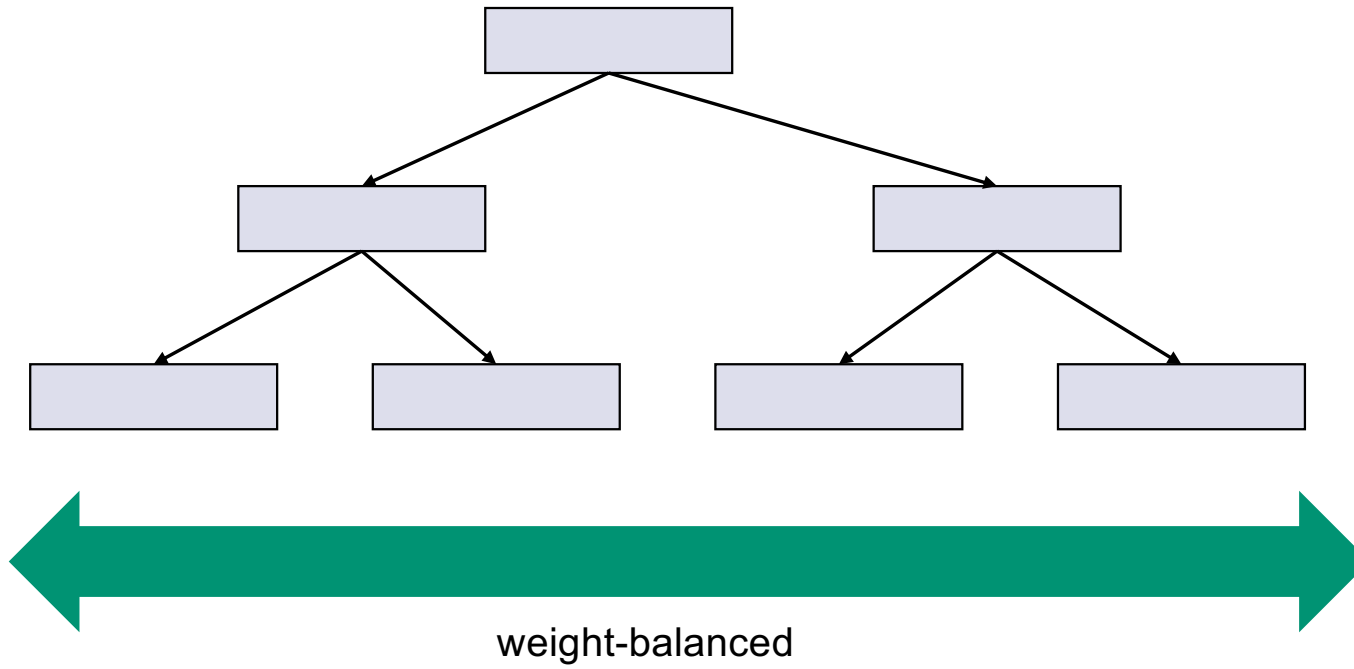
Binary tree



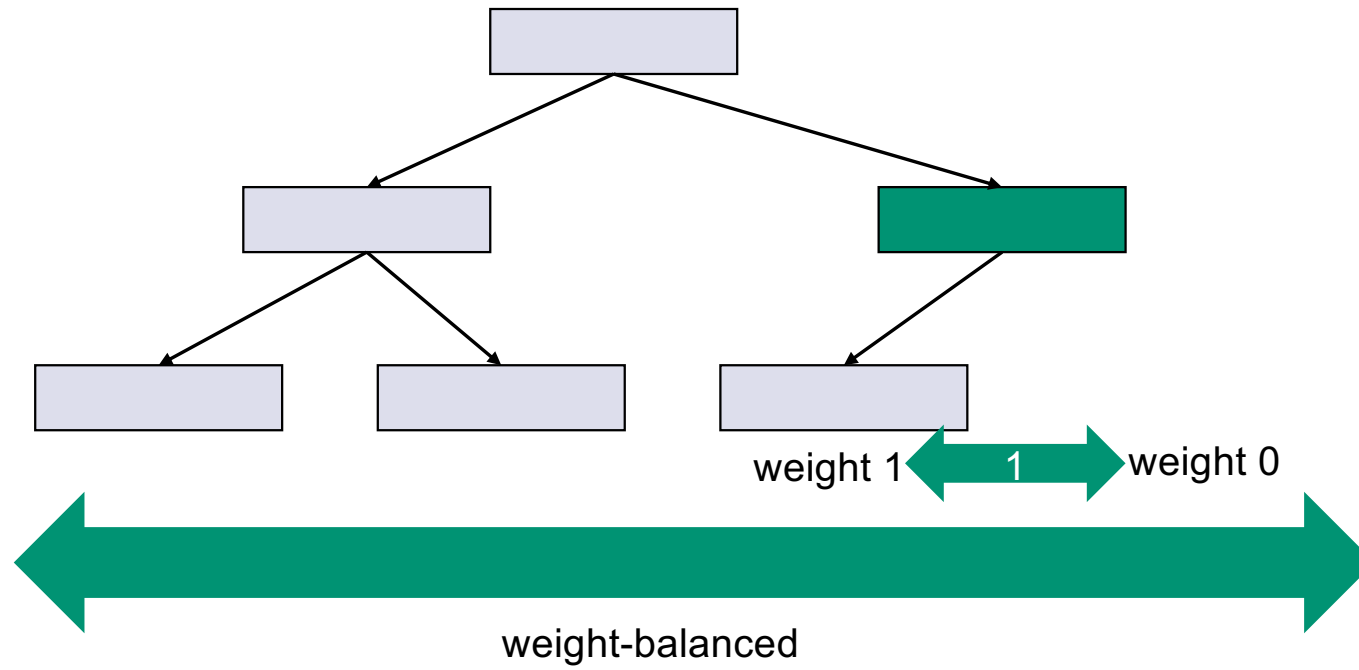
Binary tree



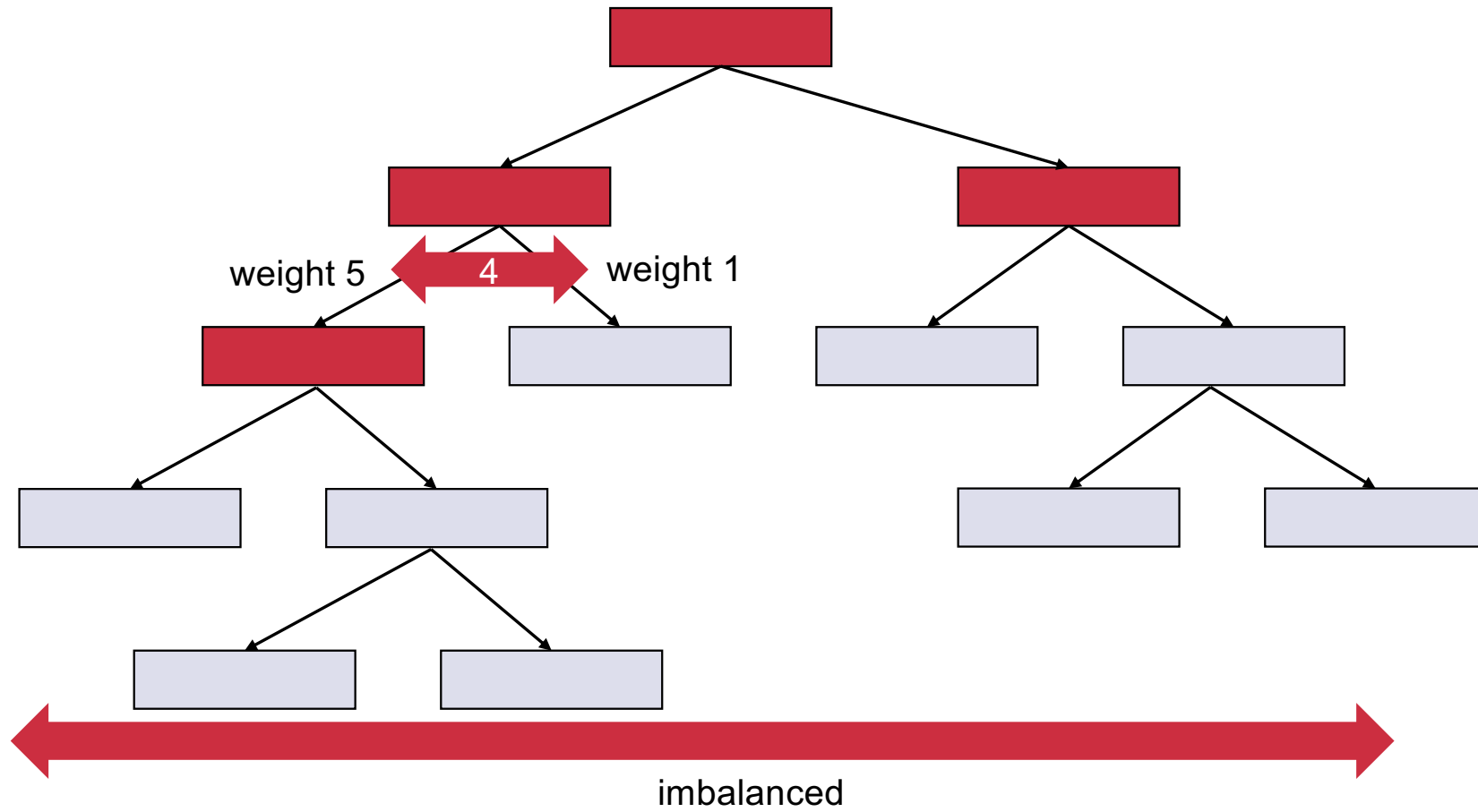
Binary tree



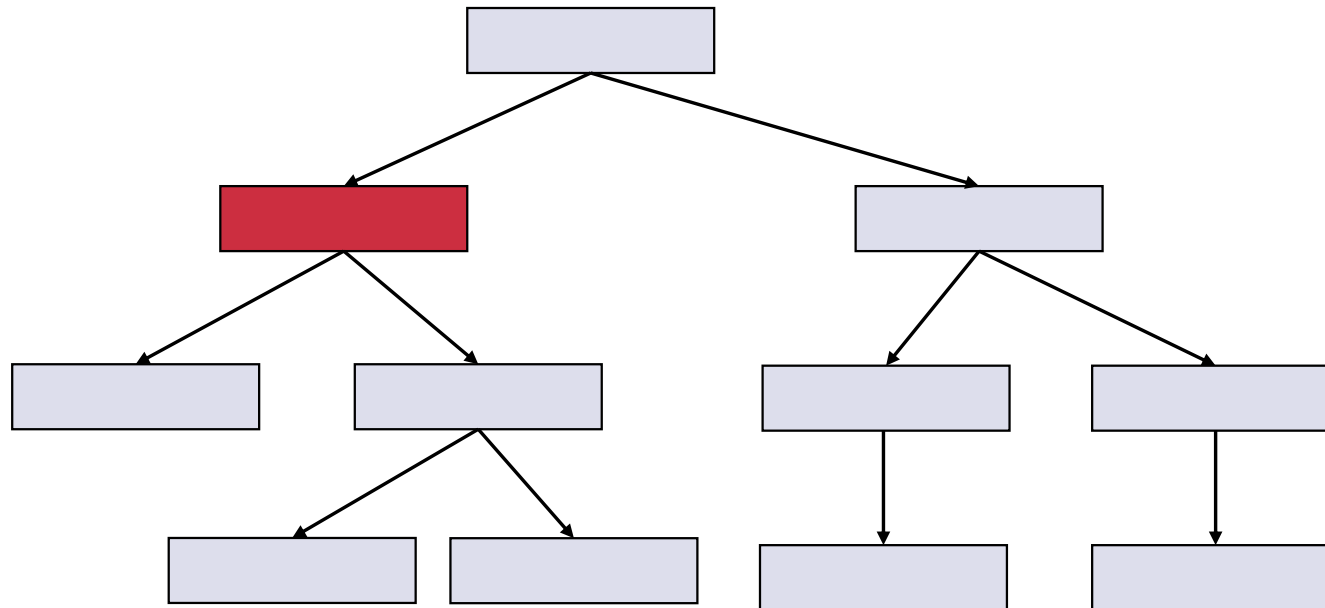
Binary tree



Binary tree

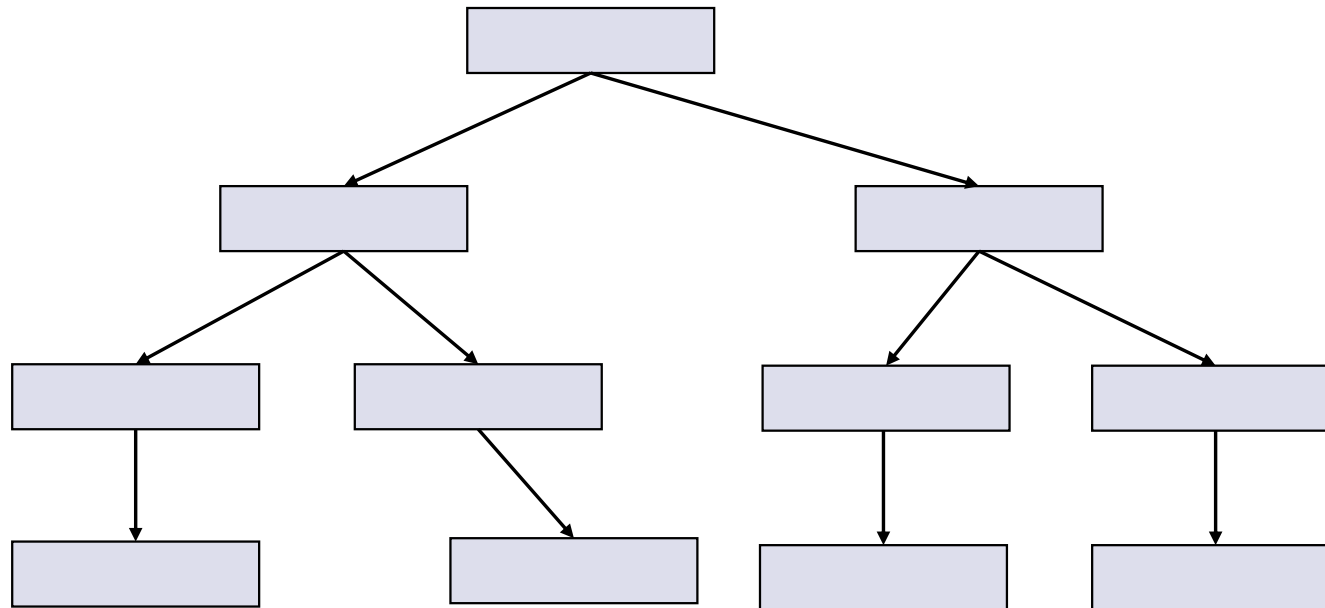


Binary tree

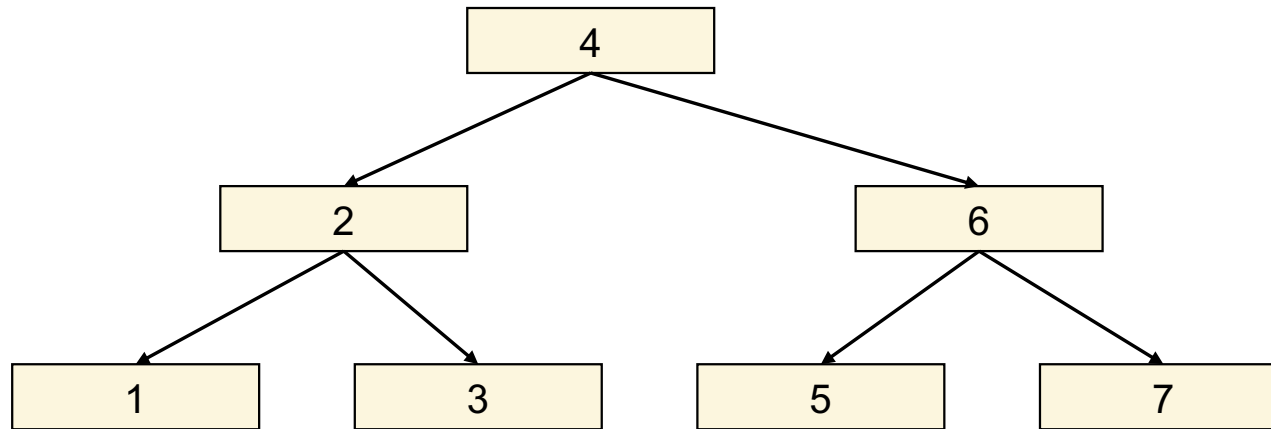


imbalanced

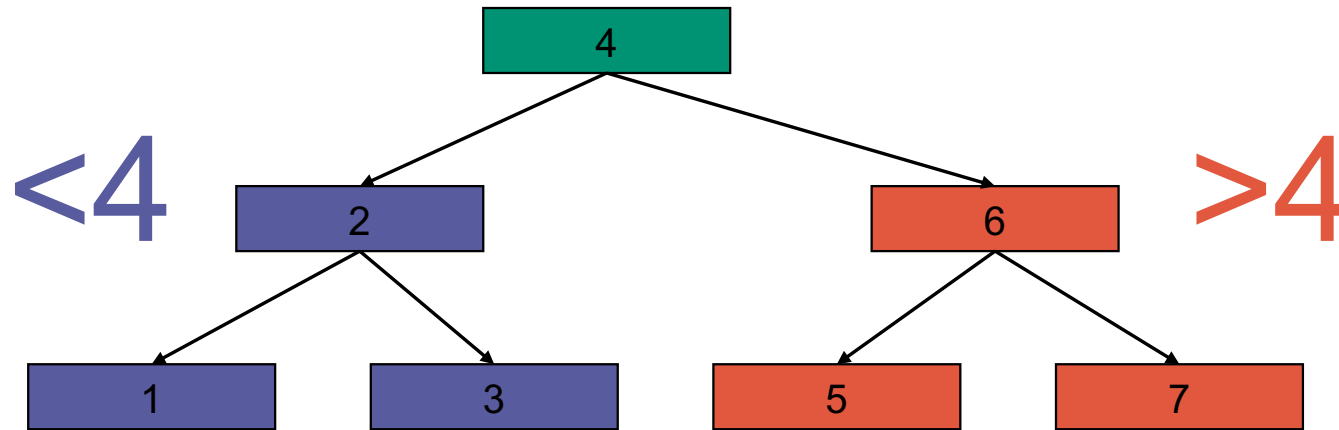
Binary tree



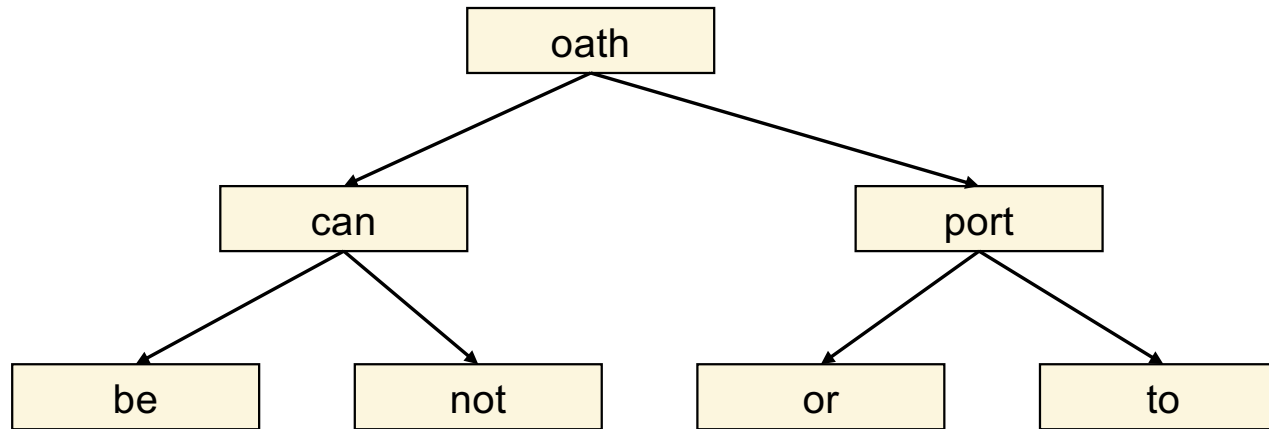
Binary search tree



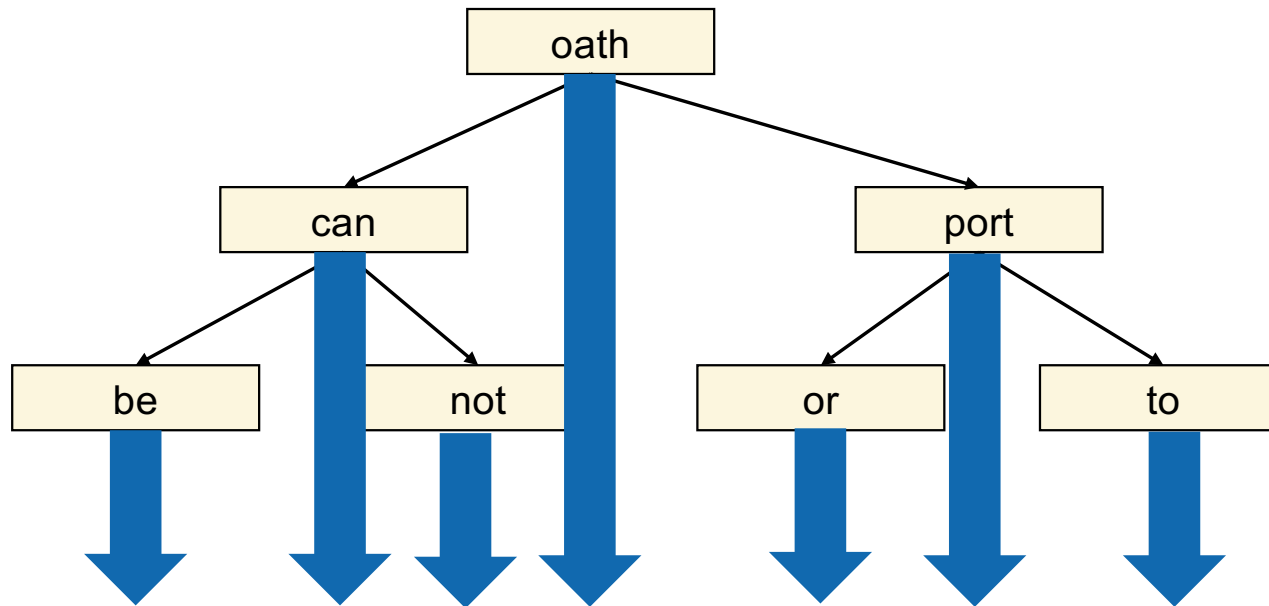
Binary search tree



Binary search tree

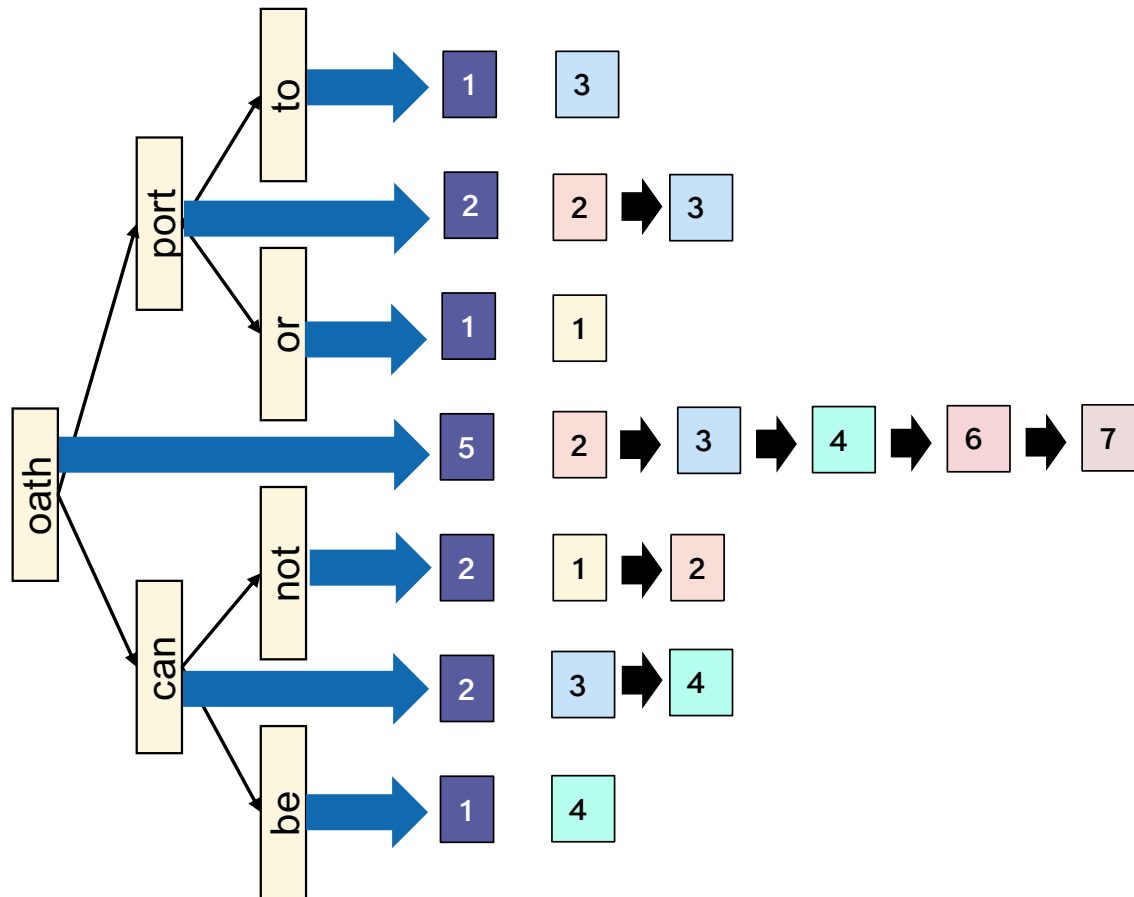


Binary search tree

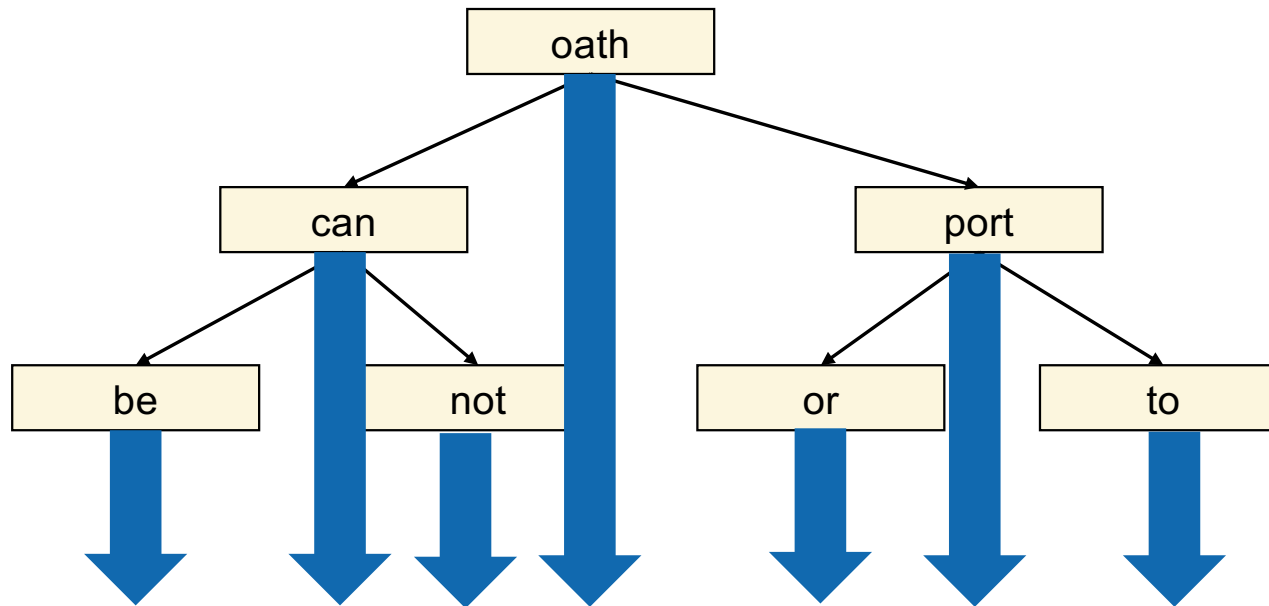


Postings lists

Other angle

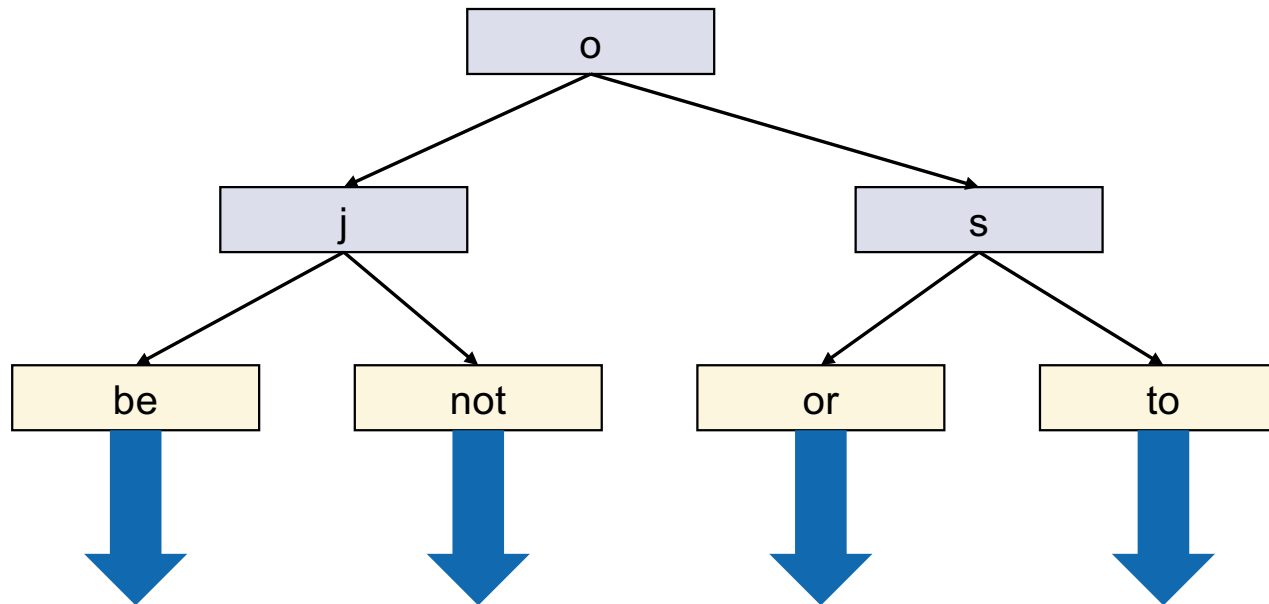


Binary search tree



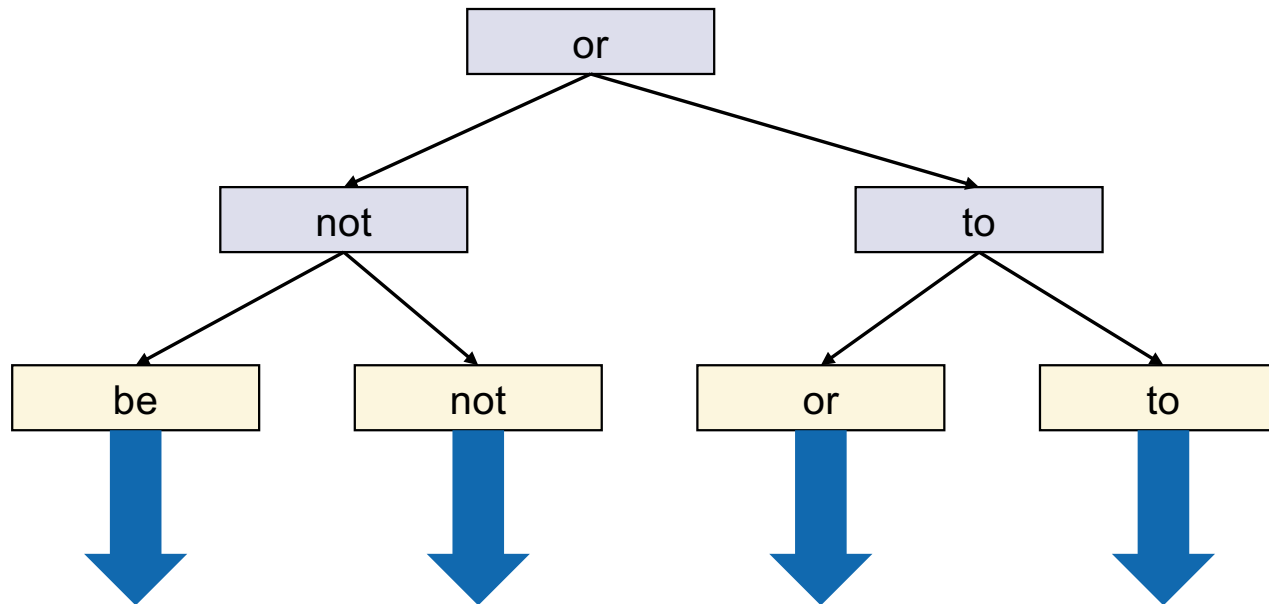
Postings lists

Binary search tree (postings lists only at leaves)



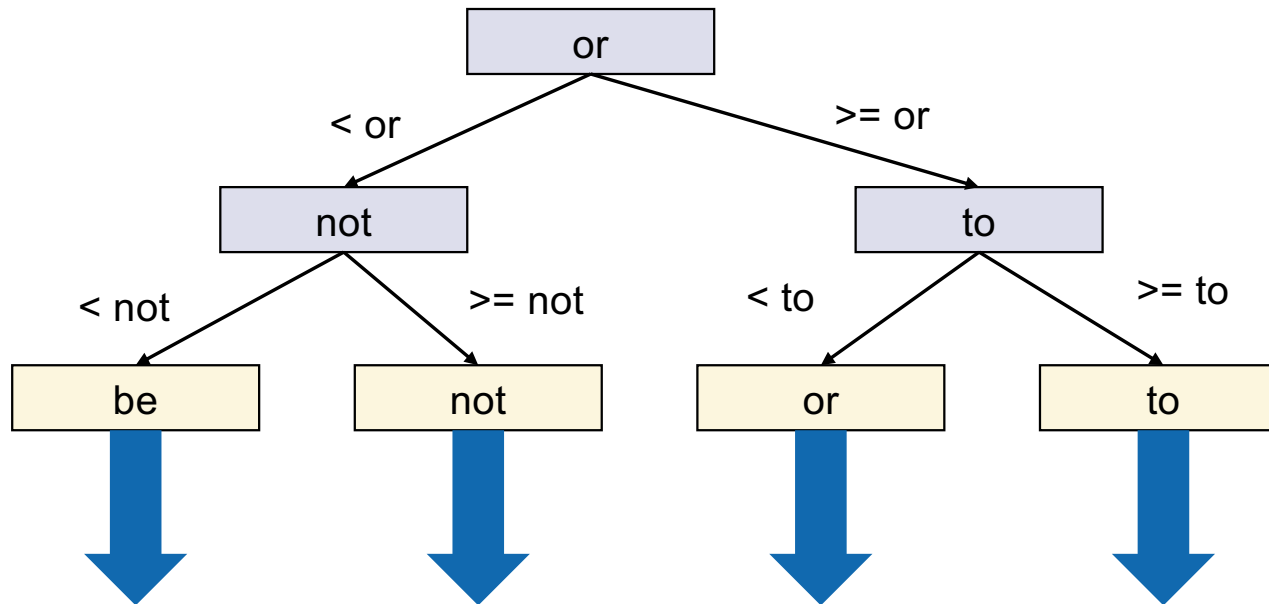
Postings lists

Design choice 2: [)-intervals



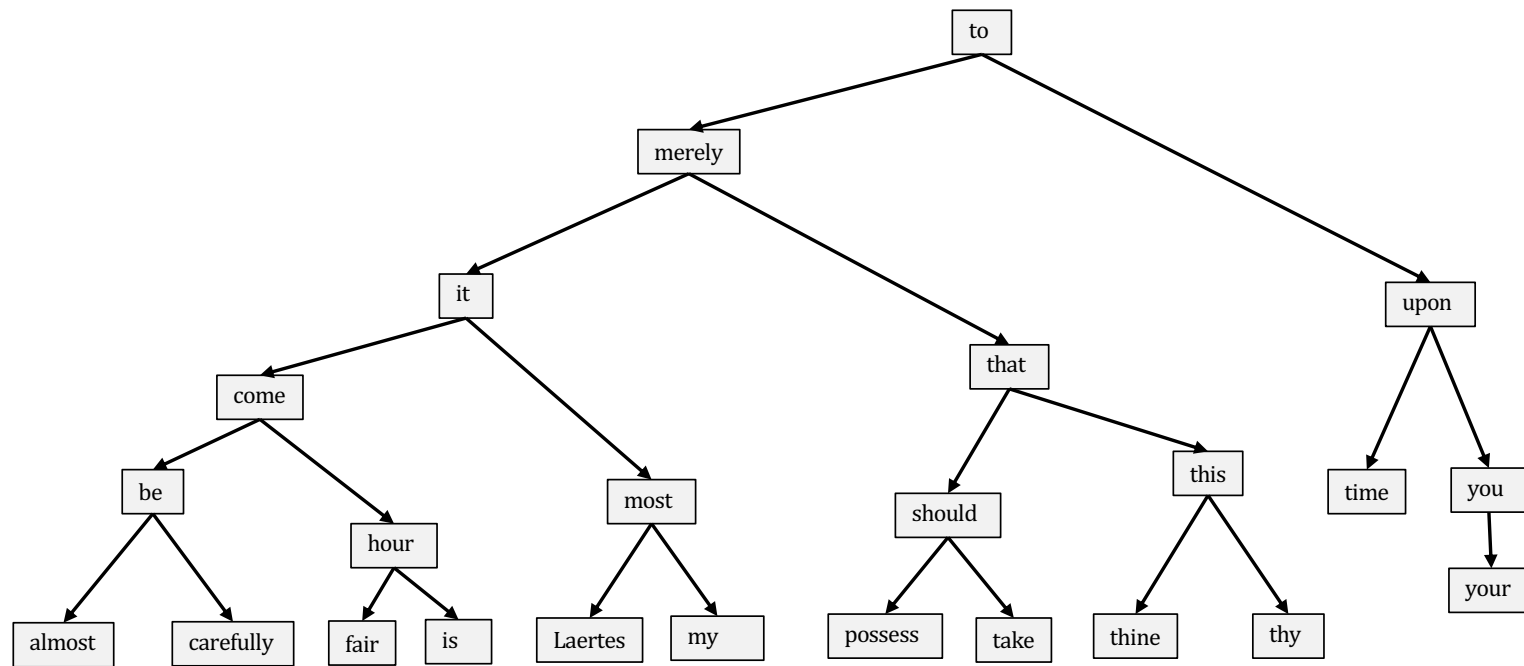
Postings lists

Design choice 2: [])-intervals

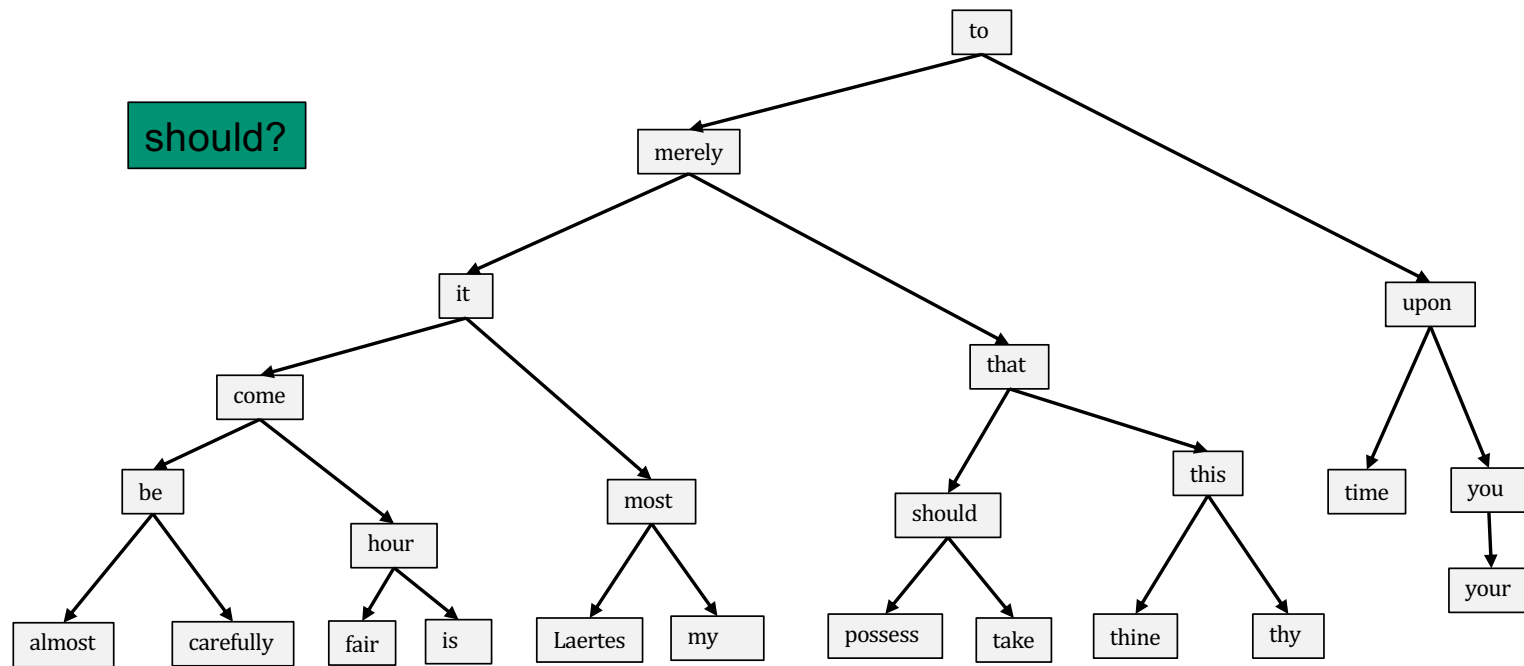


Postings lists

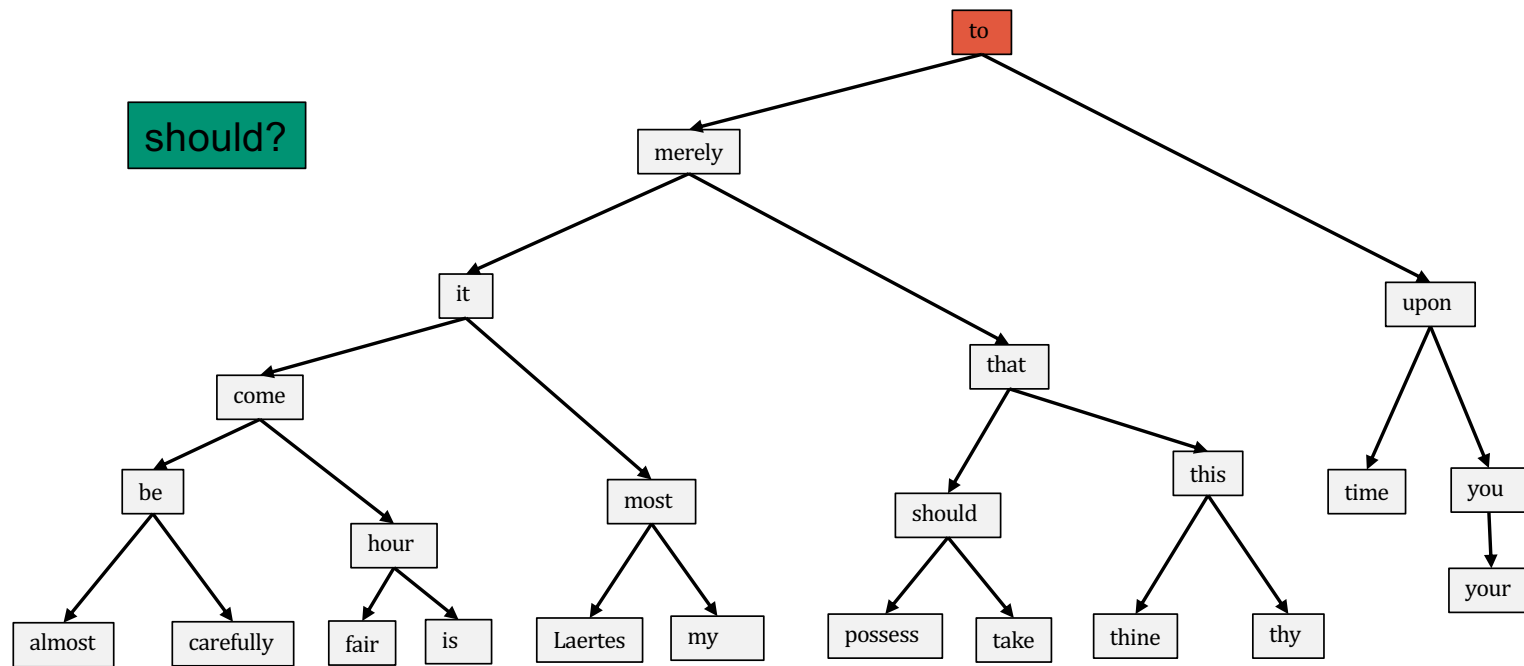
Binary search tree example



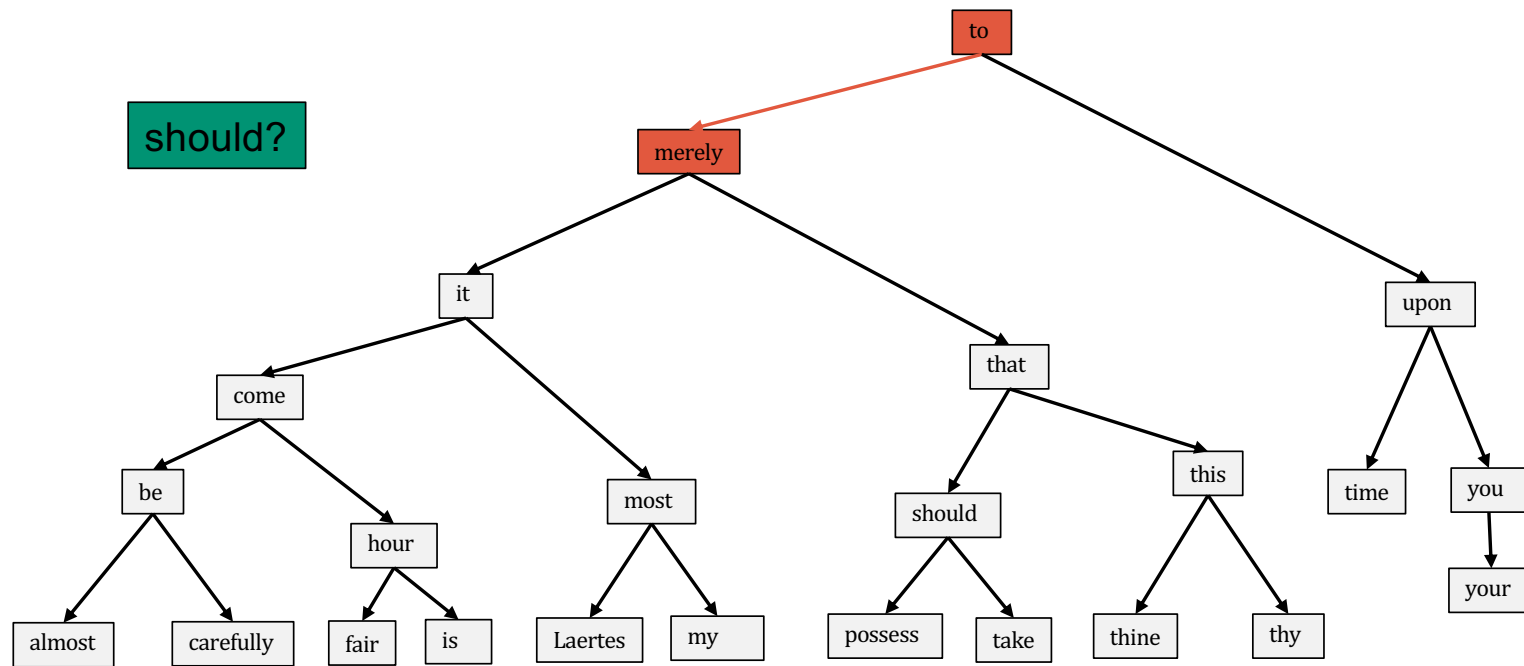
Binary tree example



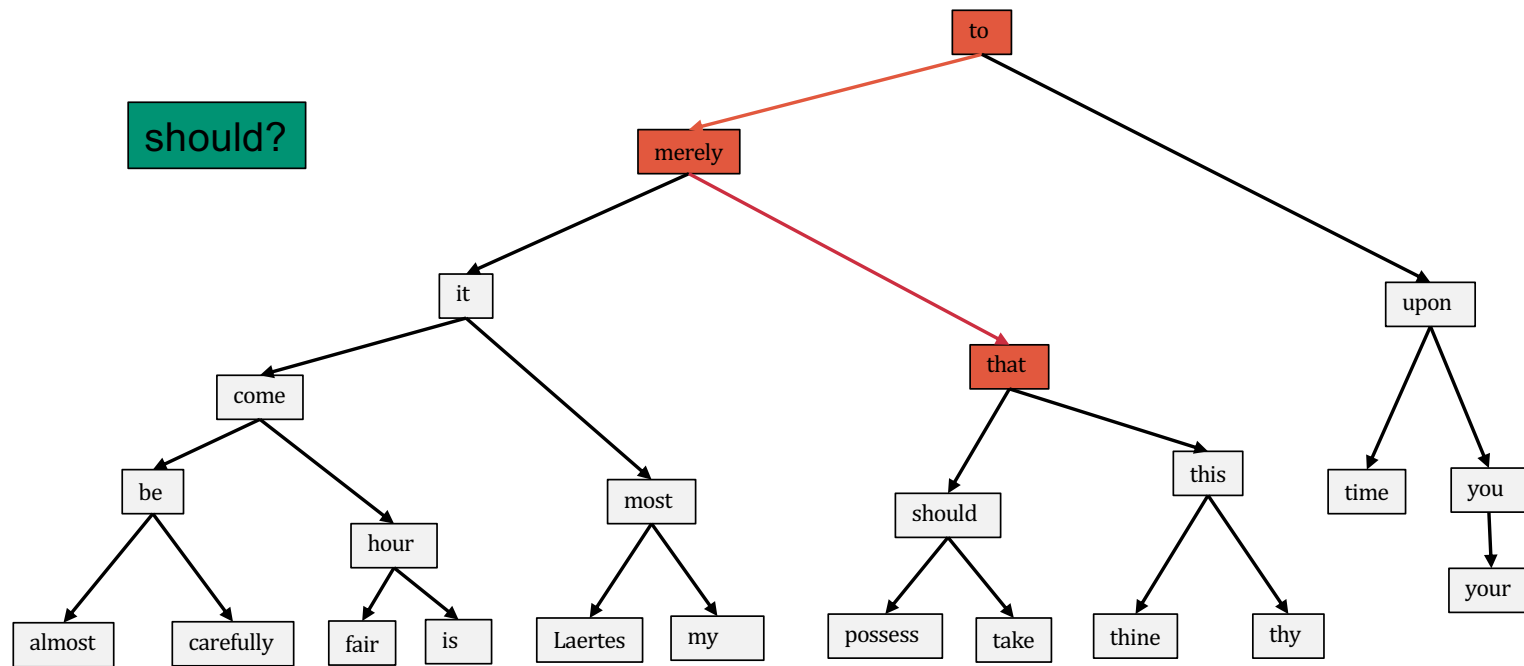
Binary tree example



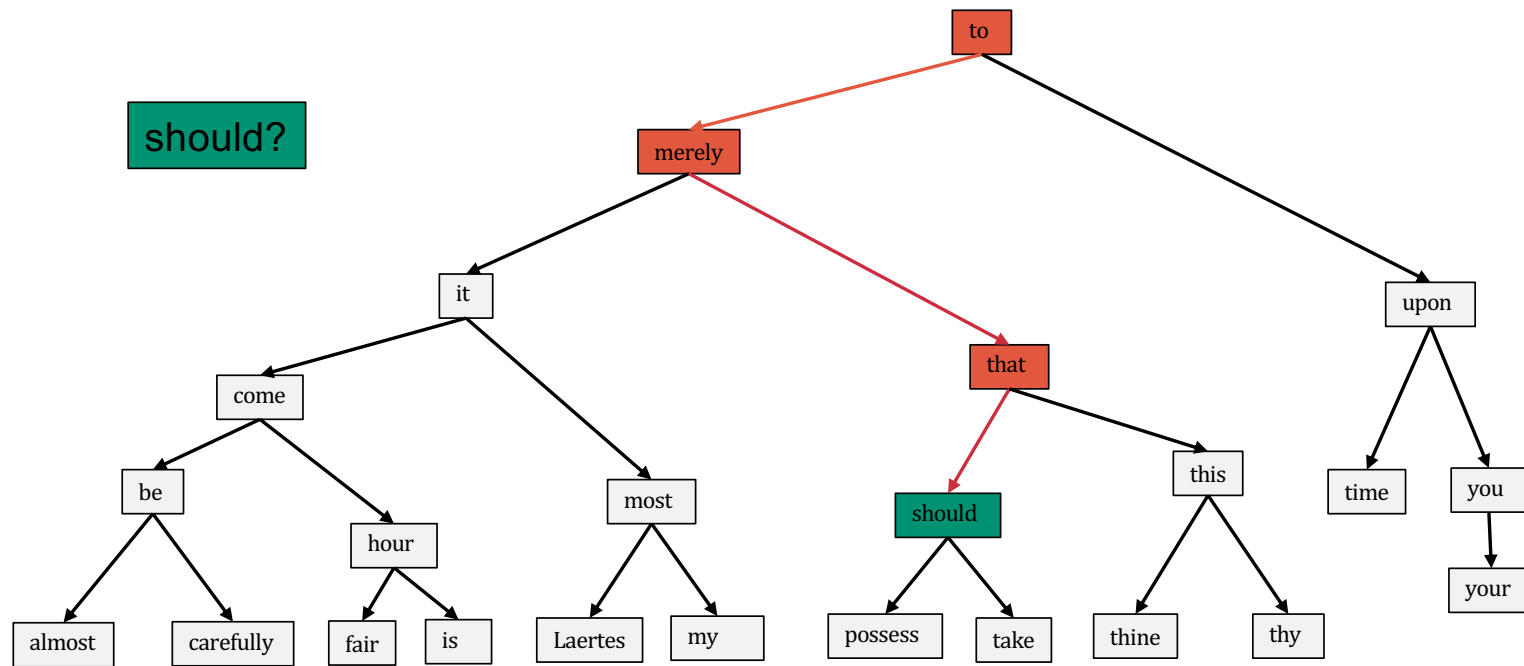
Binary tree example



Binary tree example

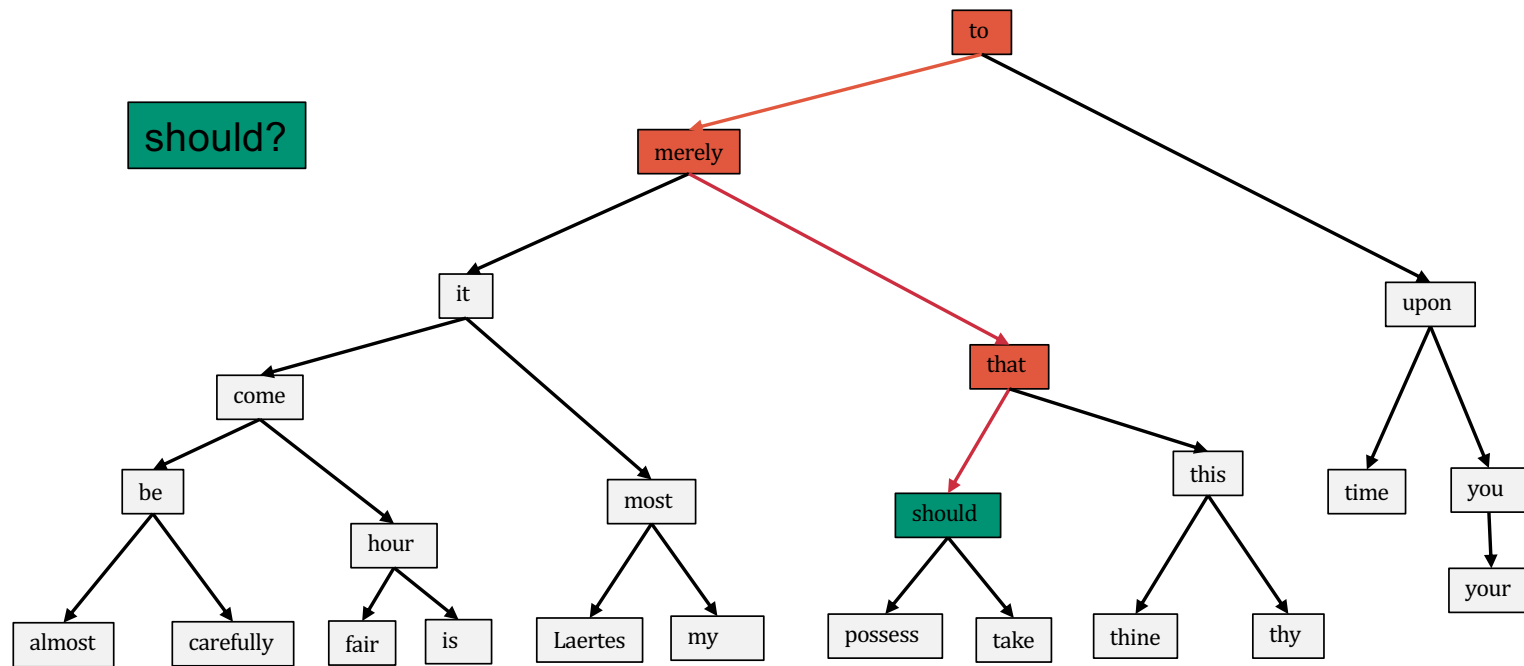


Binary tree example



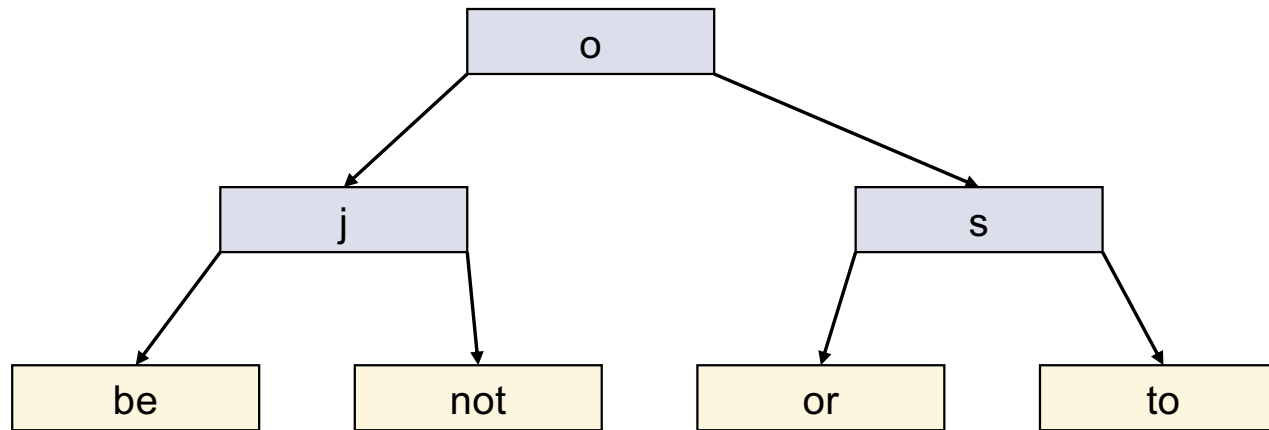
Complexity

Binary tree example

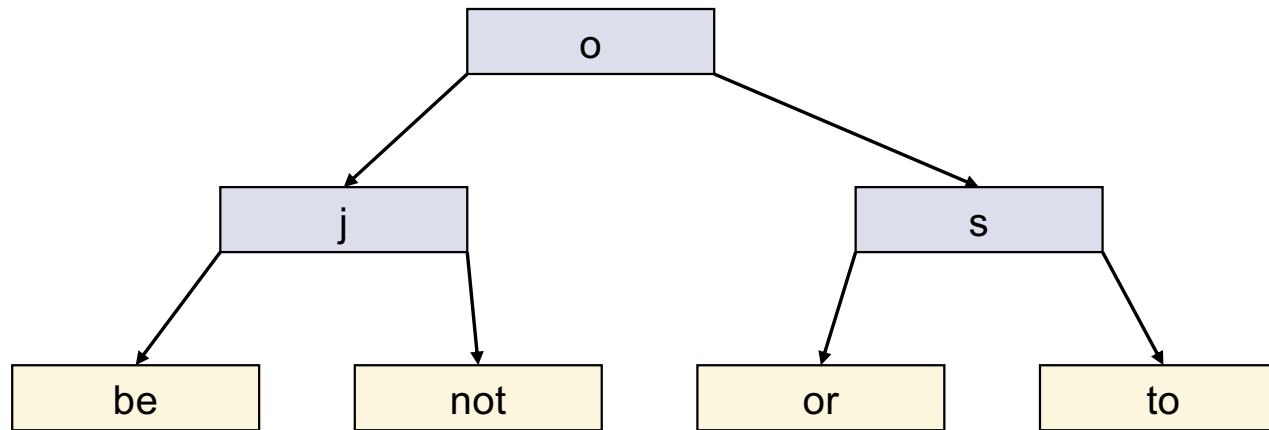


Complexity: logarithmic

B+-tree

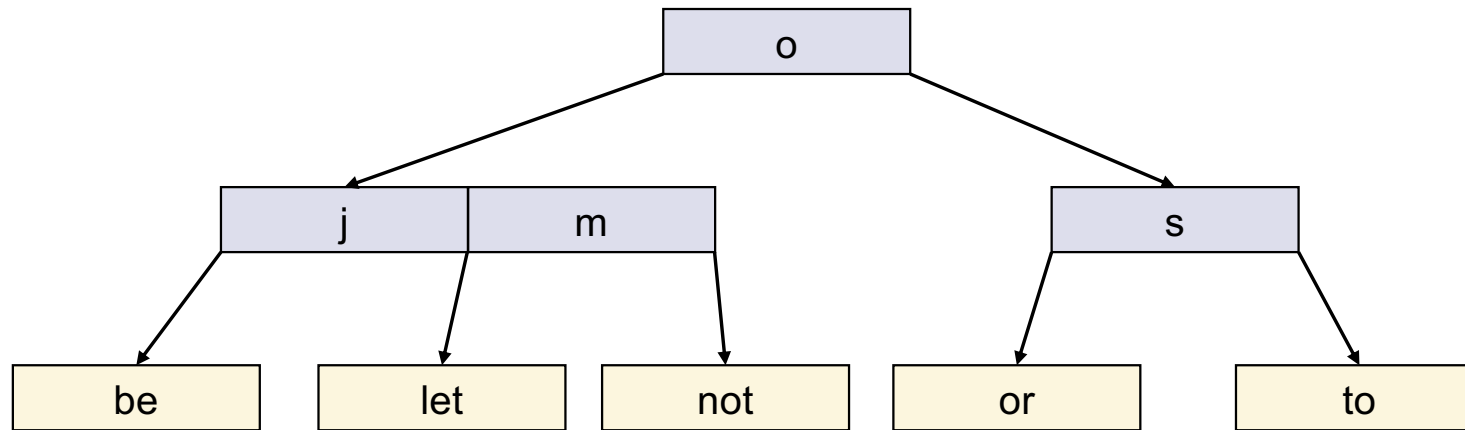


B+-tree

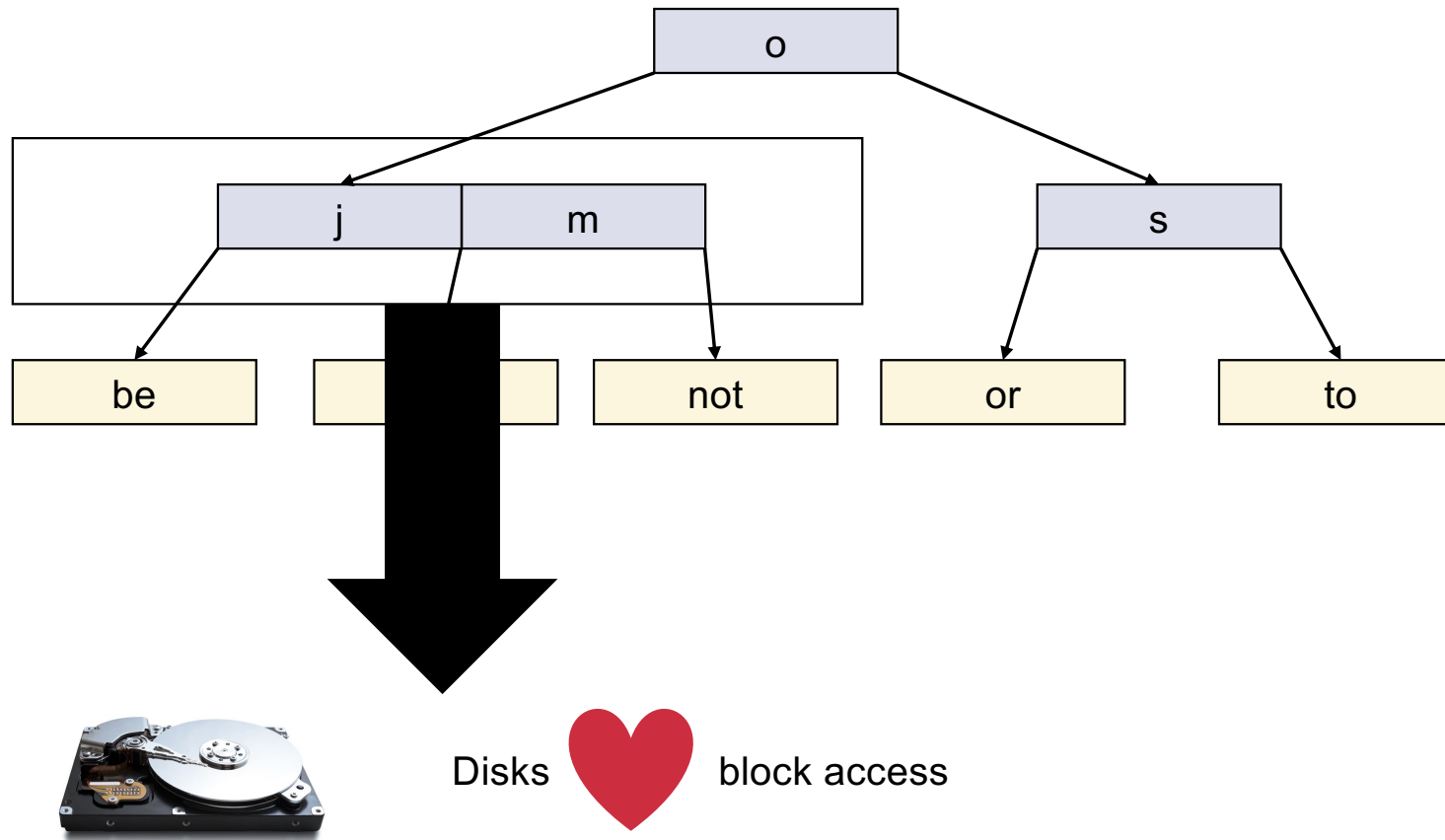


We could allow more children

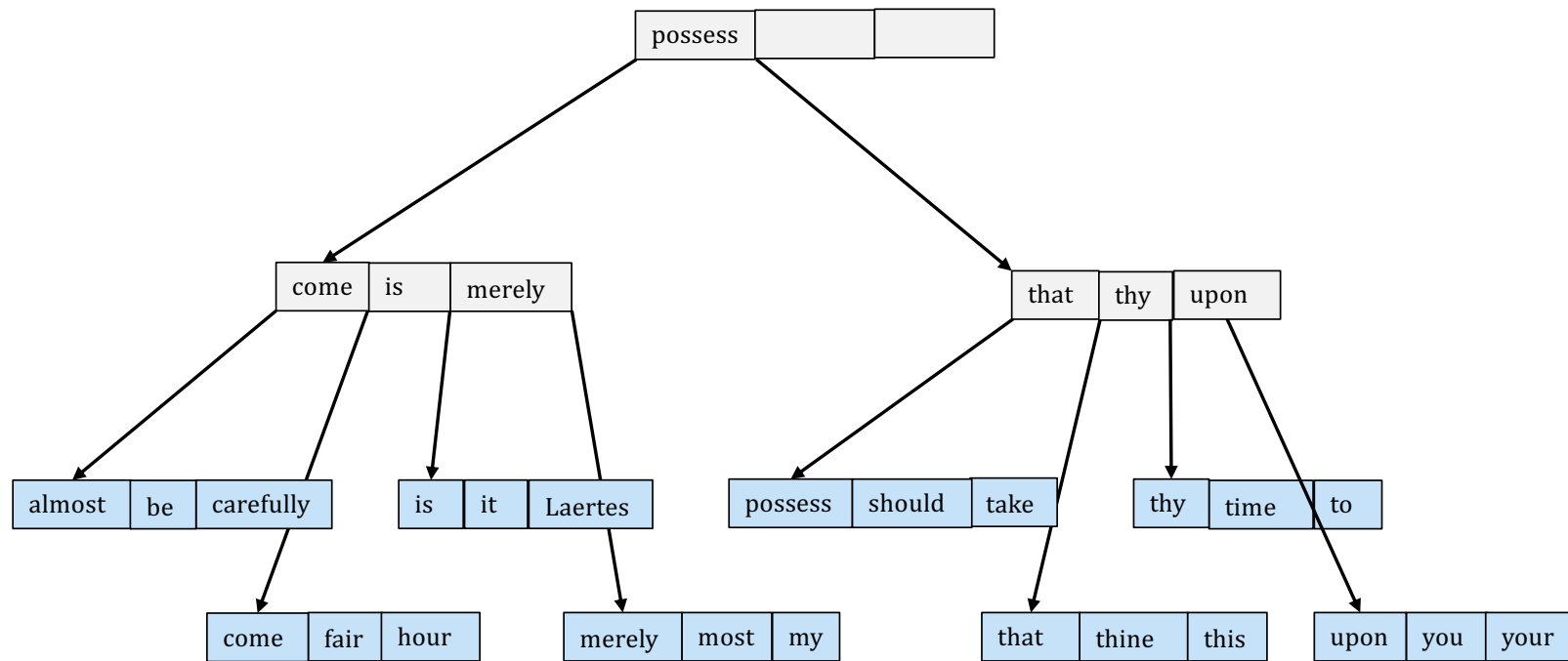
B+-tree



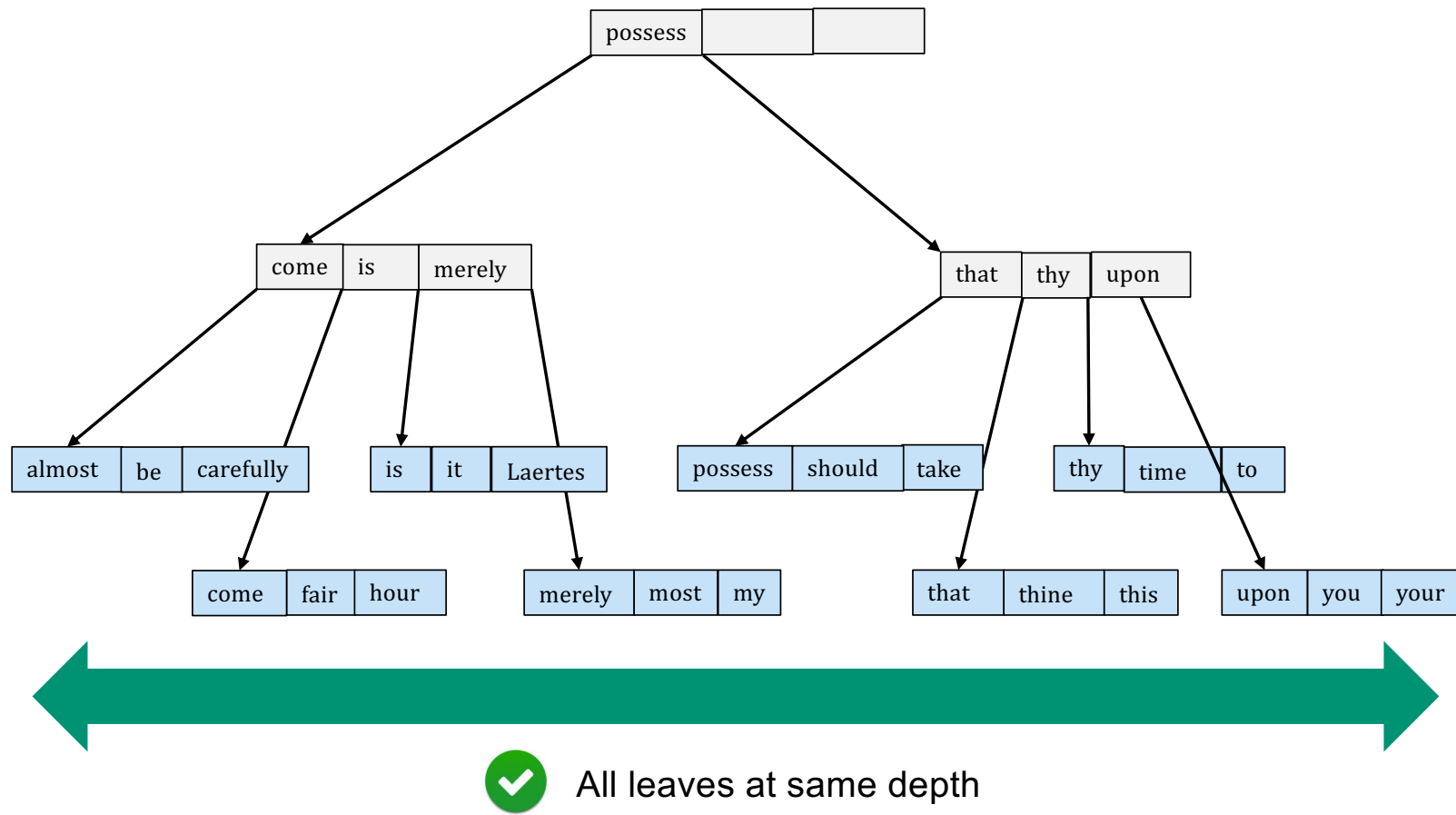
B+-tree



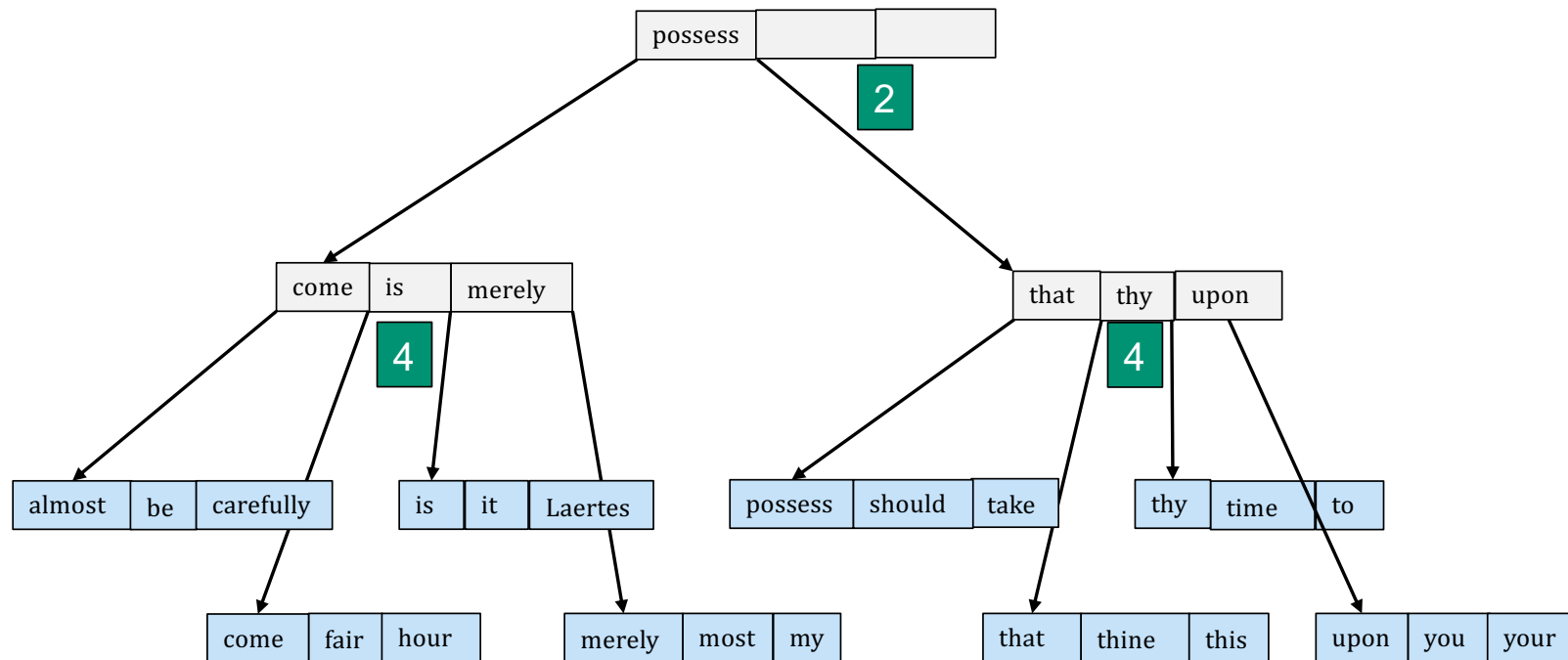
B+-tree example



B+-tree example

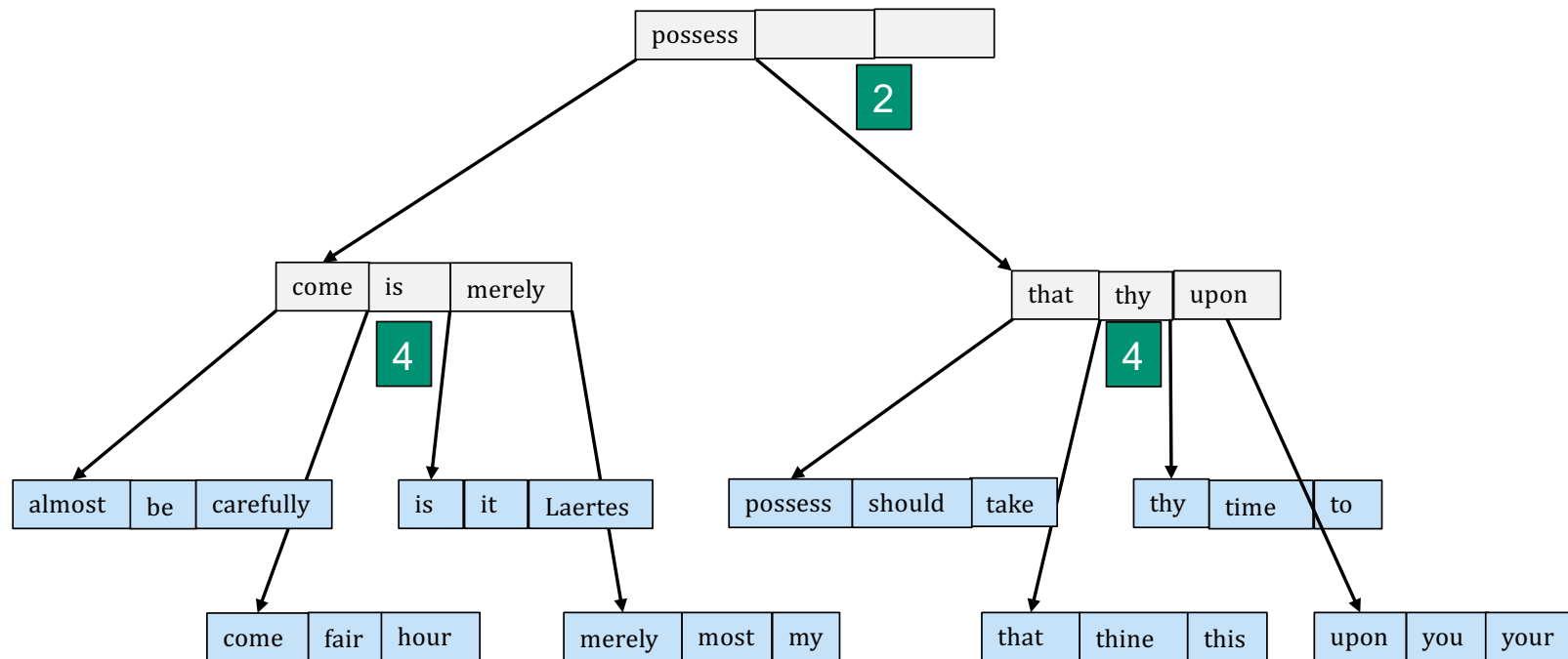


B+-tree example



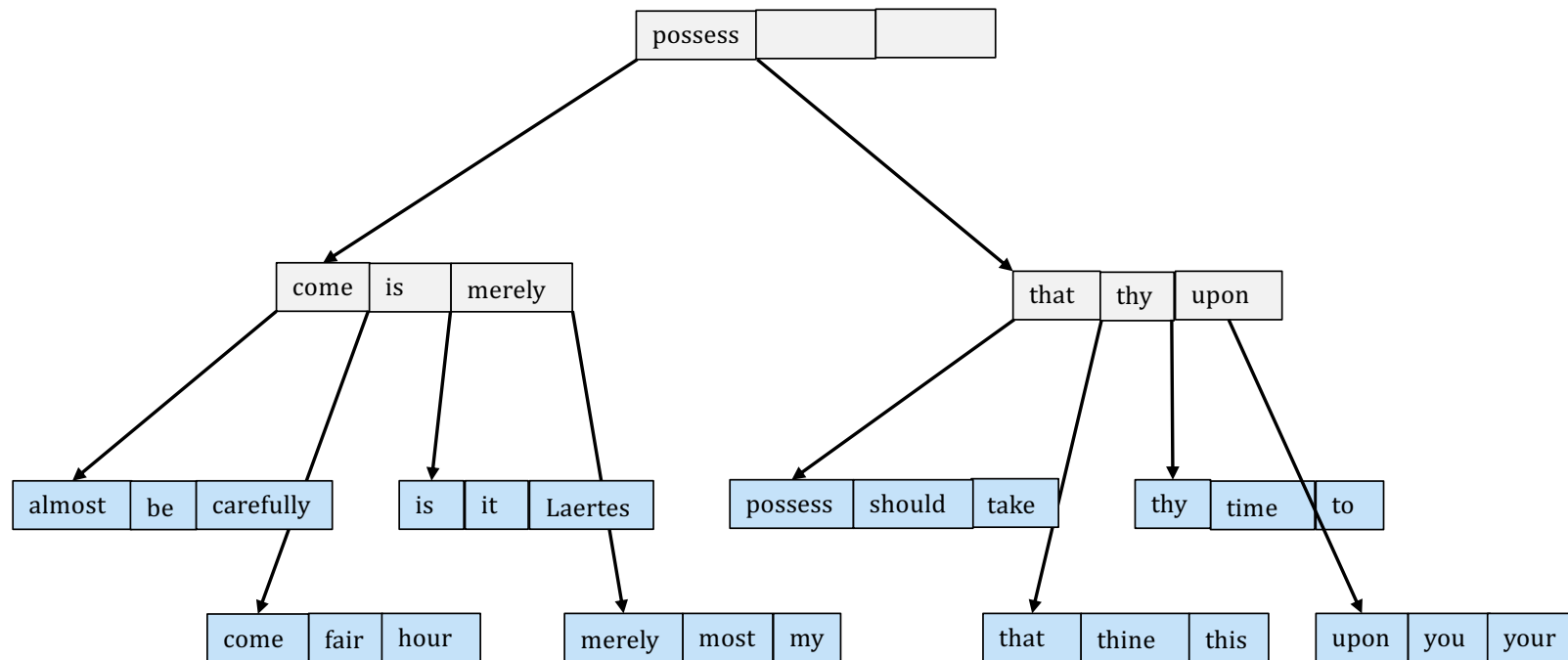
All non-leaf nodes have between 3 and 5 children

B+-tree example



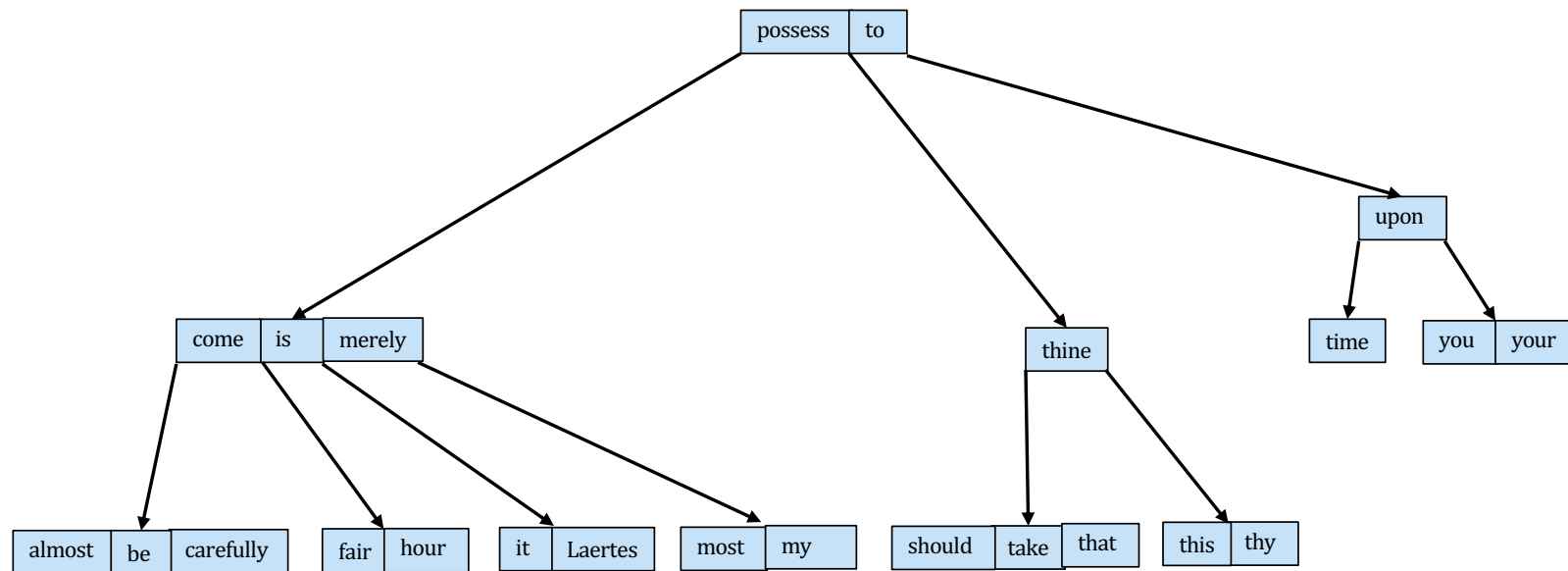
✓ But it's fine if the root has less.

B+-tree example



Actual terms (and posting lists) only at the leaves

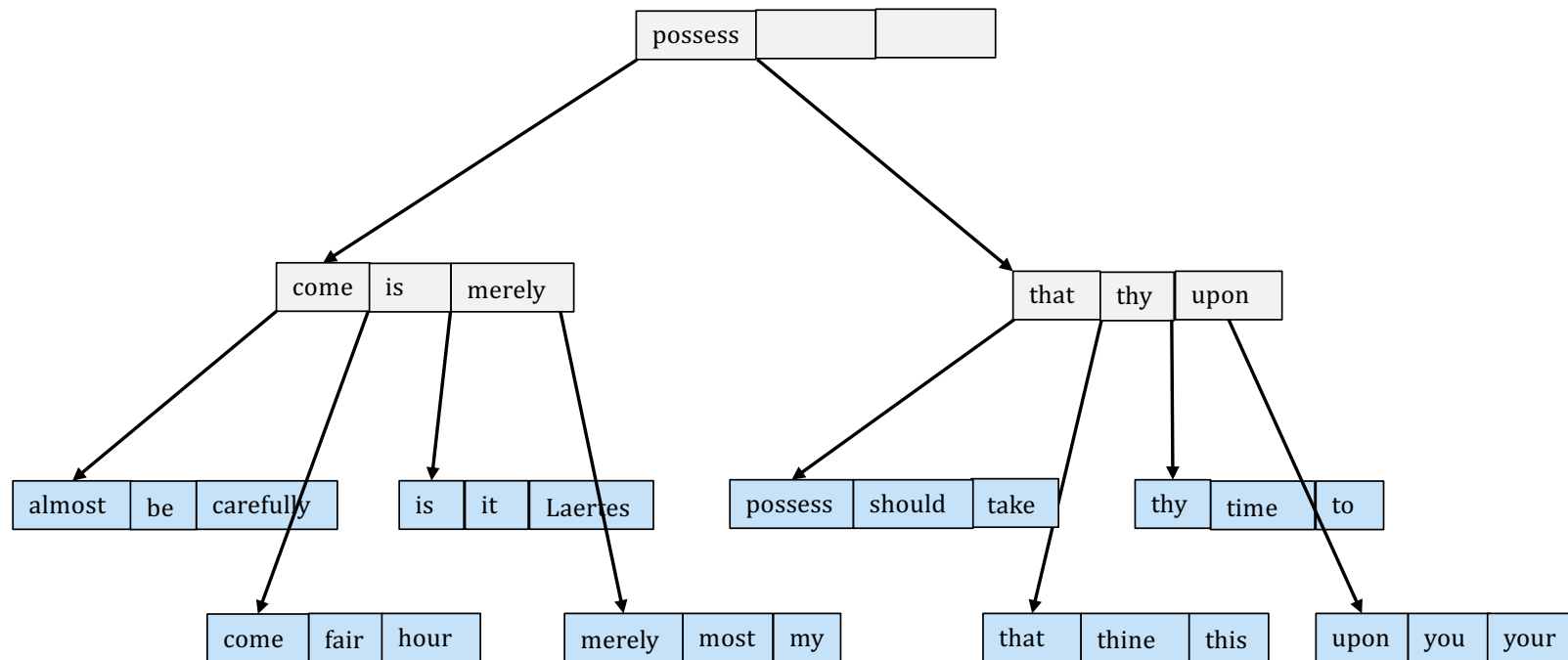
B-tree example



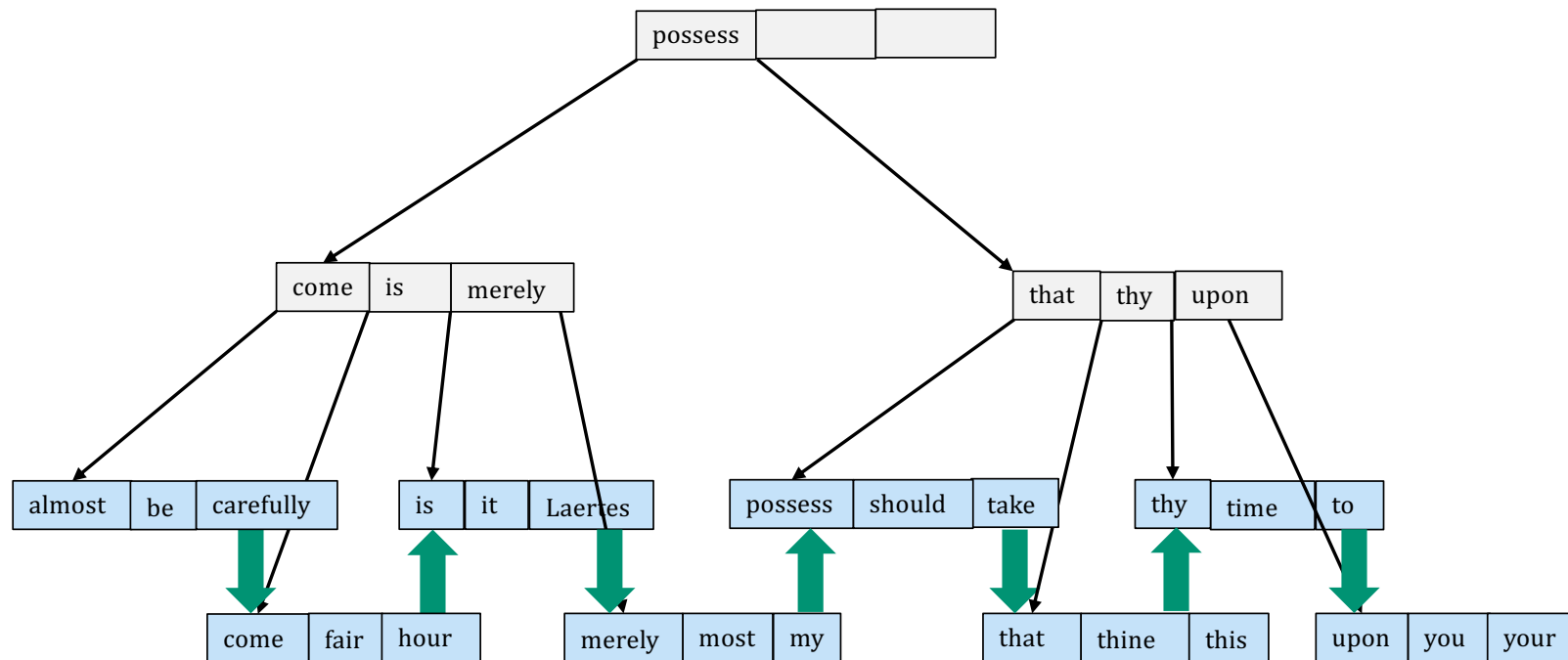
2-4 B-tree

Children per node

B+-trees often have extra leaf pointers



B+-trees often have extra leaf pointers



B+-tree example

General case: #children between $d+1$ and $2d+1$

B+-tree example

General case: #children between $d+1$ and $2d+1$

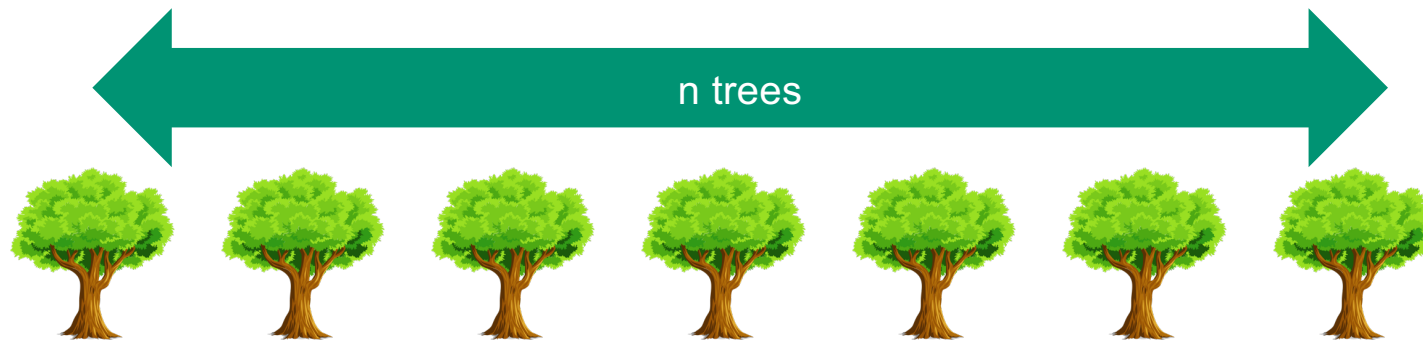
1-1 B+-tree

2-3 B+-tree

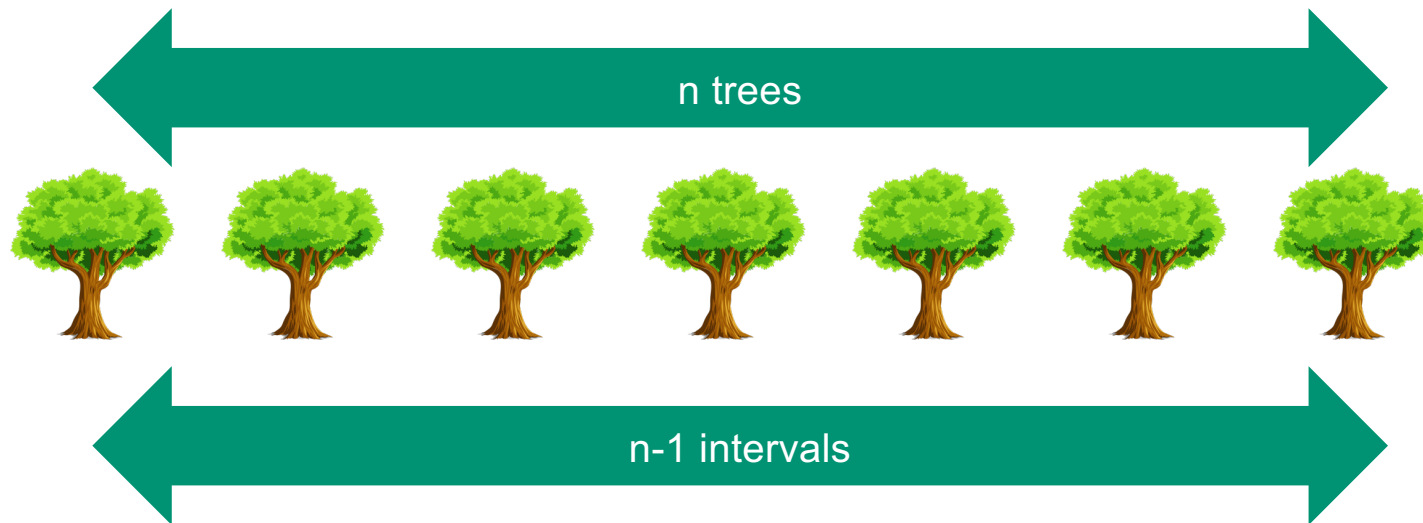
3-5 B+-tree

4-7 B+-tree

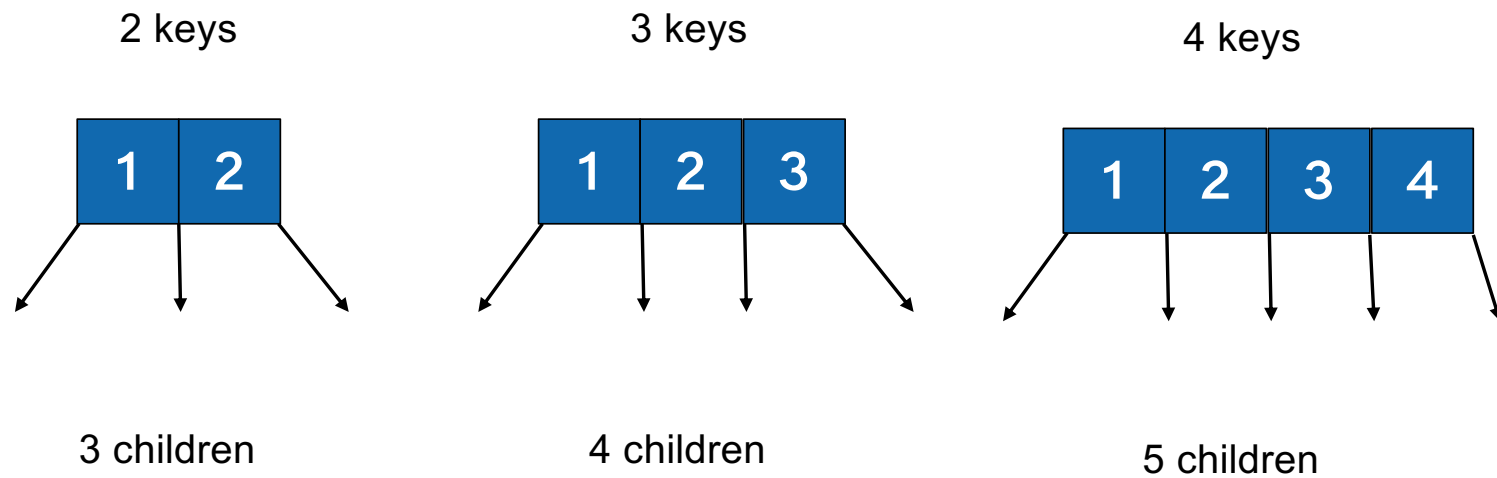
Warning: intervals!



Warning: intervals!



Warning: intervals!



B+-tree example

General case: #children between $d+1$ and $2d+1$

1-1 B+-tree

2-3 B+-tree

3-5 B+-tree

4-7 B+-tree

(means #keys between d and $2d$)

B+-tree example

General case: #children between $d+1$ and $2d+1$

1-1 B+-tree

2-3 B+-tree

3-5 B+-tree

4-7 B+-tree

(means #keys between d and $2d$)

Insertion

1

Insertion



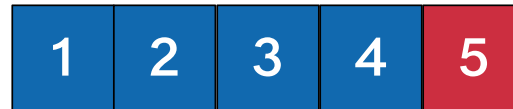
Insertion

1	2	3
---	---	---

Insertion

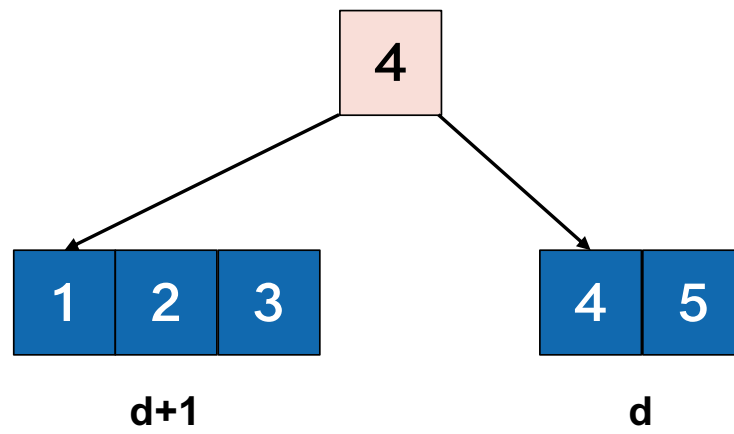


Insertion

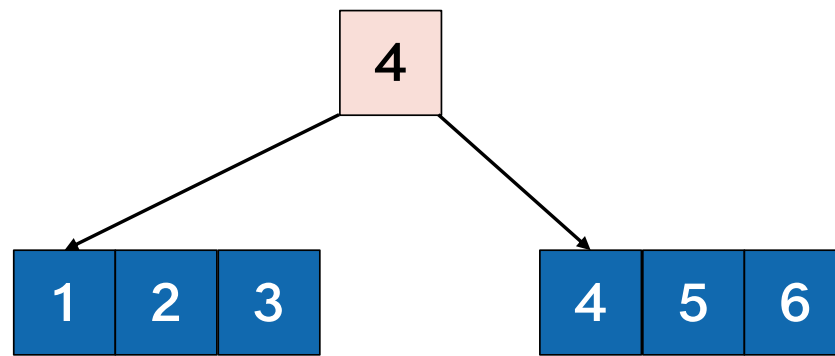


More than 2d keys!

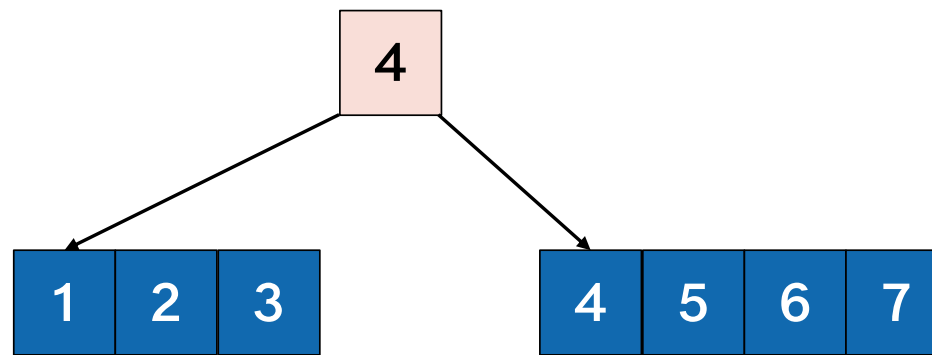
Insertion



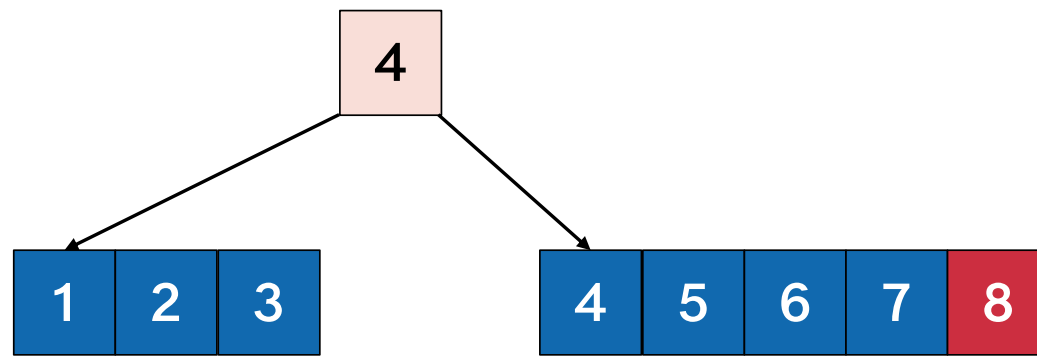
Insertion



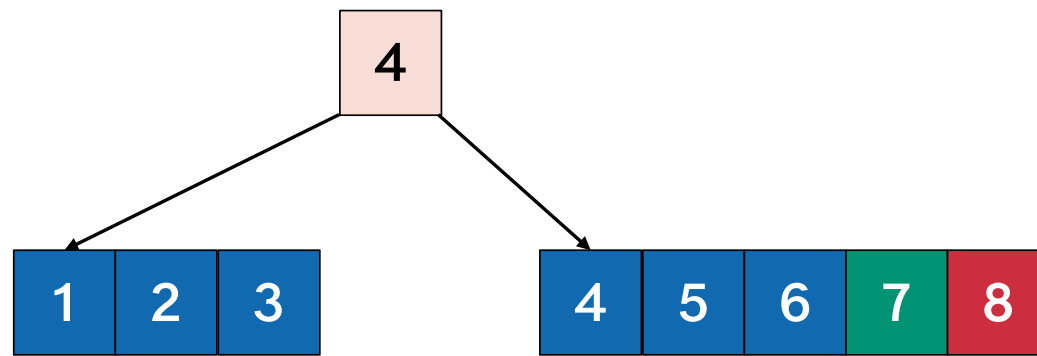
Insertion



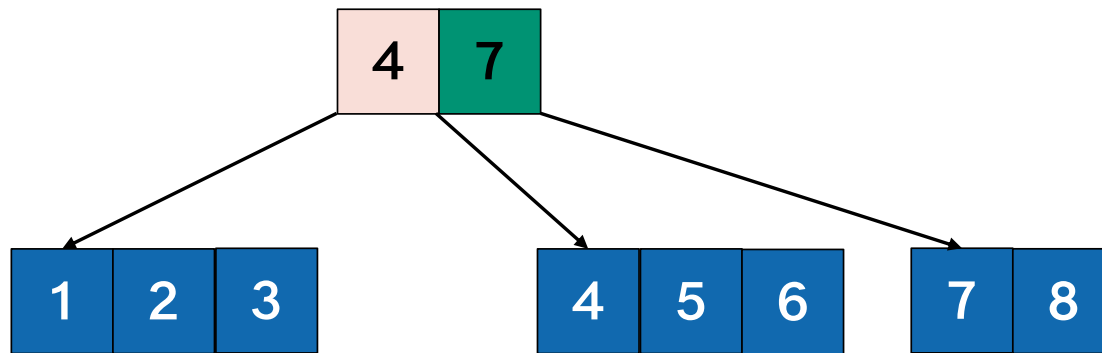
Insertion



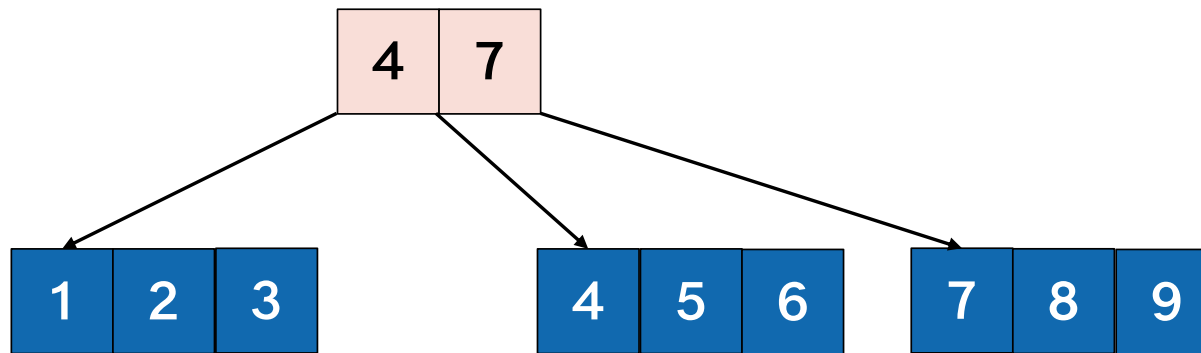
Insertion



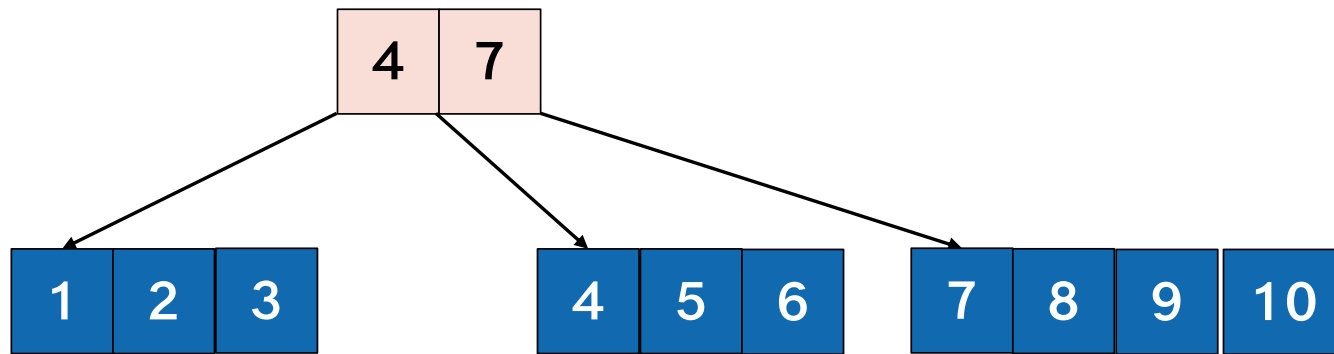
Insertion



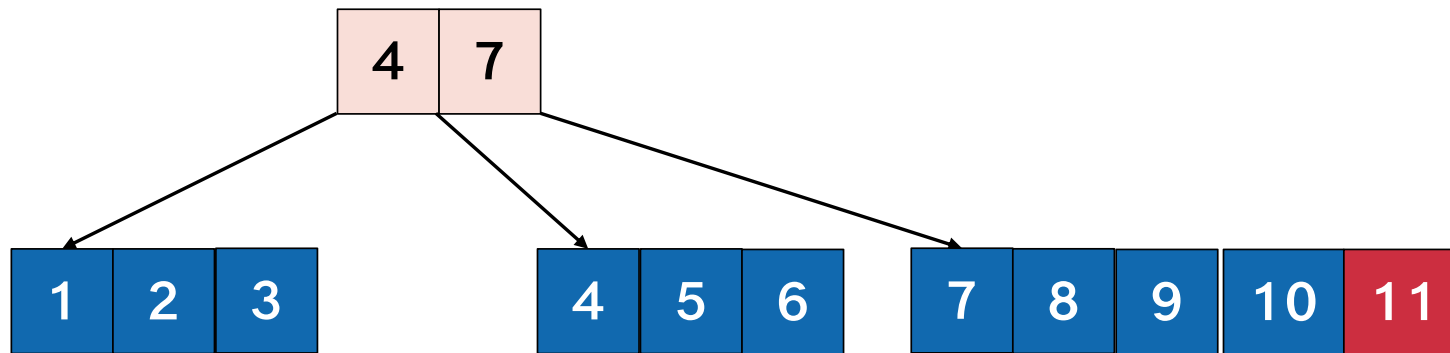
Insertion



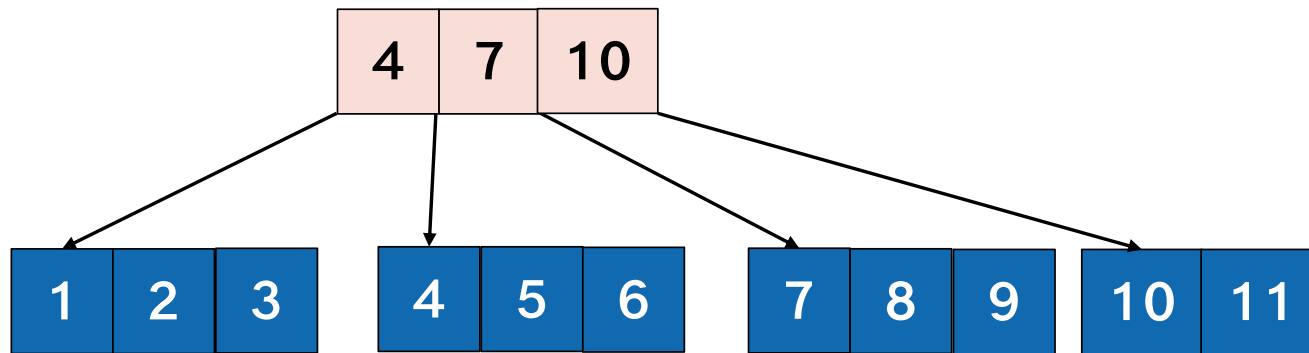
Insertion



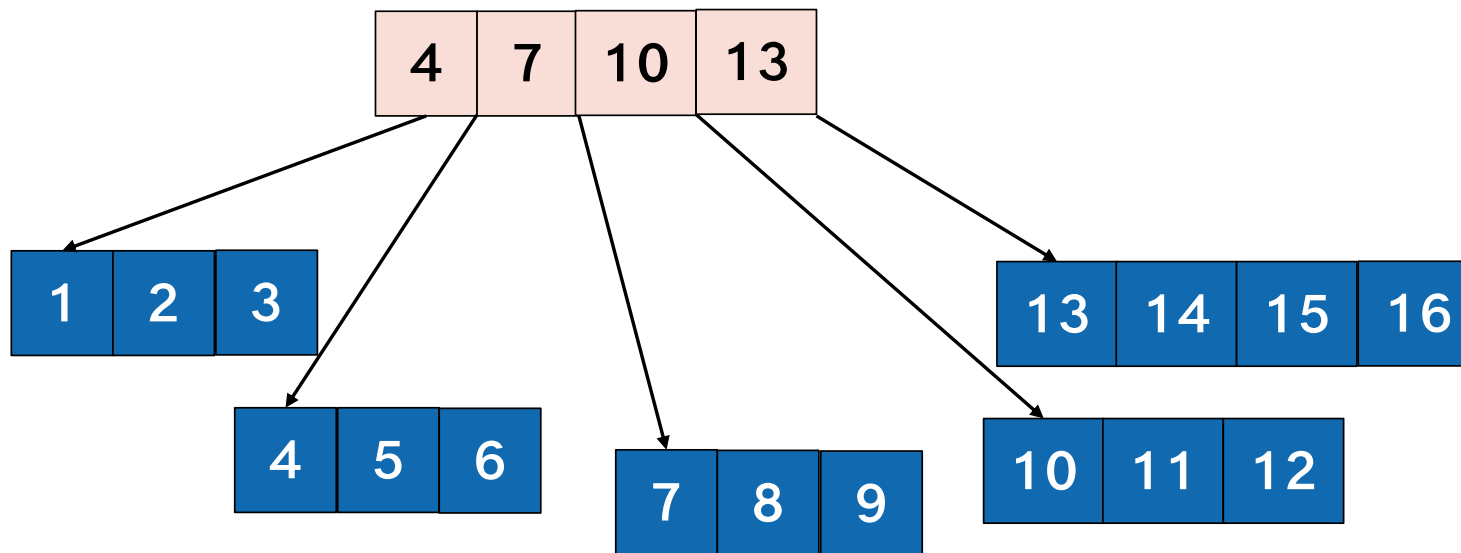
Insertion



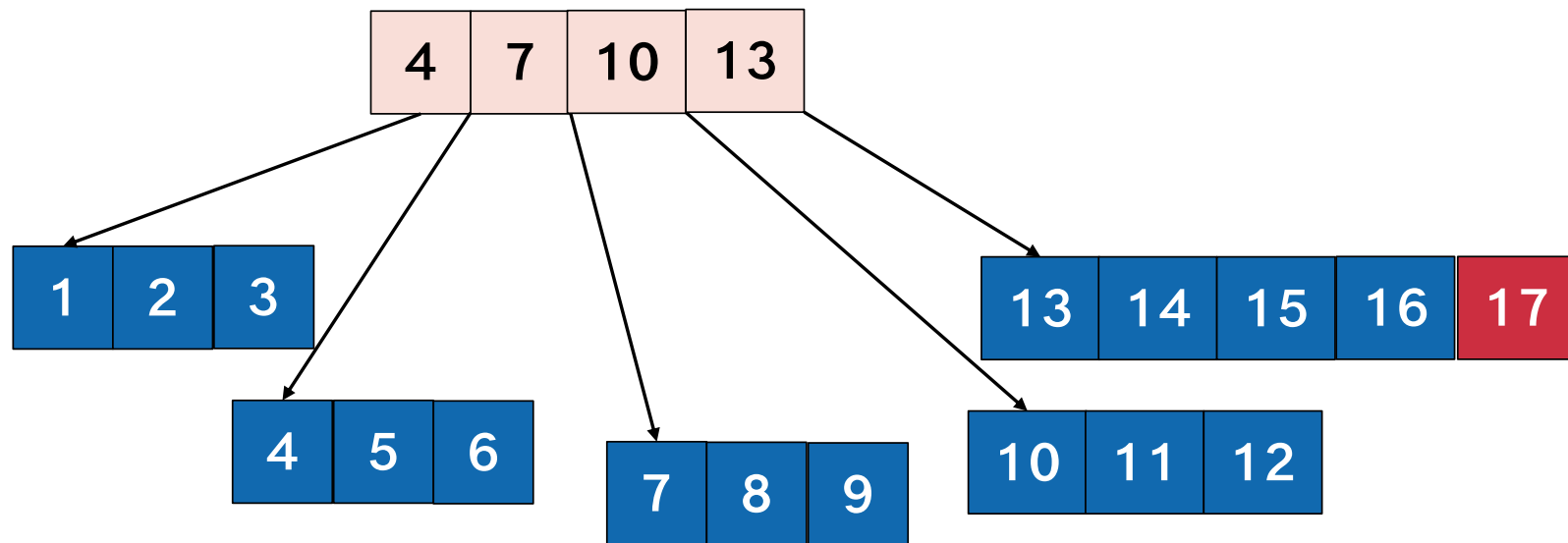
Insertion



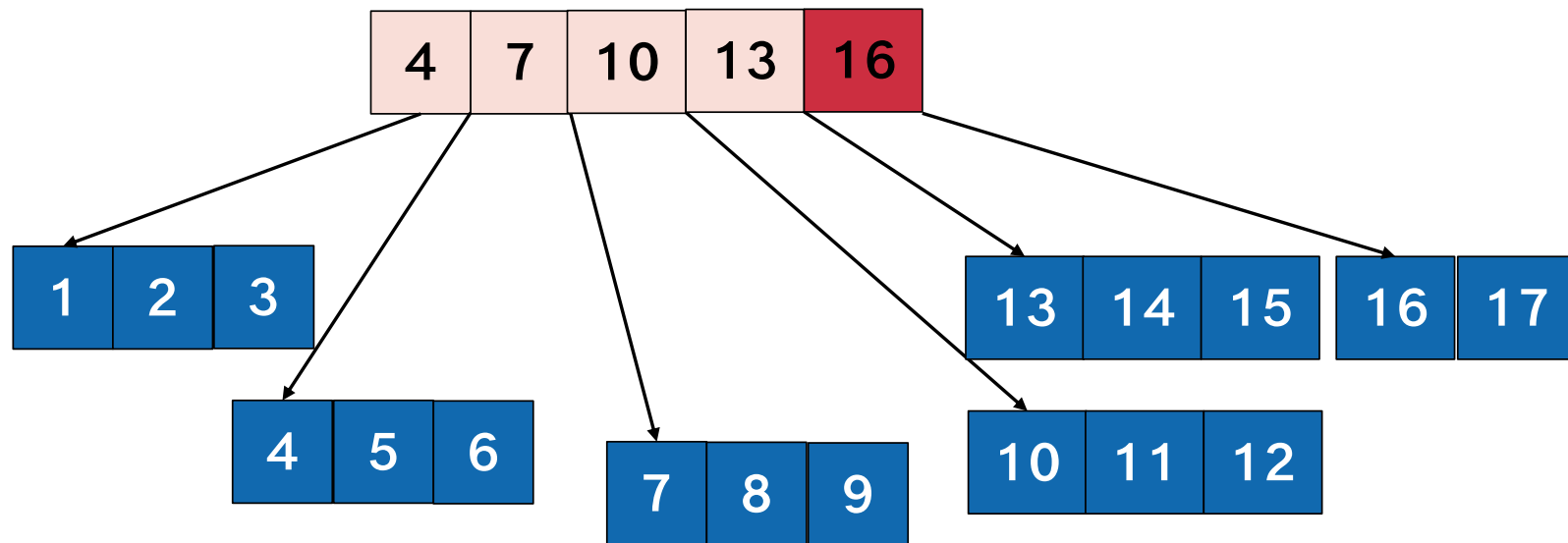
Insertion



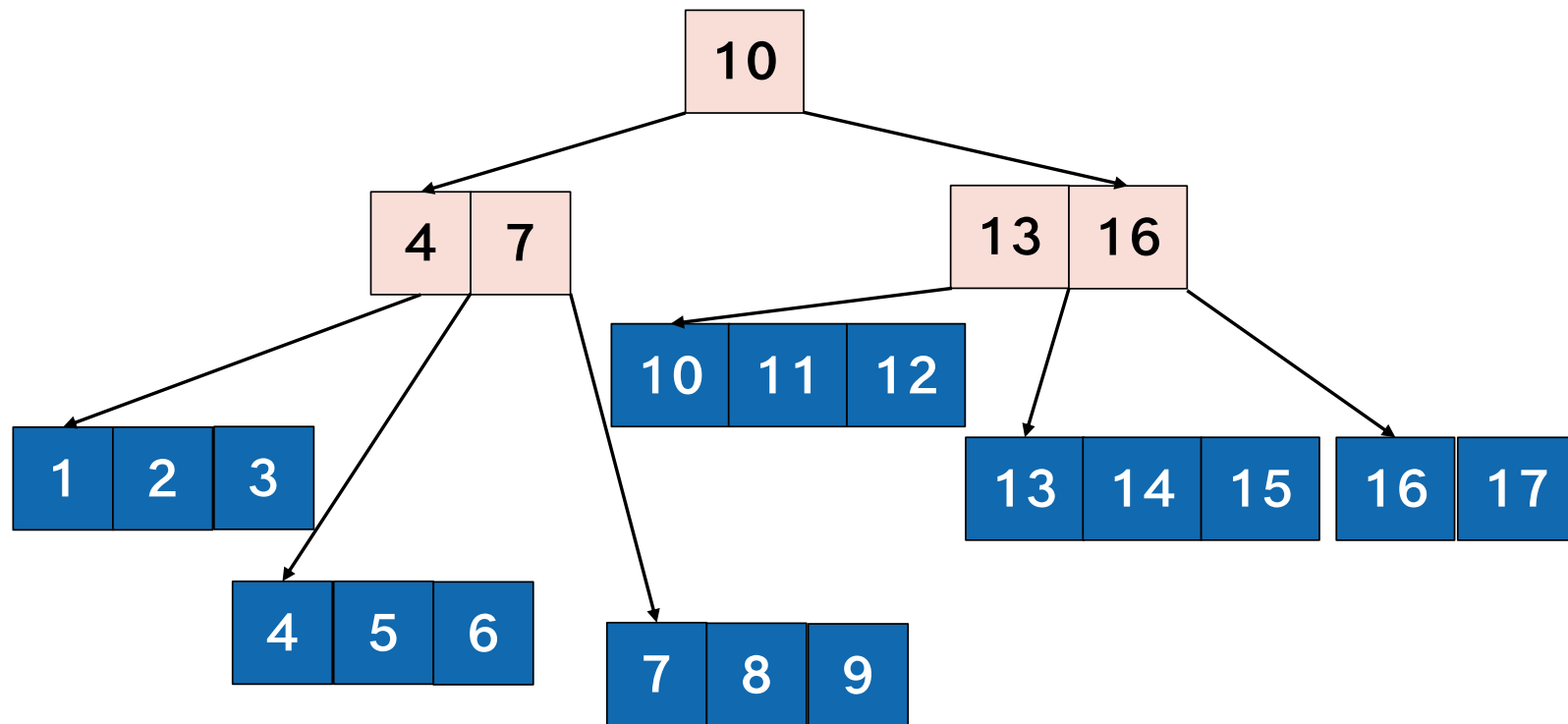
Insertion



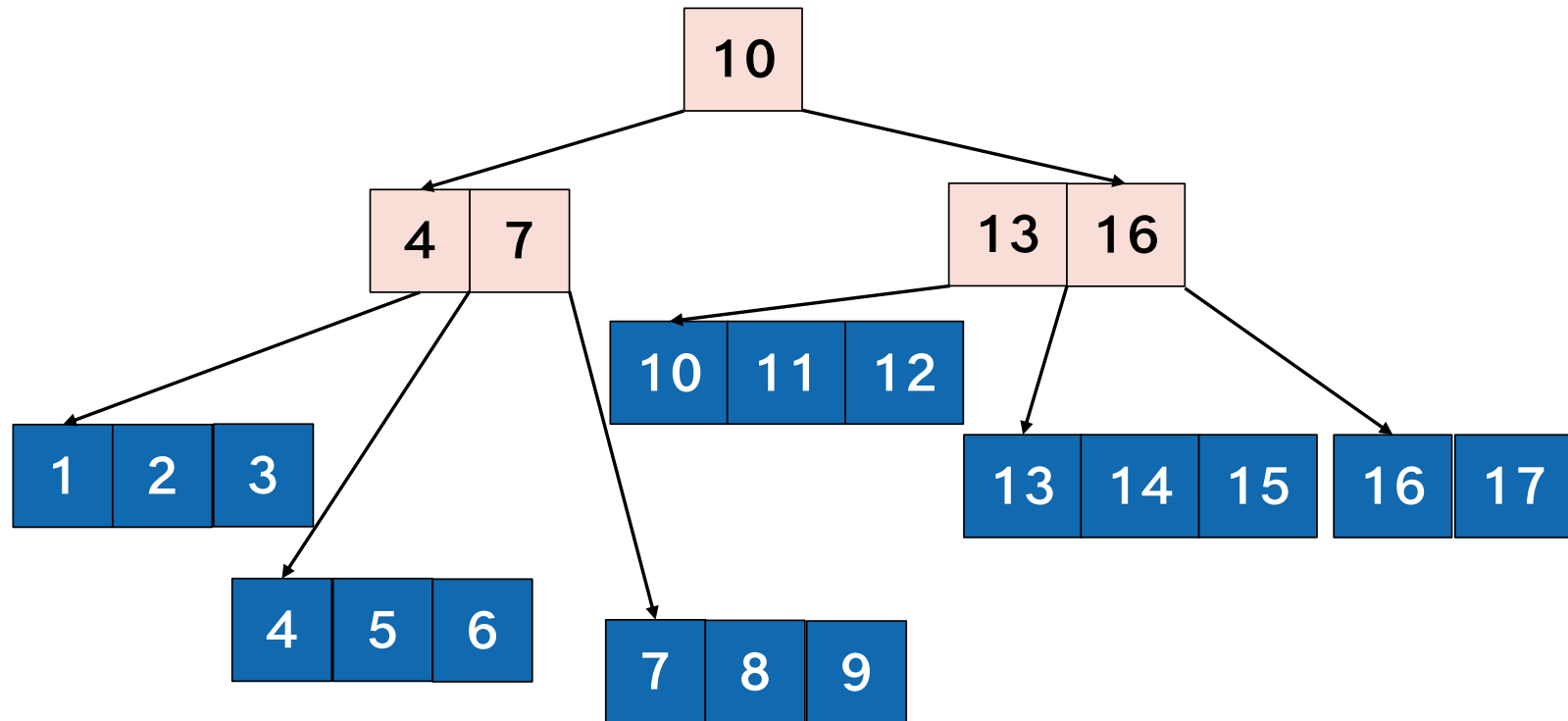
Insertion



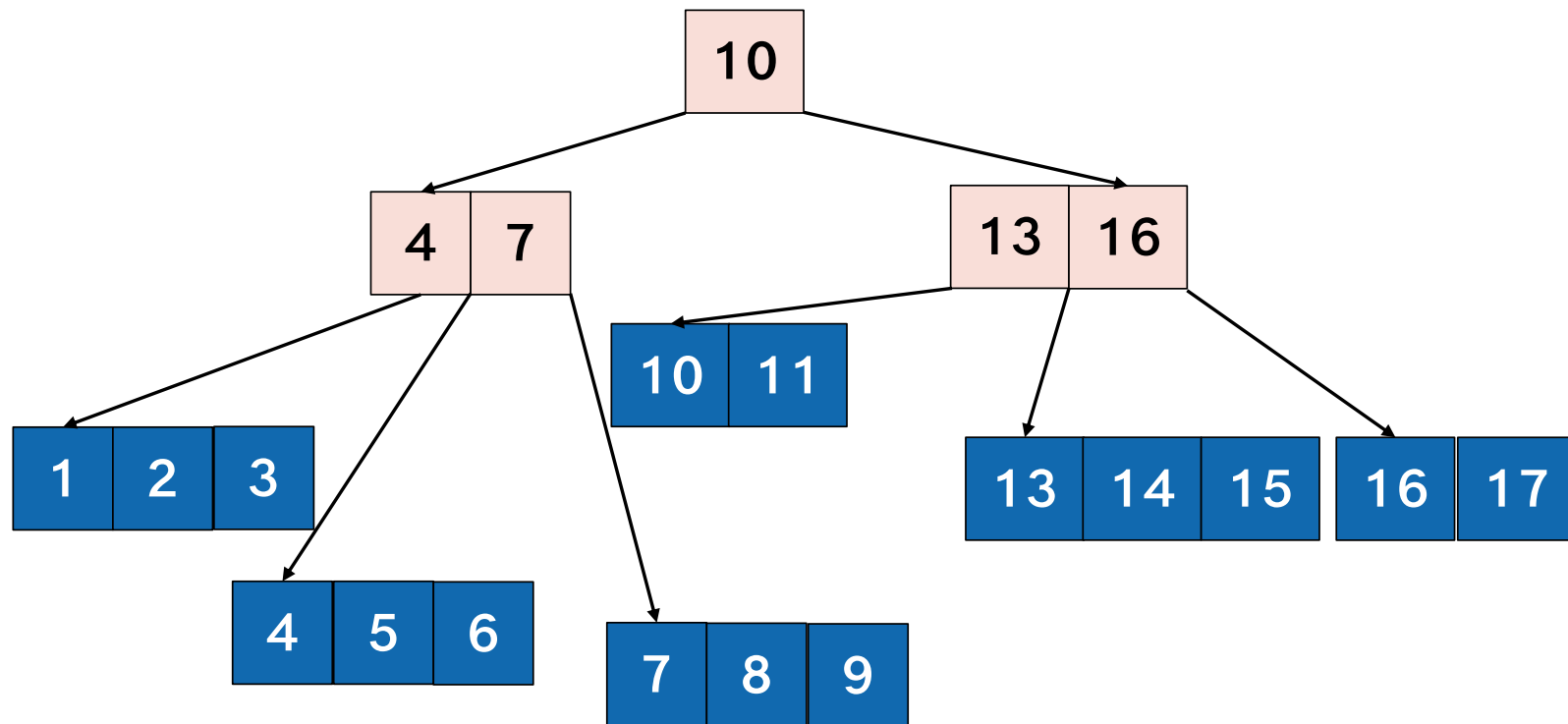
Insertion



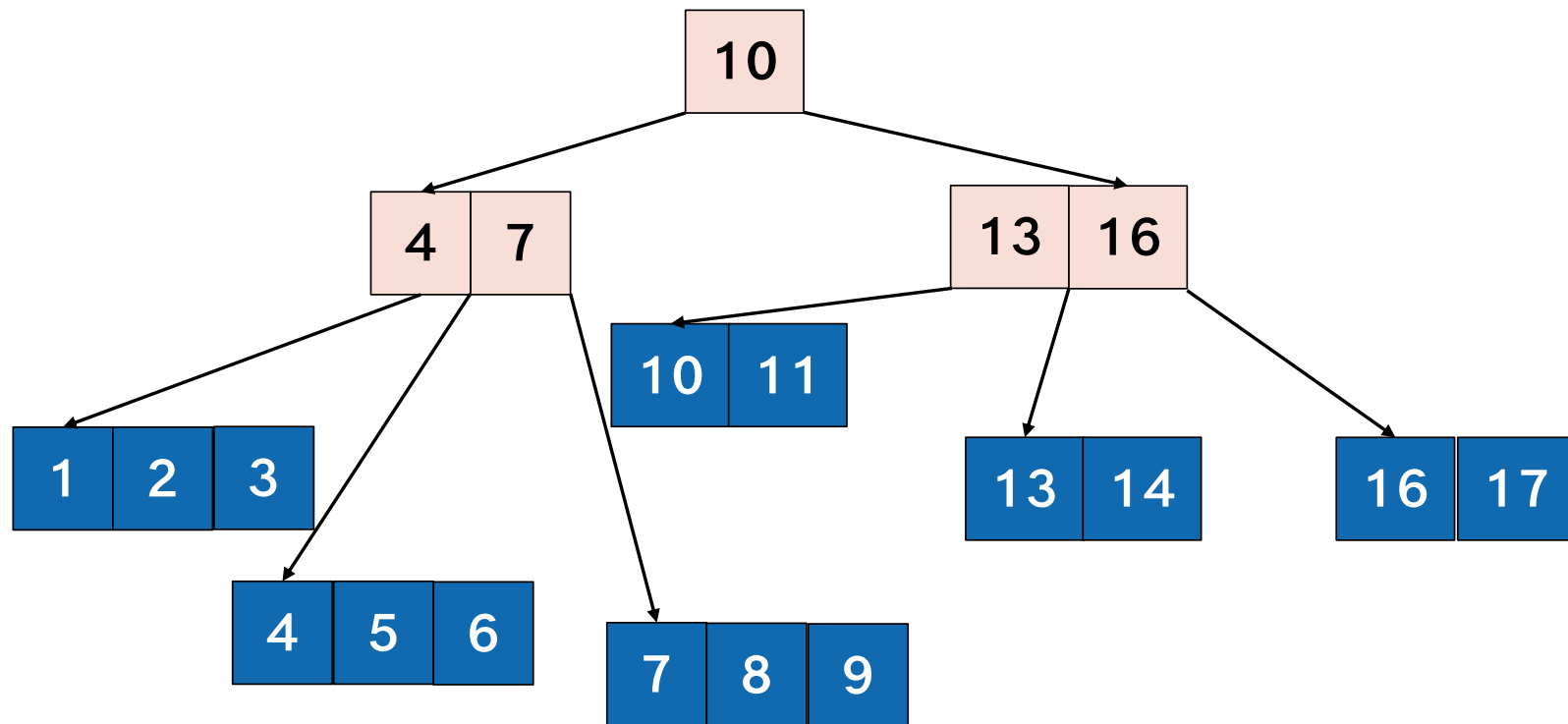
Deletion



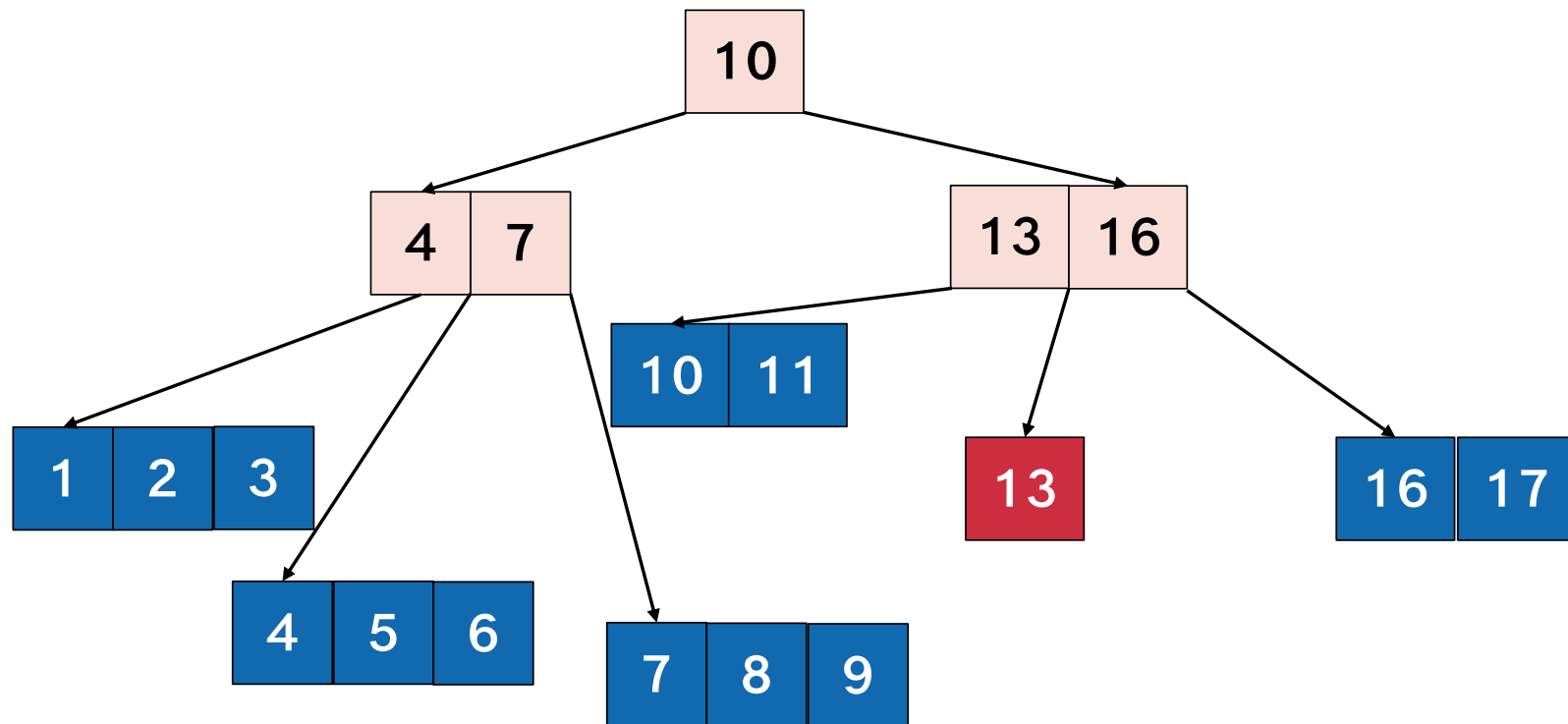
Deletion



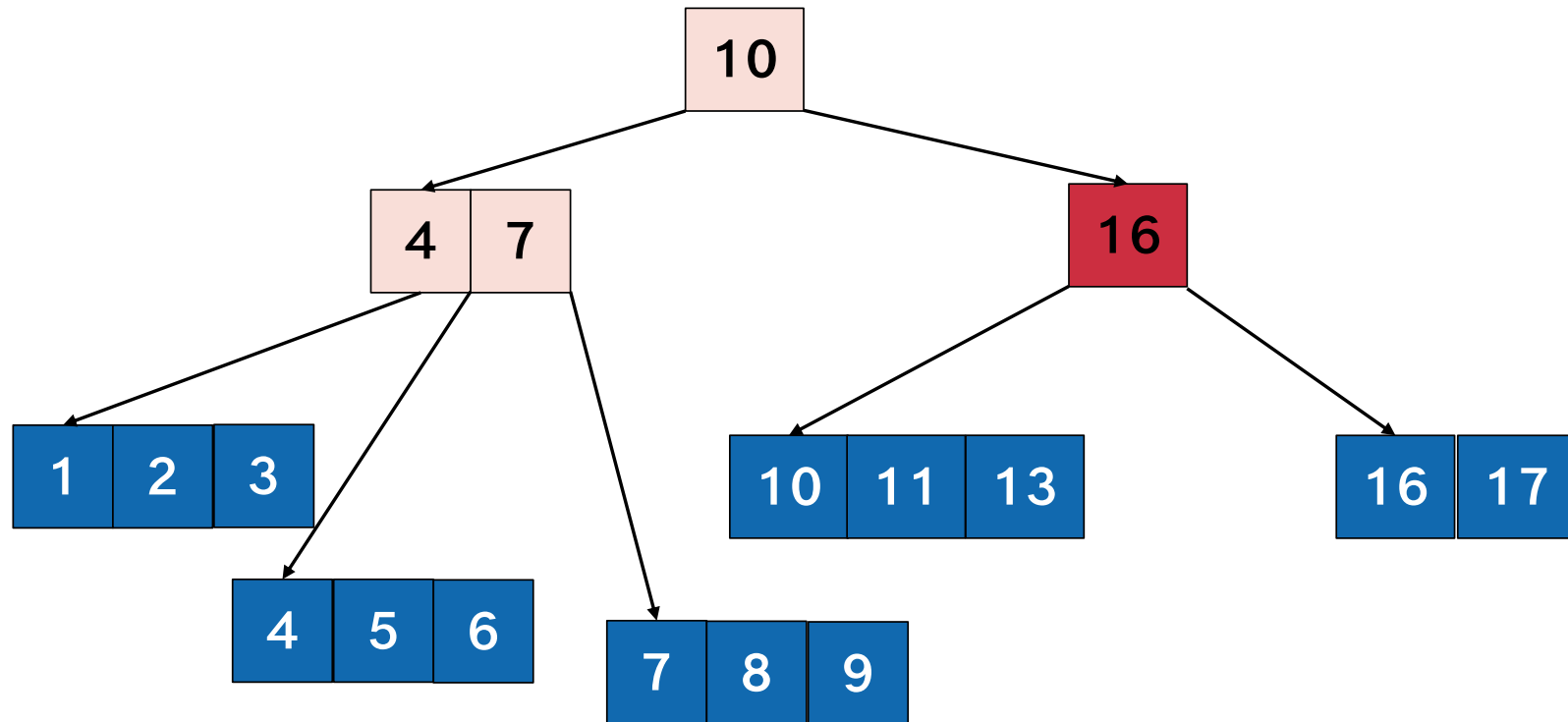
Deletion



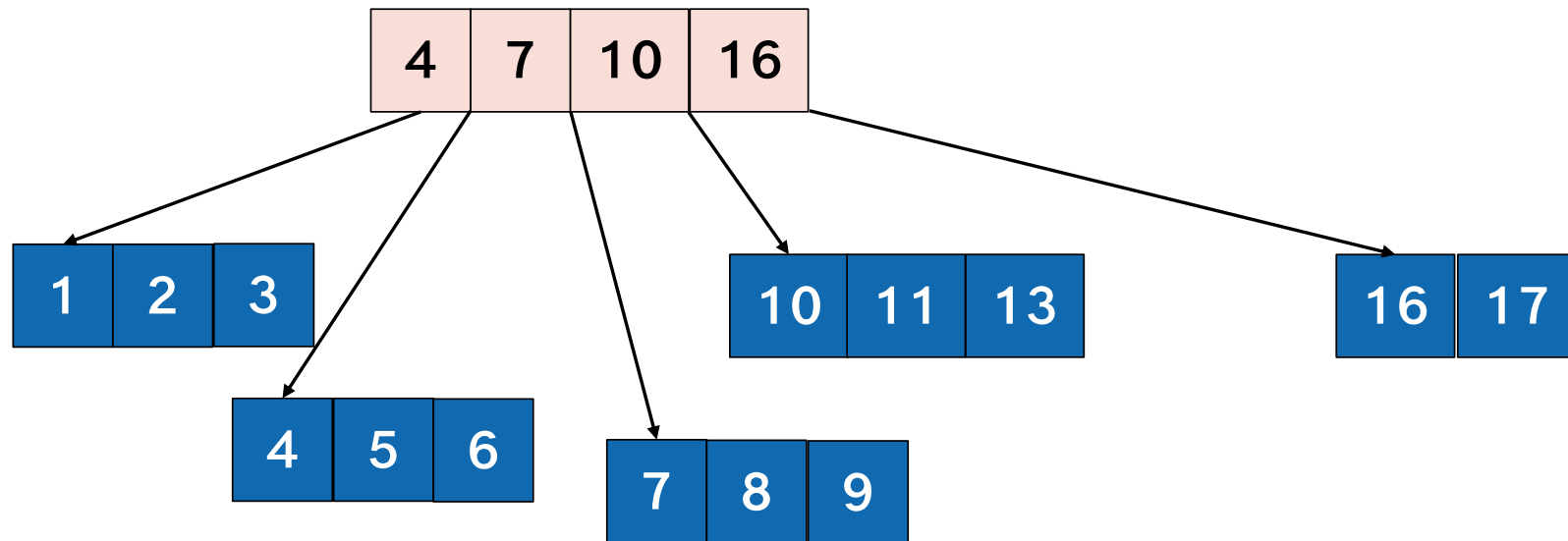
Deletion



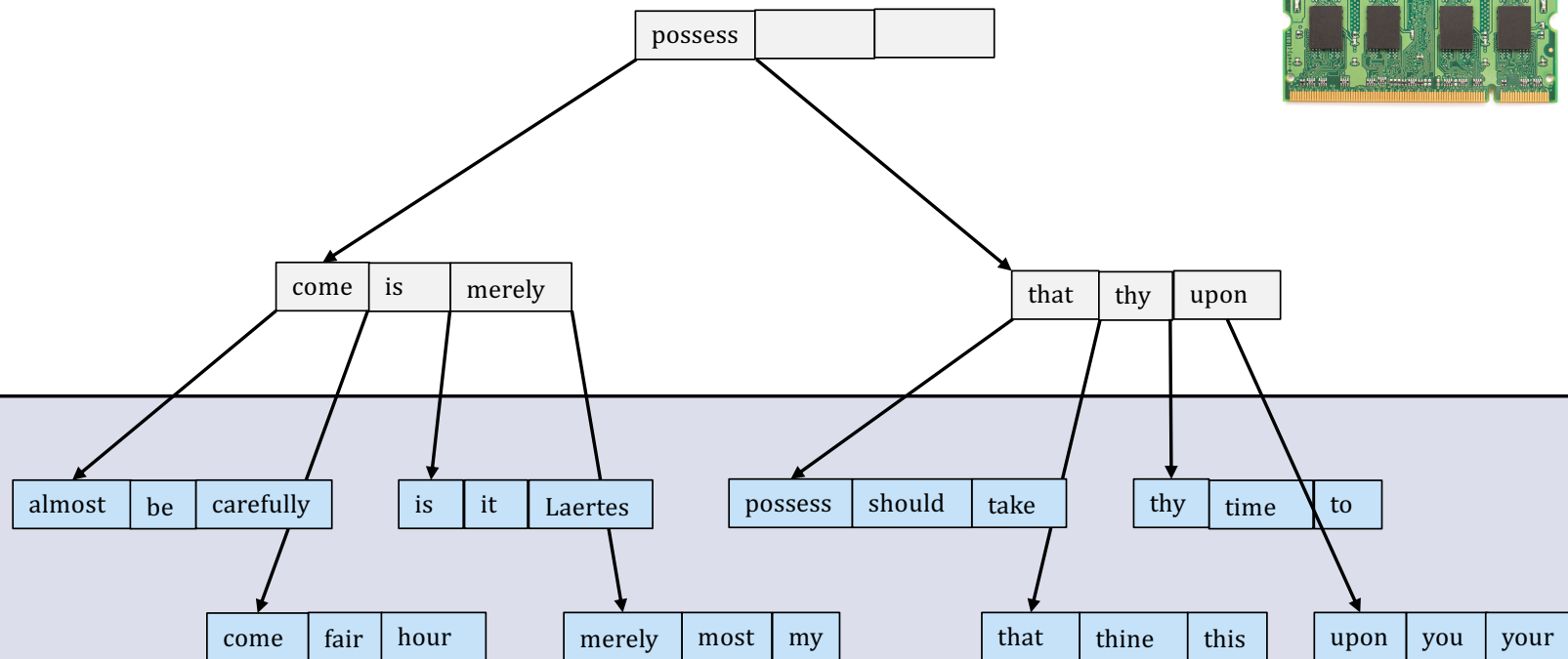
Deletion



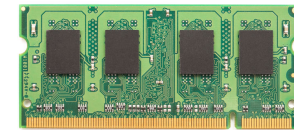
Deletion



Storing a B+-tree



+ postings lists





Wildcard queries

Wildcard



Wildcard



Matches any sequence of characters

Wildcard



Matches any sequence of characters

ETH
Switzerland
Zürich
Mathematics

Wildcard

Z*rich

multiple spellings

Wildcard

Z*rich

multiple spellings

Zurich

Zuerich

Zürich

Wildcard

informat*

word families

Wildcard

informat*

word families

informatics

information

informative

Wildcard

foo*eth*bar

multiple wildcards

Wildcard

foo*eth*bar

multiple wildcards

fooethbbar

fooethethzuerichbar

Leading wildcard queries

*ics

economics

physics

Trailing wildcard queries

Math*

Mathematician

Mathematics

Mathematical

Which is the easiest?

*ics

or

Math*

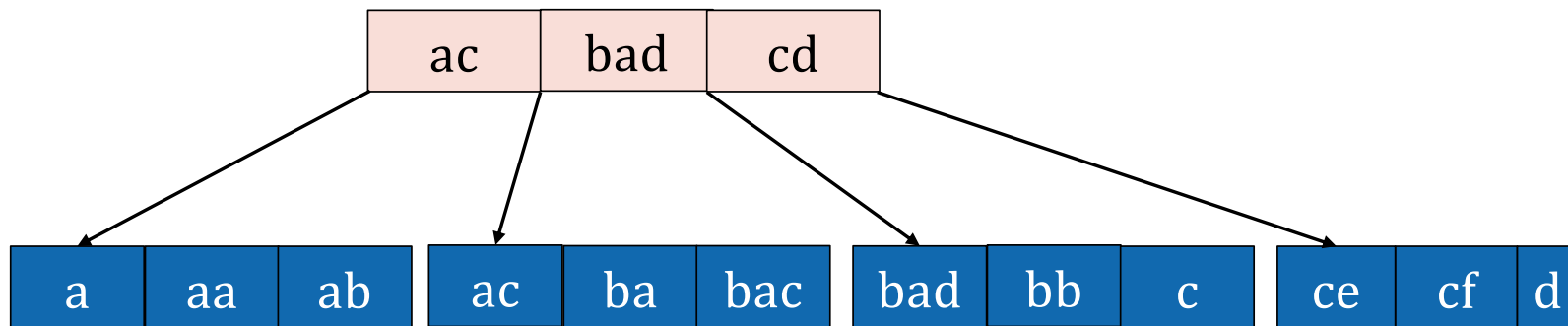
Which is the easiest?

*ics

or

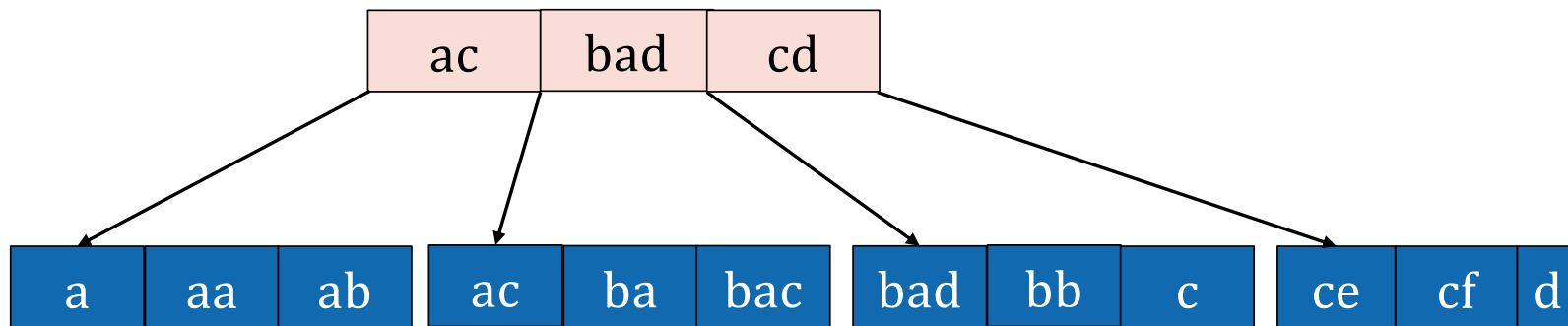
Math*

B+-tree



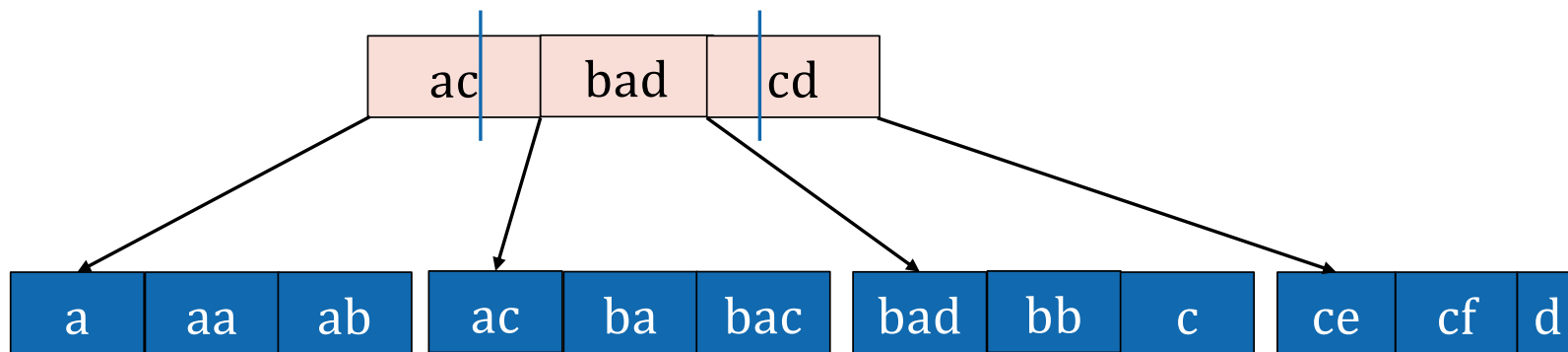
B+-tree

ba^*



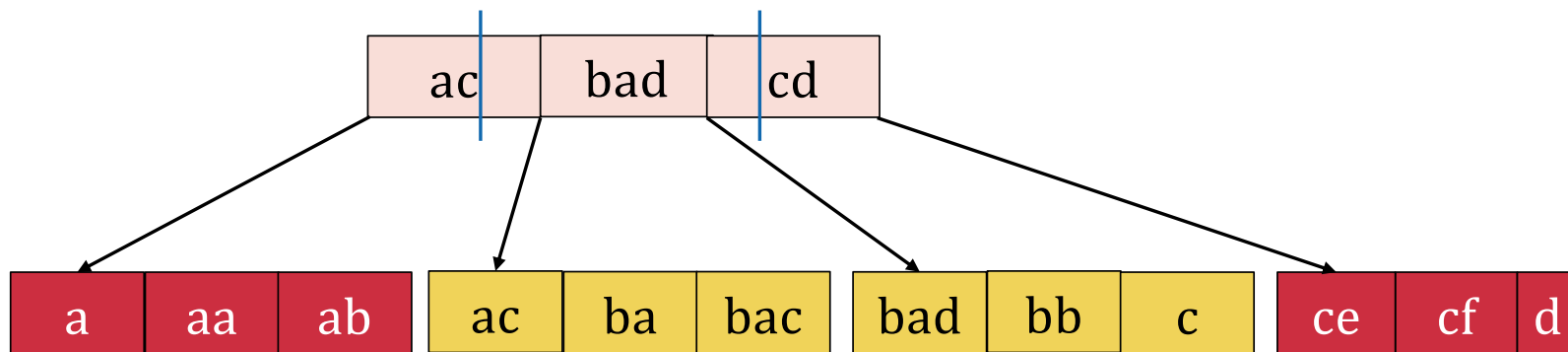
B+-tree

ba^*



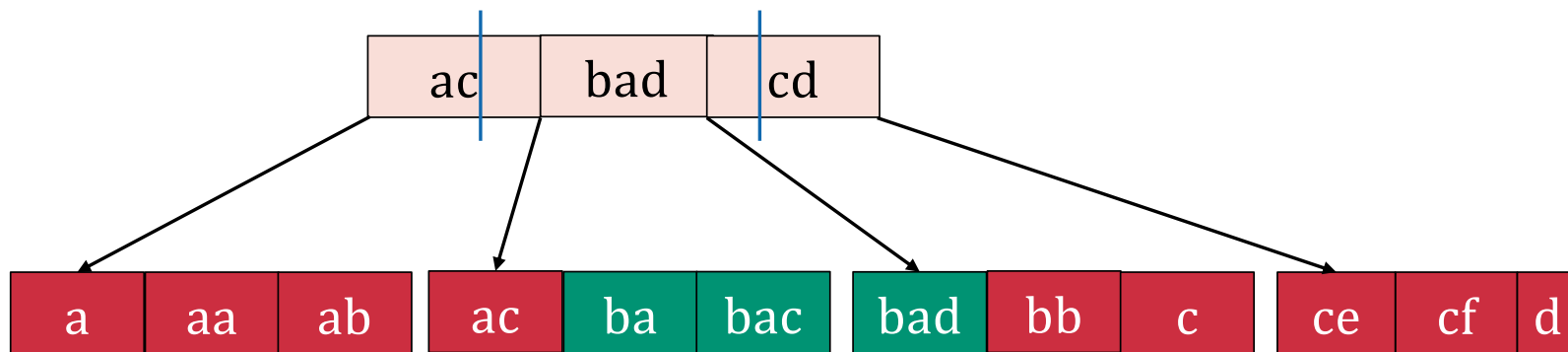
B+-tree

ba^*



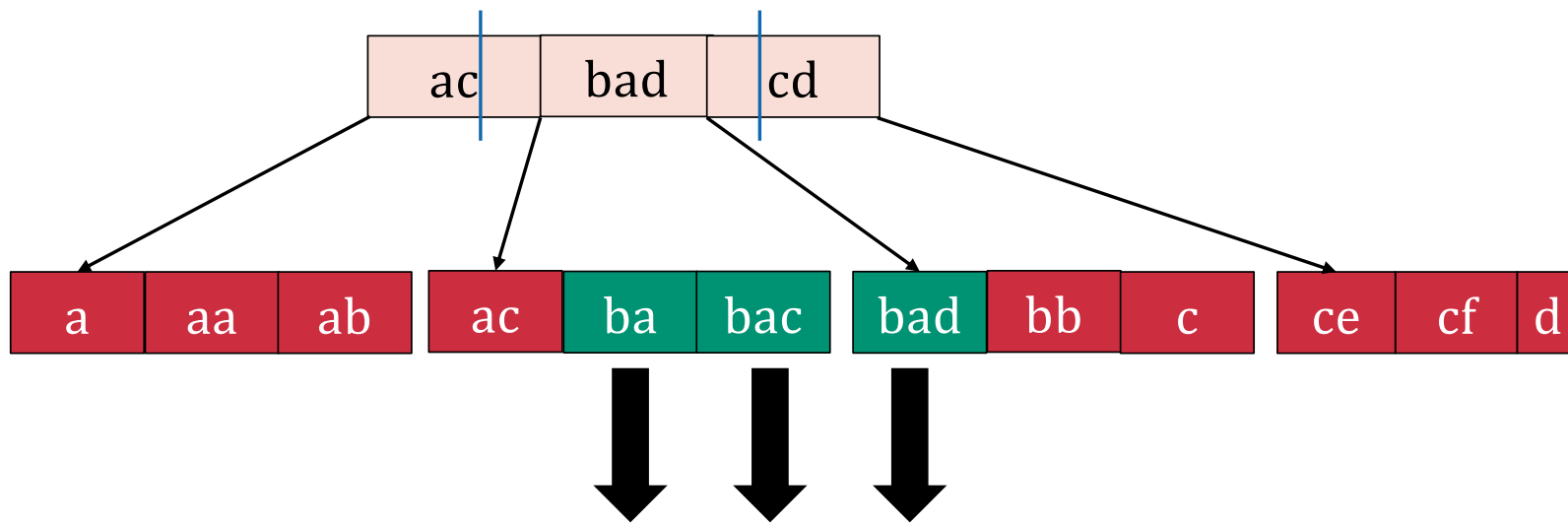
B+-tree

ba^*



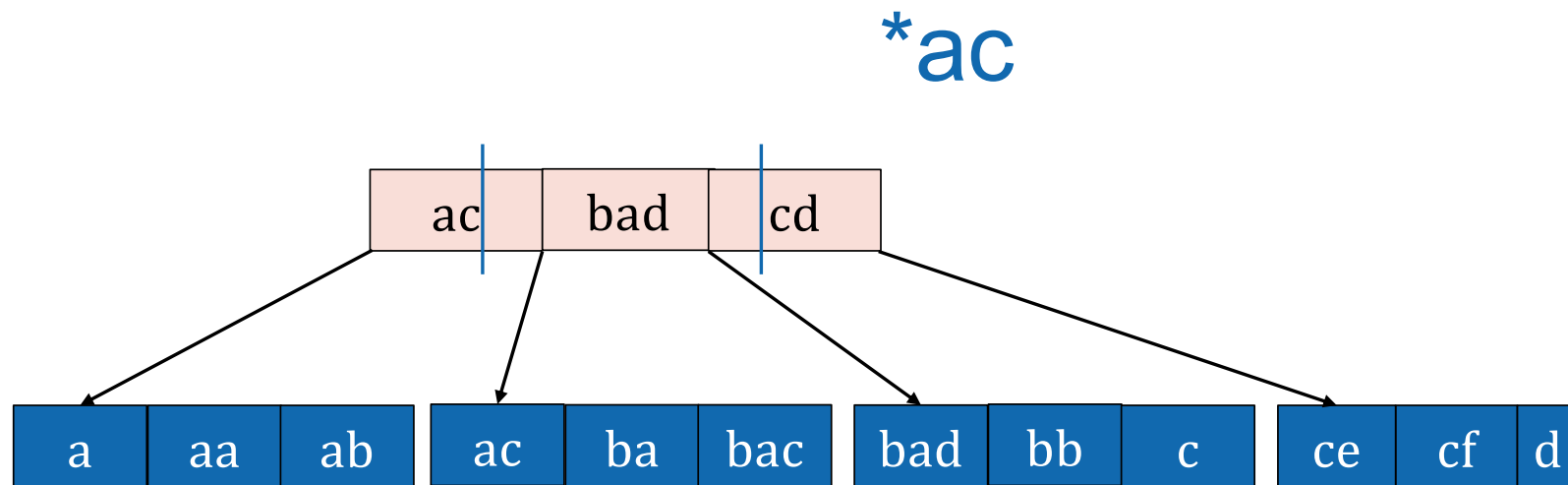
B+-tree

ba*



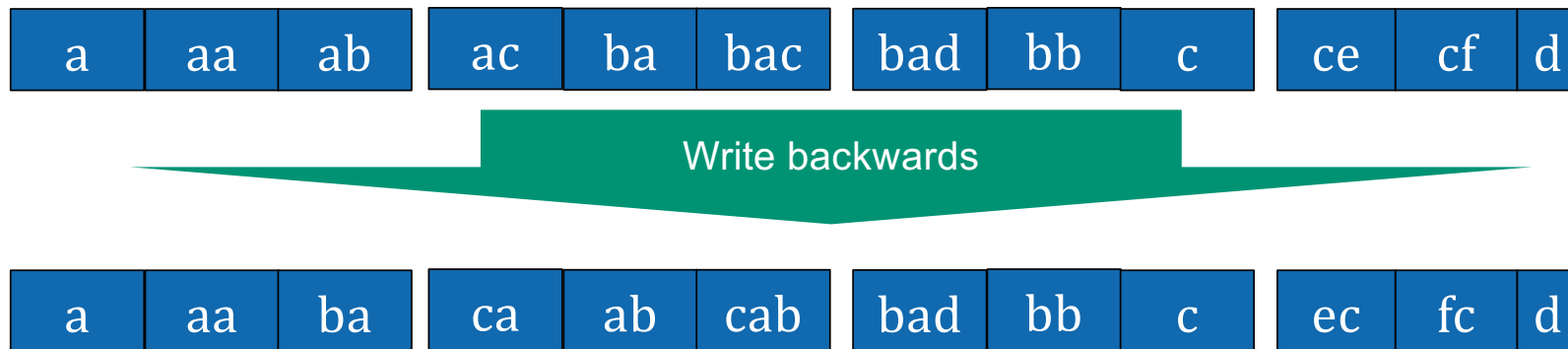
Posting lists

B+-tree



B+-tree

*ac



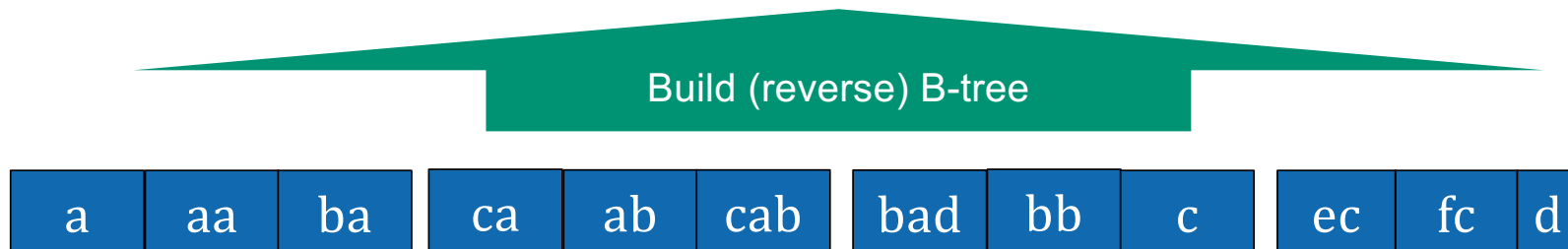
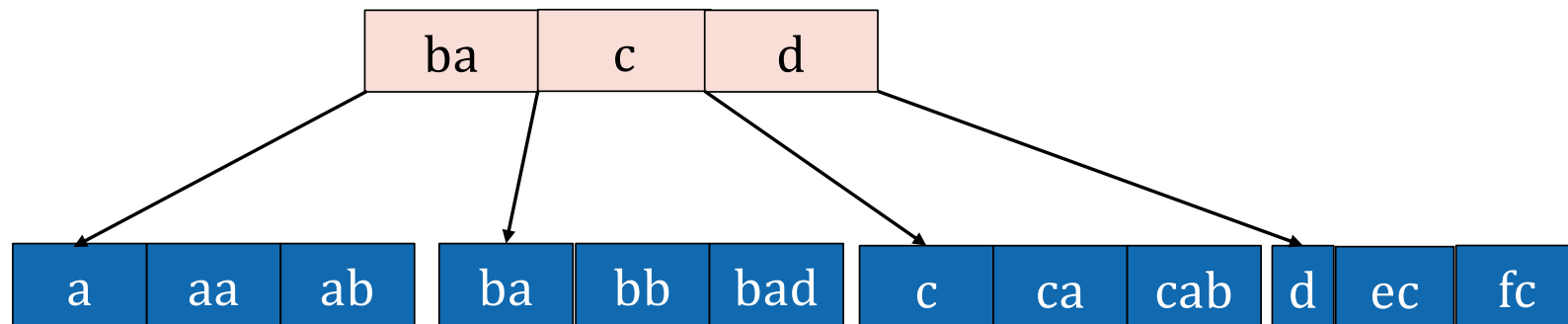
B+-tree

*ac

a	aa	ba	ca	ab	cab	bad	bb	c	ec	fc	d
---	----	----	----	----	-----	-----	----	---	----	----	---

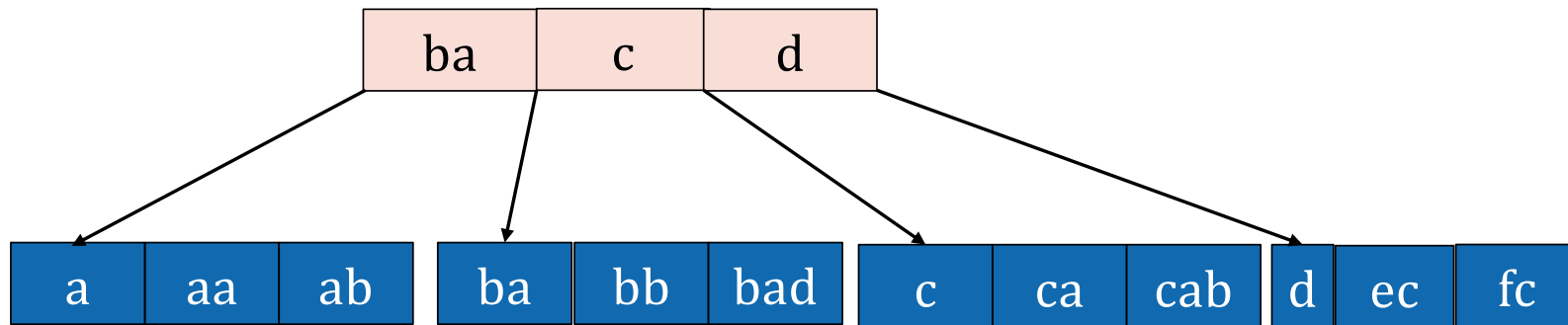
B+-tree

*ac



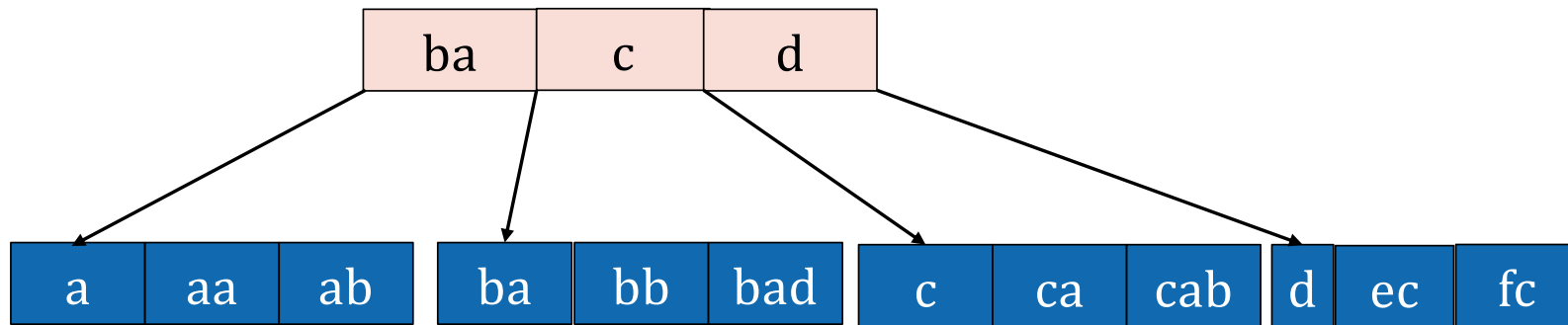
B+-tree

*ac

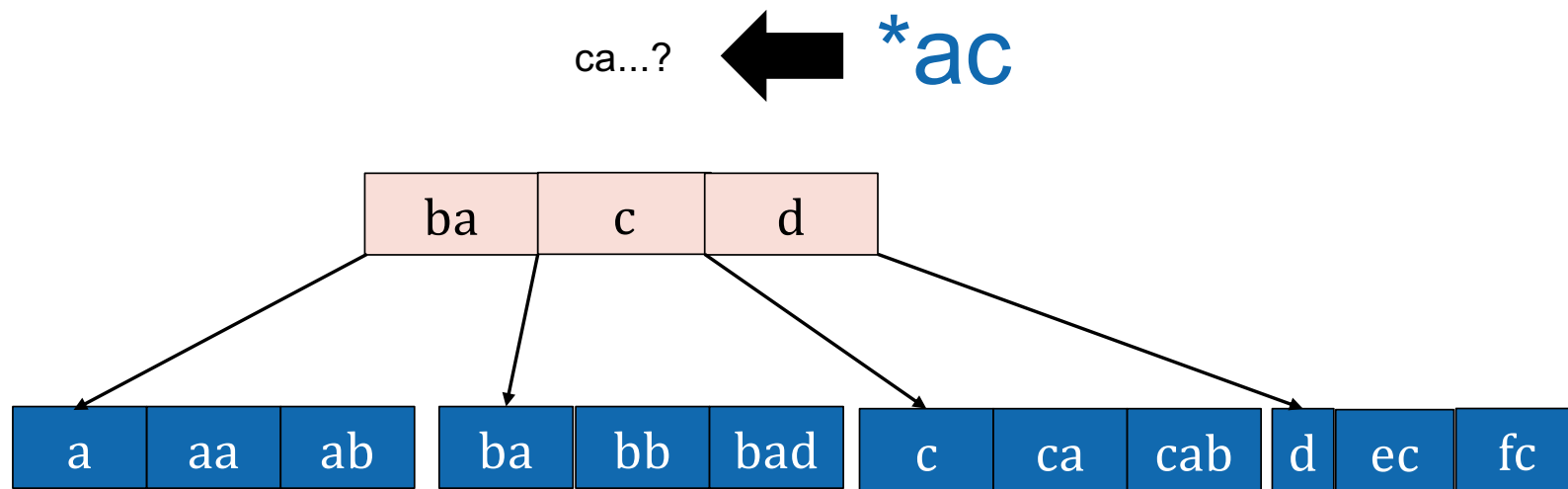


B+-tree

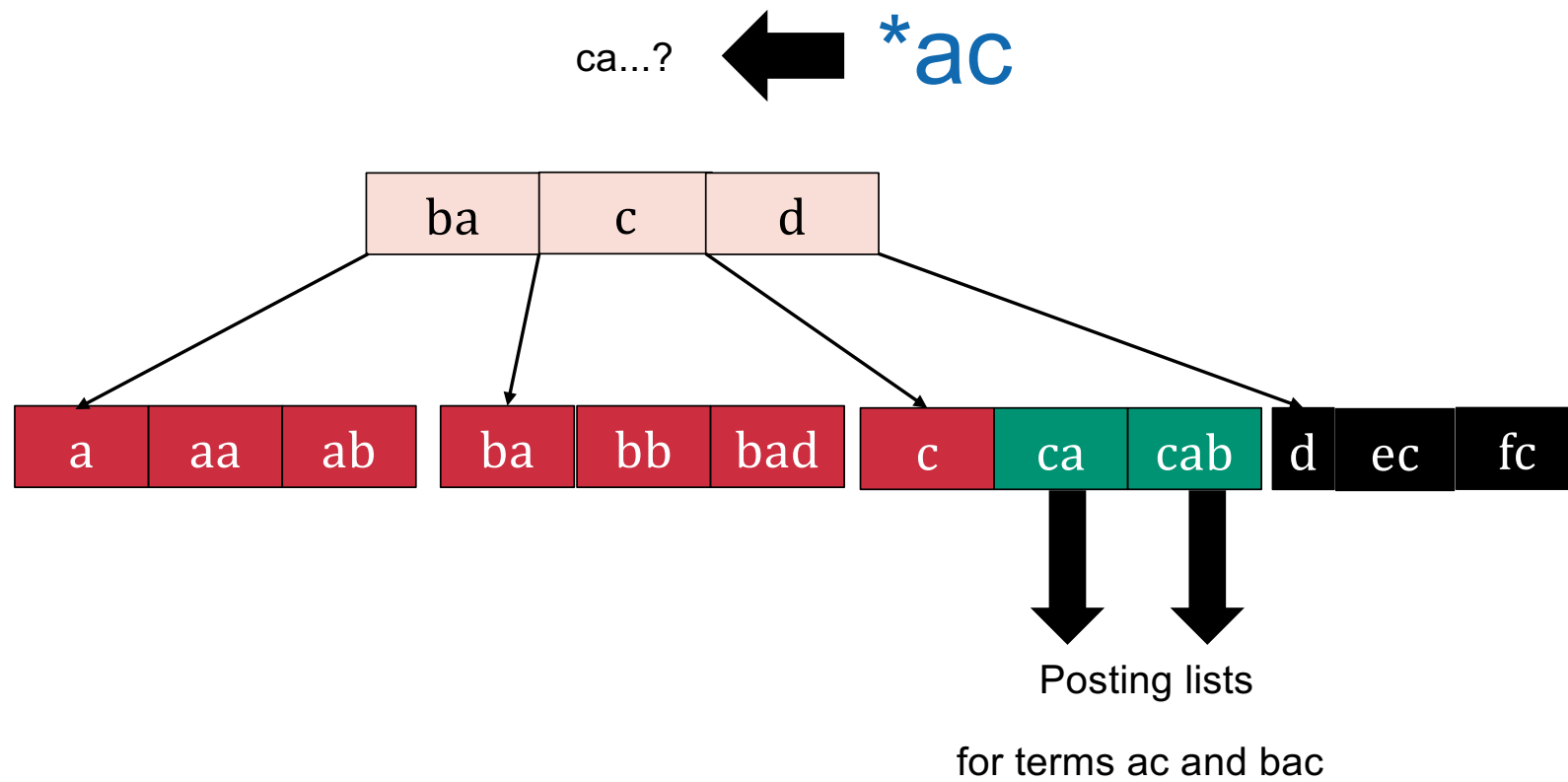
*ac



B+-tree



B+-tree



Single-wildcard query

Zu*ch

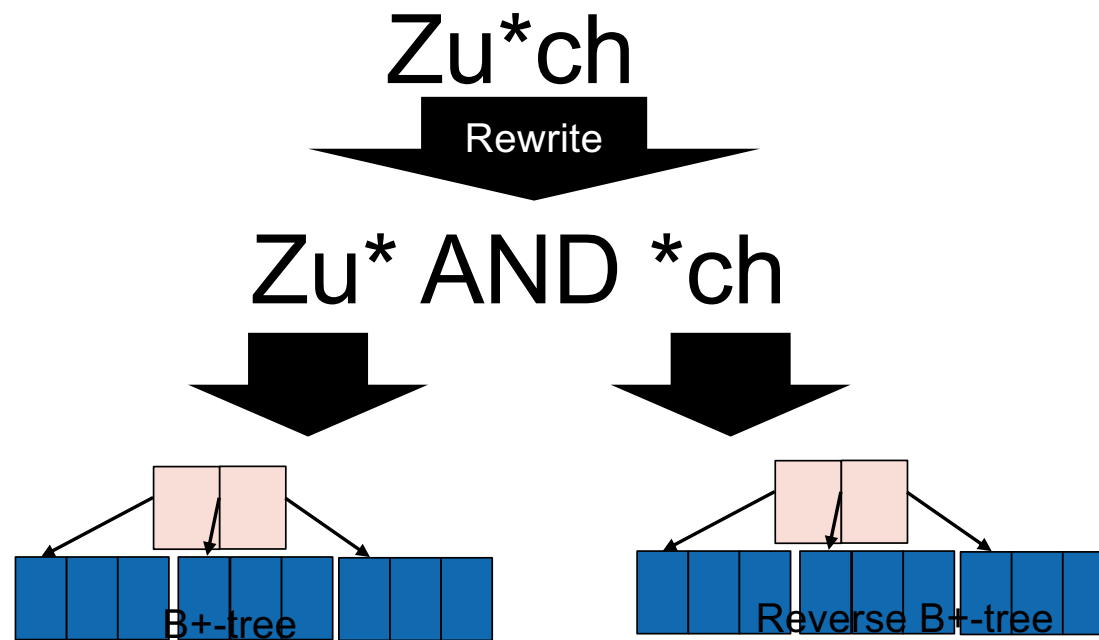
Single-wildcard query

Zu*ch

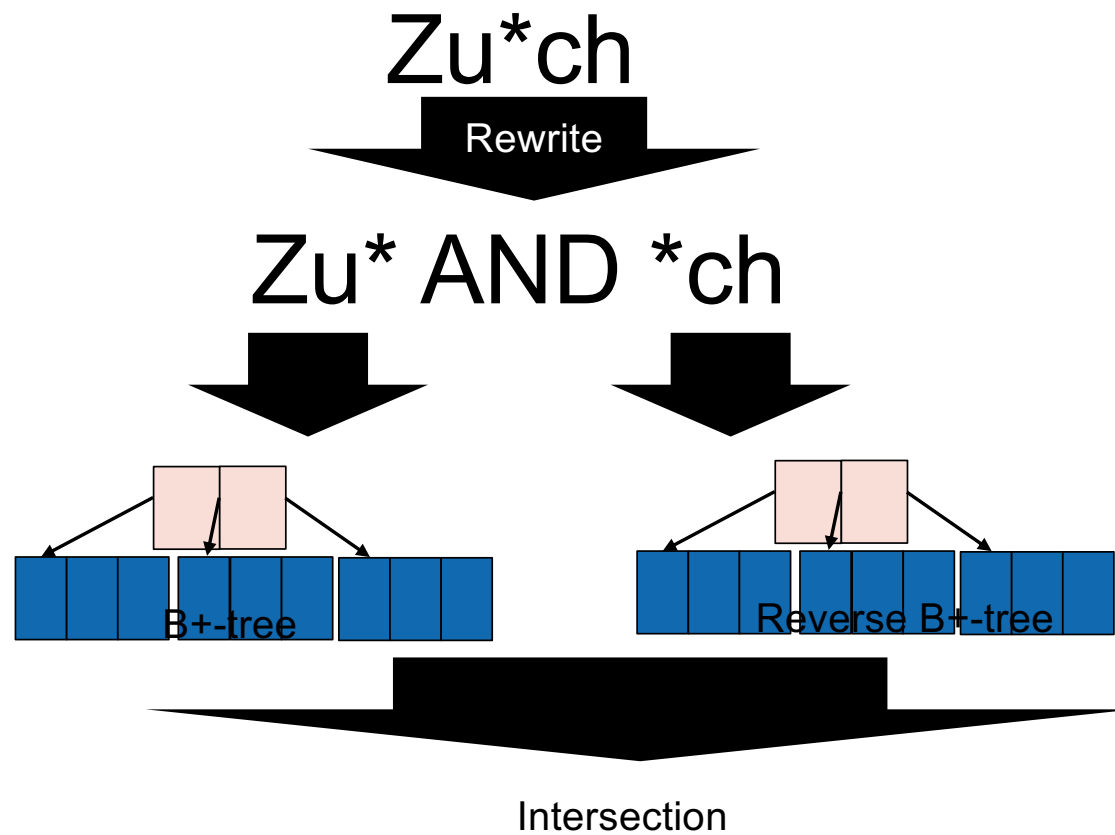


Zu* AND *ch

Single-wildcard query



Single-wildcard query



Single-wildcard query

stati*tics

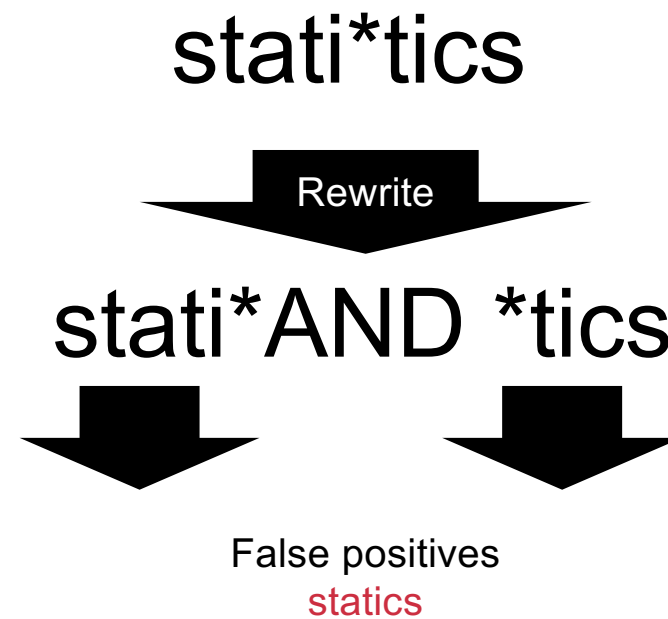
Single-wildcard query

stati*tics

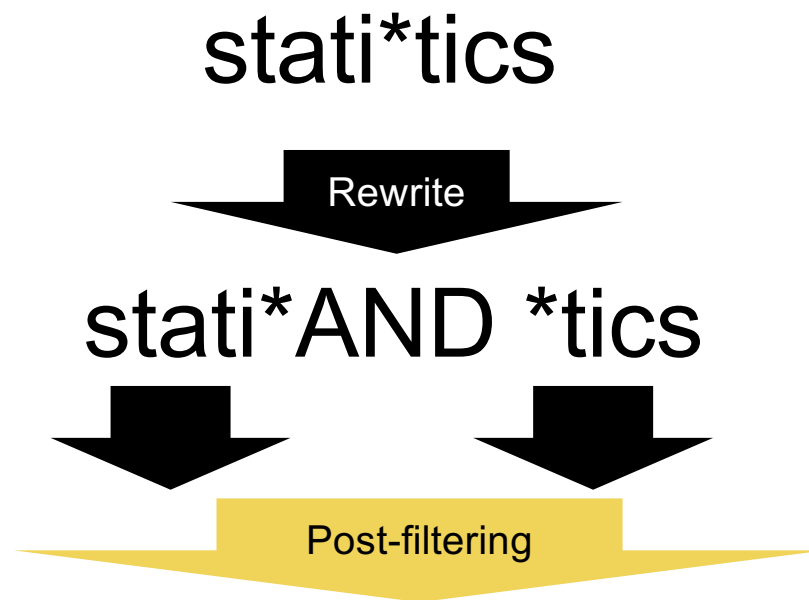


stati*AND *tics

Single-wildcard query



Single-wildcard query



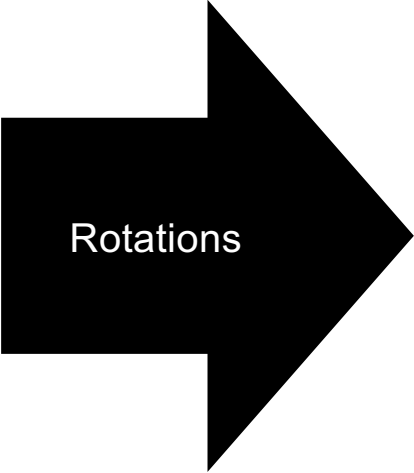
Permuterm index

Permuterm index

plant

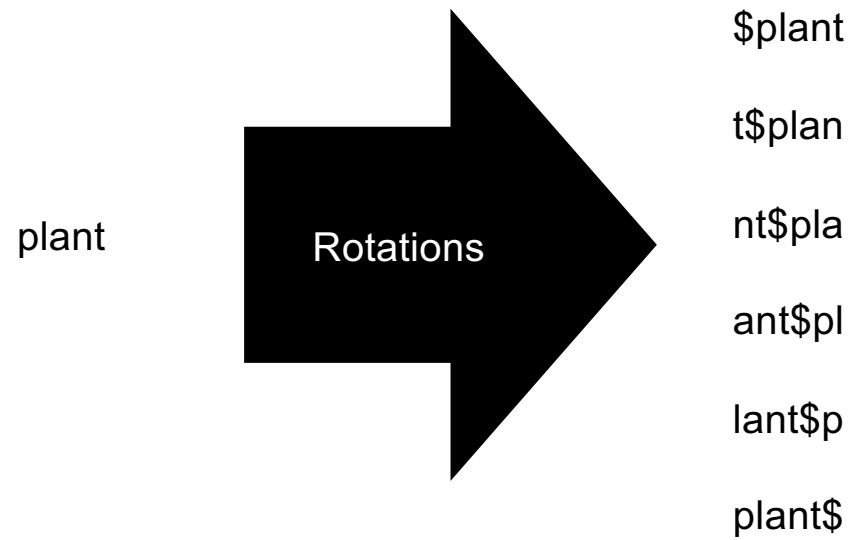
Permuterm index

plant

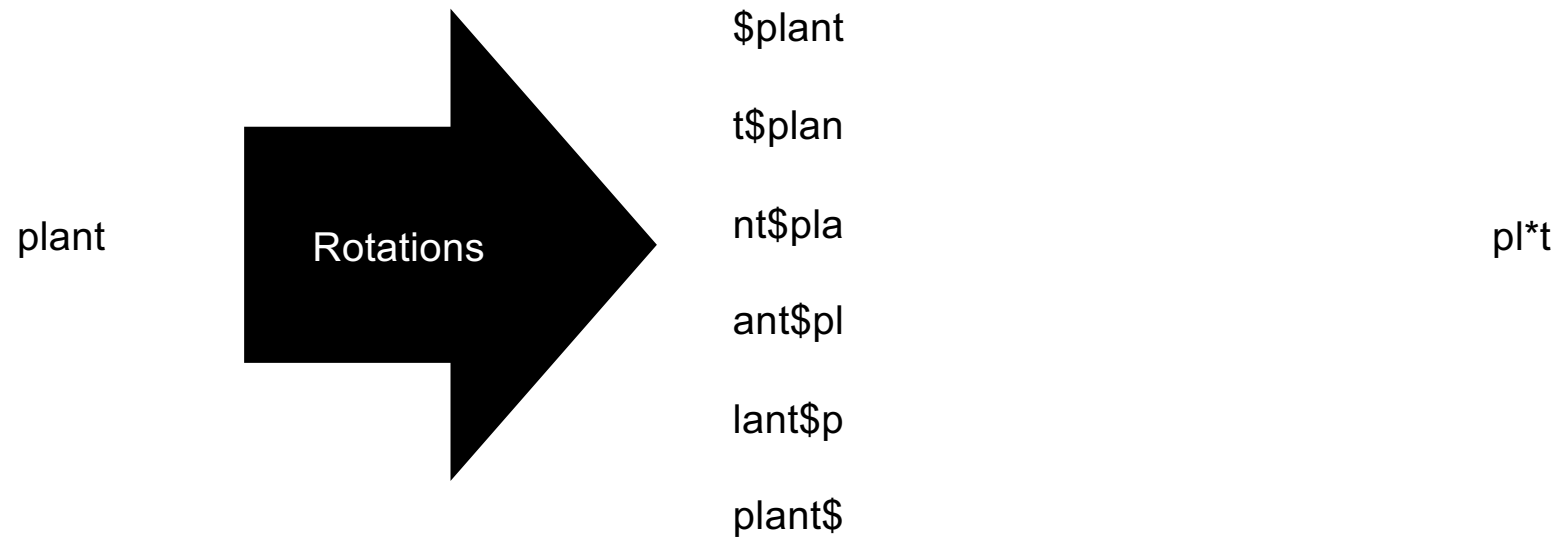


Rotations

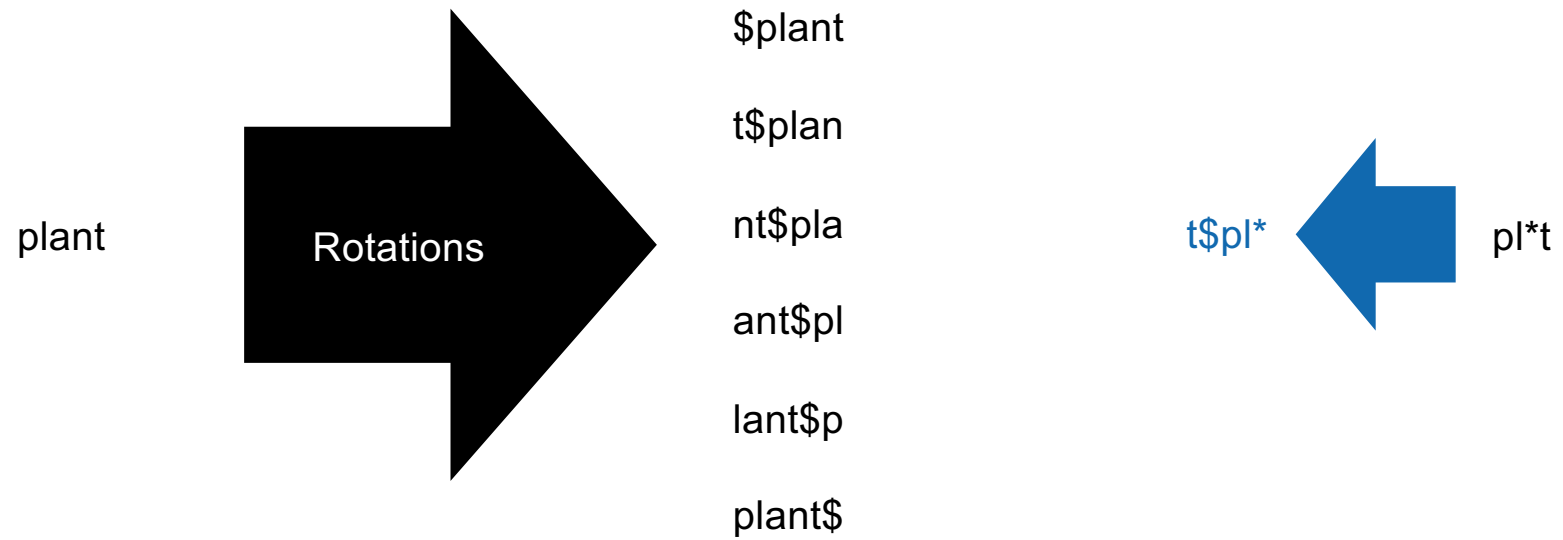
Permuterm index



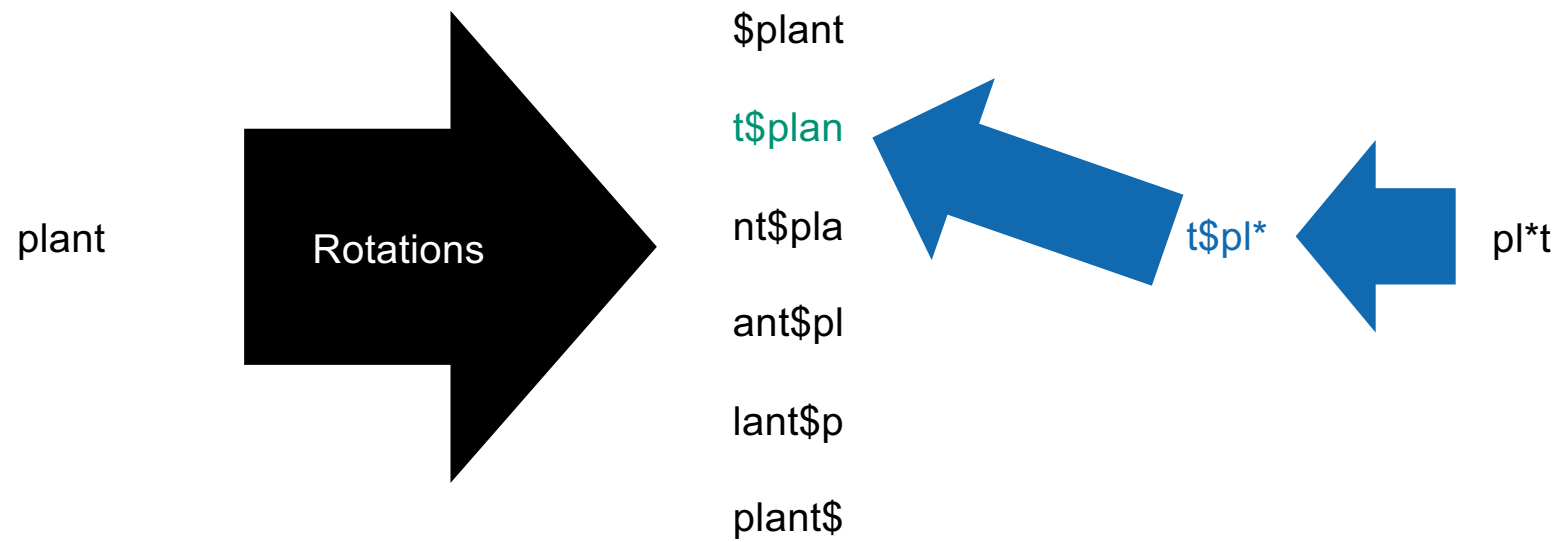
Permuterm index



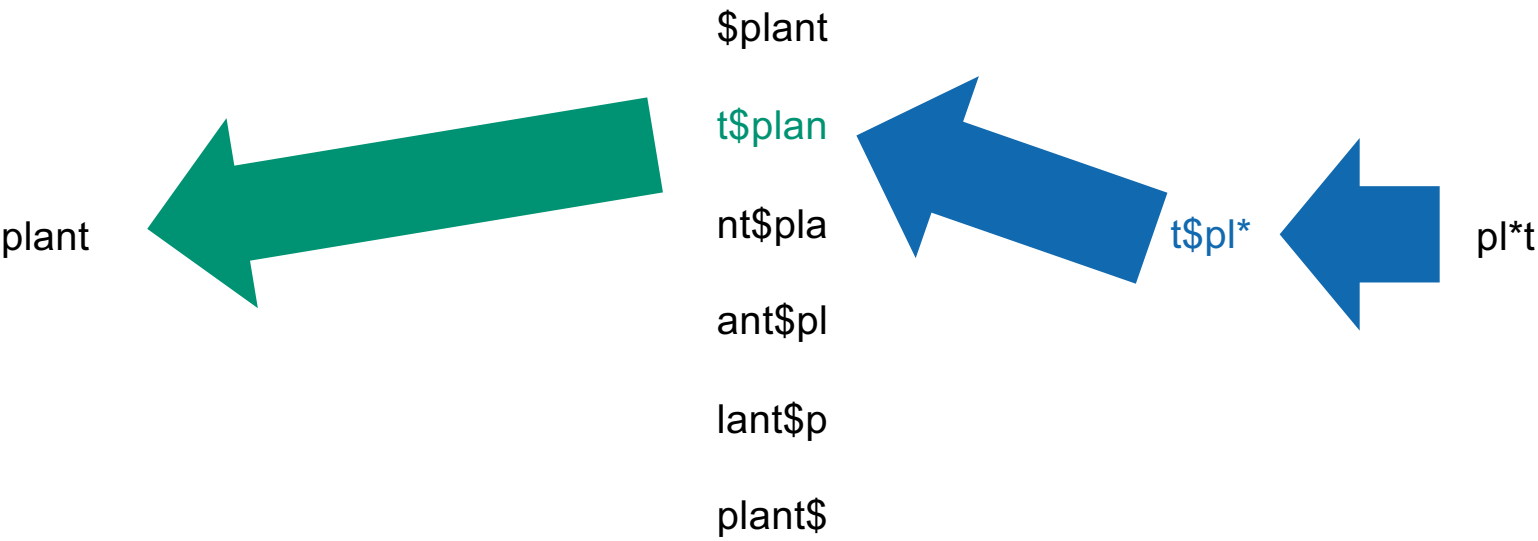
Permuterm index



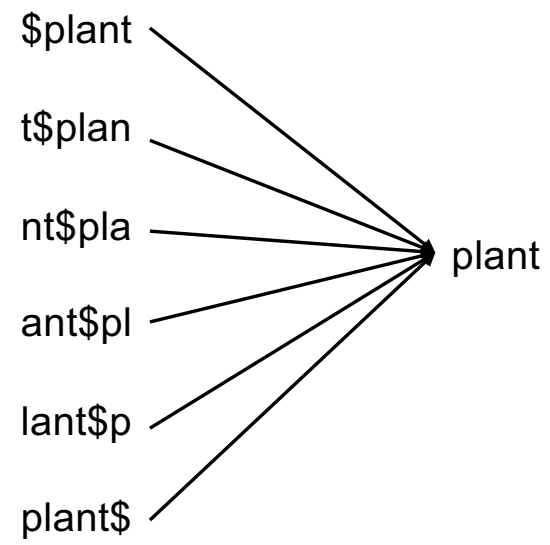
Permuterm index



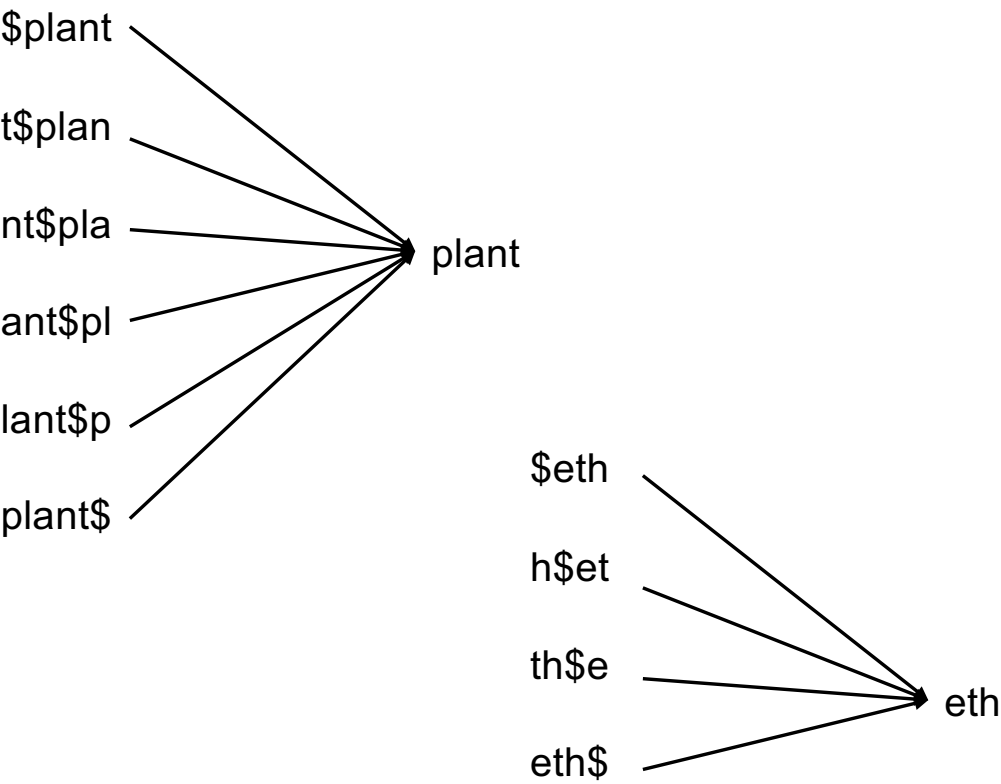
Permuterm index



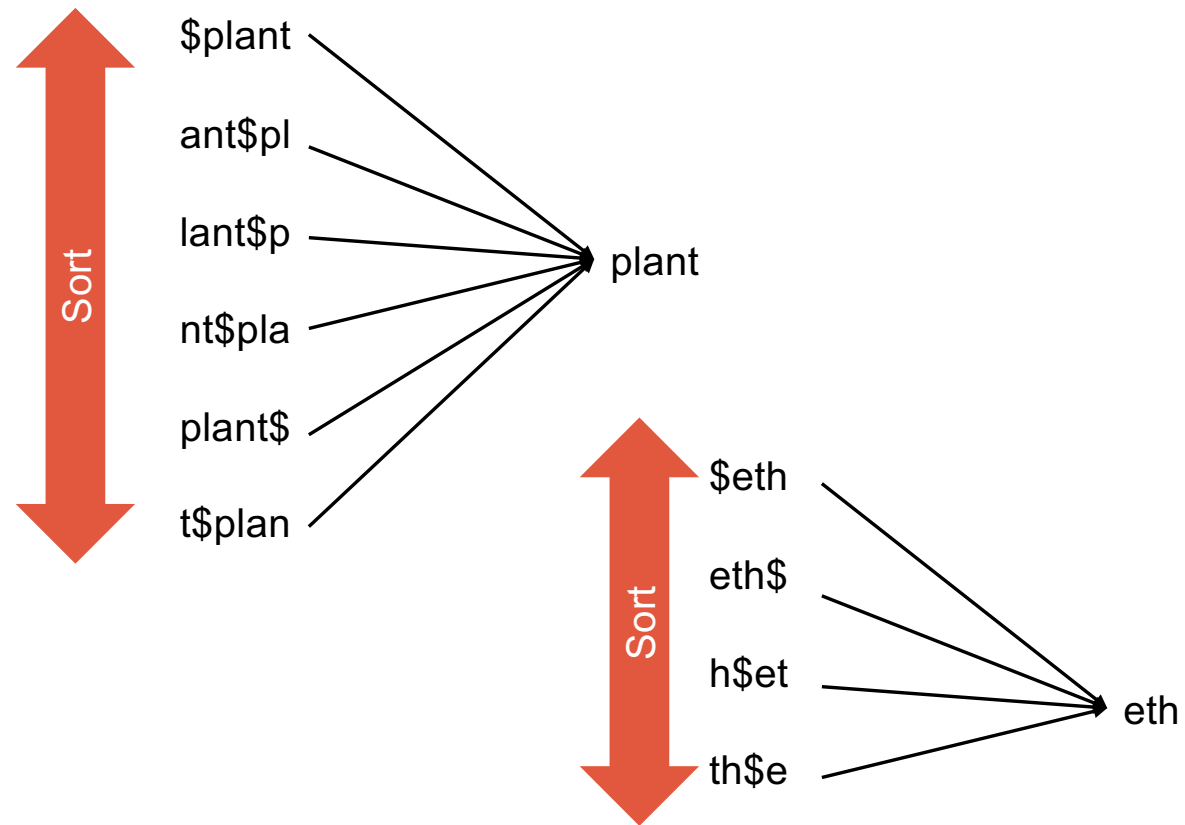
Permuterm index



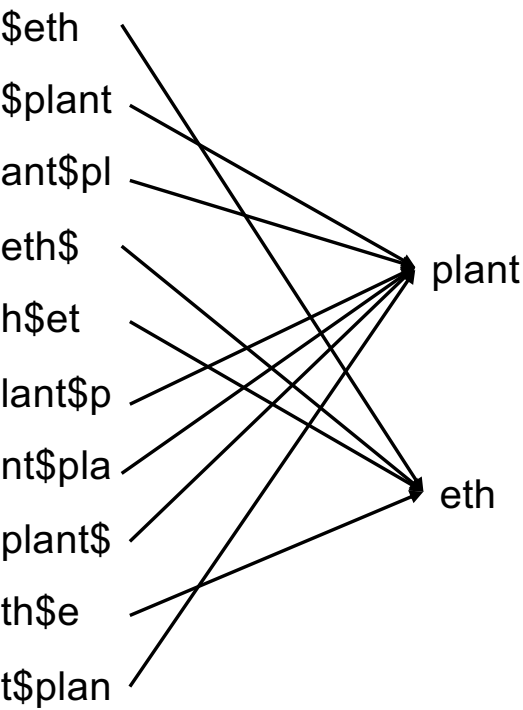
Permuterm index



Permuterm index

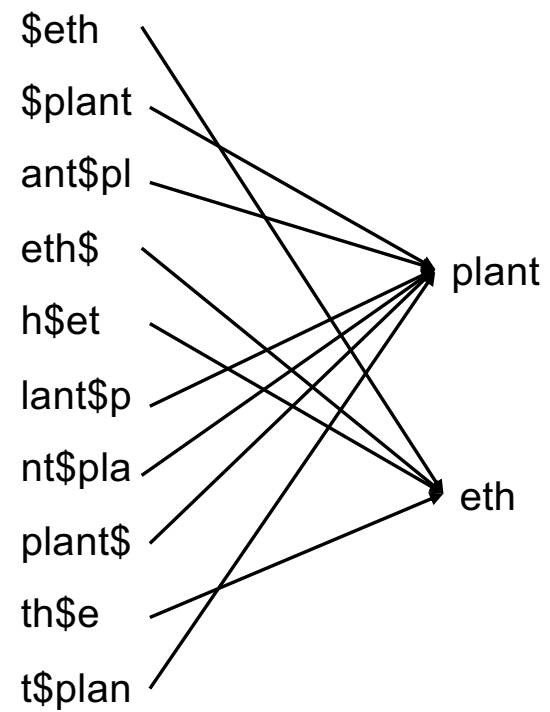


Permuterm index

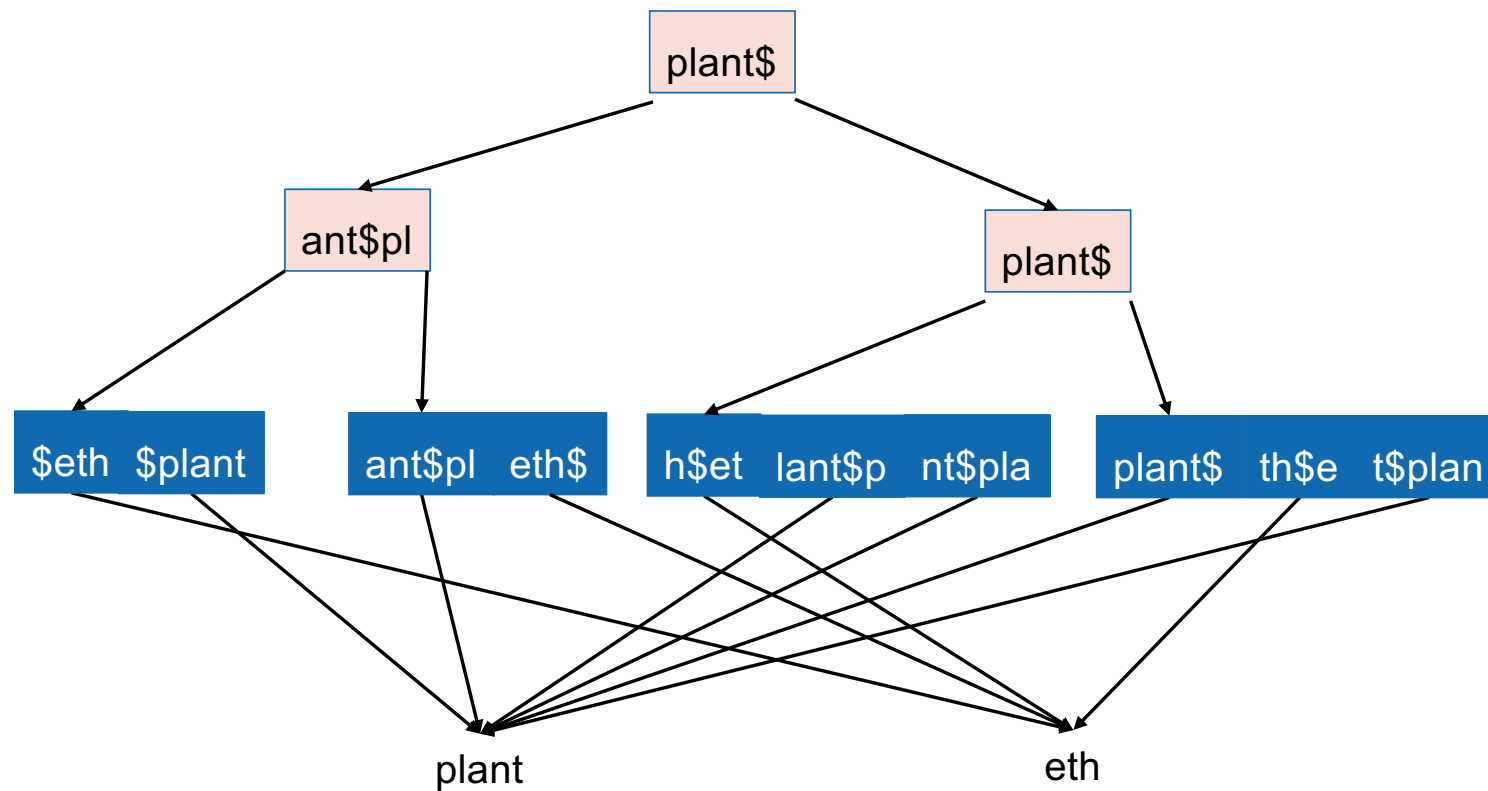


Permuterm index

Use a
B+-tree!



Permuterm index (as a B+-tree)



k-gram

mpu

k-gram

computer

k-gram

1-grams \$, c, o, m, p, u, t, e, r, \$

computer

k-gram

1-grams \$, c, o, m, p, u, t, e, r, \$

2-grams \$c, co, om, mp, pu, ut, te, er, r\$

computer

k-gram

1-grams \$, c, o, m, p, u, t, e, r, \$

2-grams \$c, co, om, mp, pu, ut, te, er, r\$

3-grams \$co, com, omp, mpu, put, ute, ter, er\$

computer

k-gram

computer	1-grams	\$, c, o, m, p, u, t, e, r, \$
	2-grams	\$c, co, om, mp, pu, ut, te, er, r\$
	3-grams	\$co, com, omp, mpu, put, ute, ter, er\$
	4-grams	\$com, comp, ompu, mput, pute, uter, ter\$
	5-grams	\$comp, compu, omput, mpute, puter, uter\$
	6-grams	\$compu, comput, ompute, mputer, puter\$
	7-grams	\$comput, compute, omputer, mputer\$
...		

k-gram

computer	1-grams	\$, c, o, m, p, u, t, e, r, \$	Not very useful
	2-grams	\$c, co, om, mp, pu, ut, te, er, r\$	
	3-grams	\$co, com, omp, mpu, put, ute, ter, er\$	
	4-grams	\$com, comp, ompu, mput, pute, uter, ter\$	
	5-grams	\$comp, compu, omput, mpute, puter, uter\$	
	6-grams	\$compu, comput, ompute, mputer, puter\$	
	7-grams	\$comput, compute, omputer, mputer\$	
	...		

k-gram

computer	1-grams	\$, c, o, m, p, u, t, e, r, \$	Not very useful
	2-grams	\$c, co, om, mp, pu, ut, te, er, r\$	
	3-grams	\$co, com, omp, mpu, put, ute, ter, er\$	
	4-grams	\$com, comp, ompu, mput, pute, uter, ter\$	
	5-grams	\$comp, compu, omput, mpute, puter, uter\$	
	6-grams	\$compu, comput, ompute, mputer, puter\$	Not space efficient
	7-grams	\$comput, compute, omputer, mputer\$	
...			

k-gram

computer	1-grams	\$, c, o, m, p, u, t, e, r, \$	Not very useful
	2-grams	\$c, co, om, mp, pu, ut, te, er, r\$	Usable zone
	3-grams	\$co, com, omp, mpu, put, ute, ter, er\$	
	4-grams	\$com, comp, ompu, mput, pute, uter, ter\$	
	5-grams	\$comp, compu, omput, mpute, puter, uter\$	
	6-grams	\$compu, comput, ompute, mputer, puter\$	Not space efficient
	7-grams	\$comput, compute, omputer, mputer\$	
...			

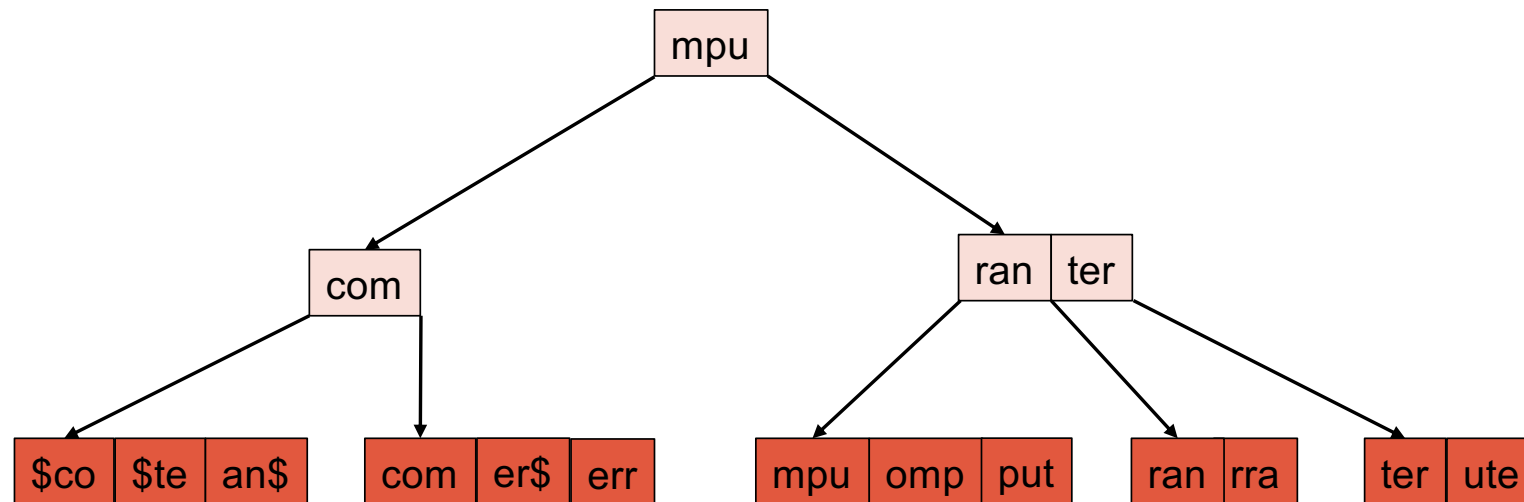
k-gram

computer	\$co	com	omp	mpu	put	ute	ter	er\$
terran	\$te	ter	err	rra	ran	an\$		

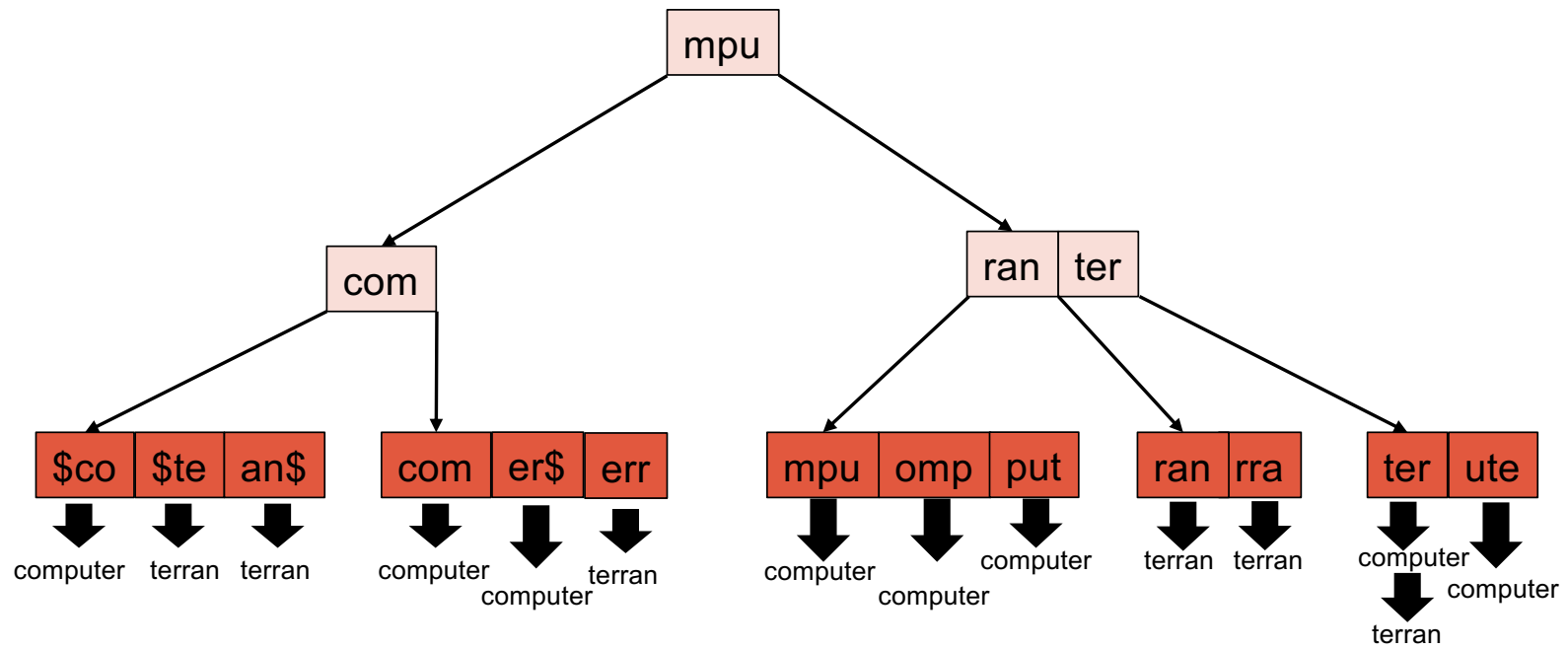
k-gram index



k-gram index

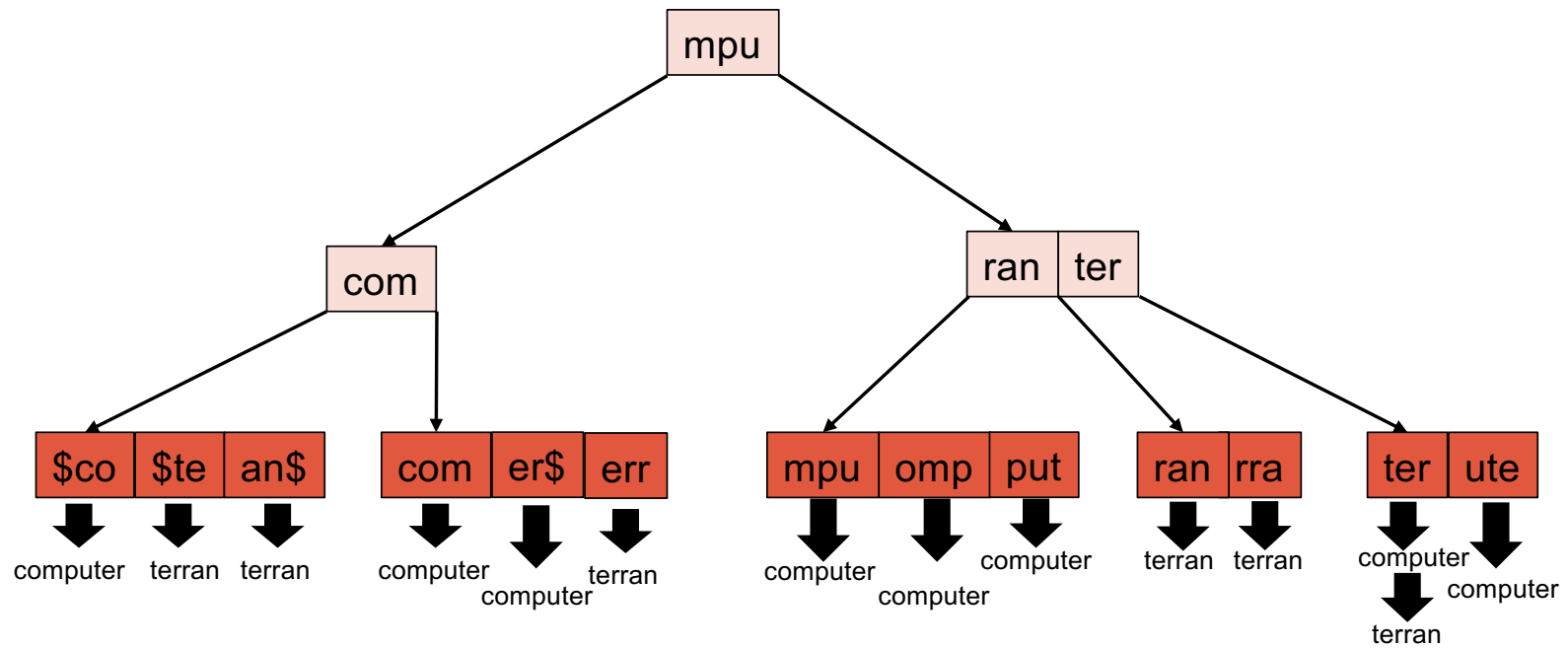


k-gram index



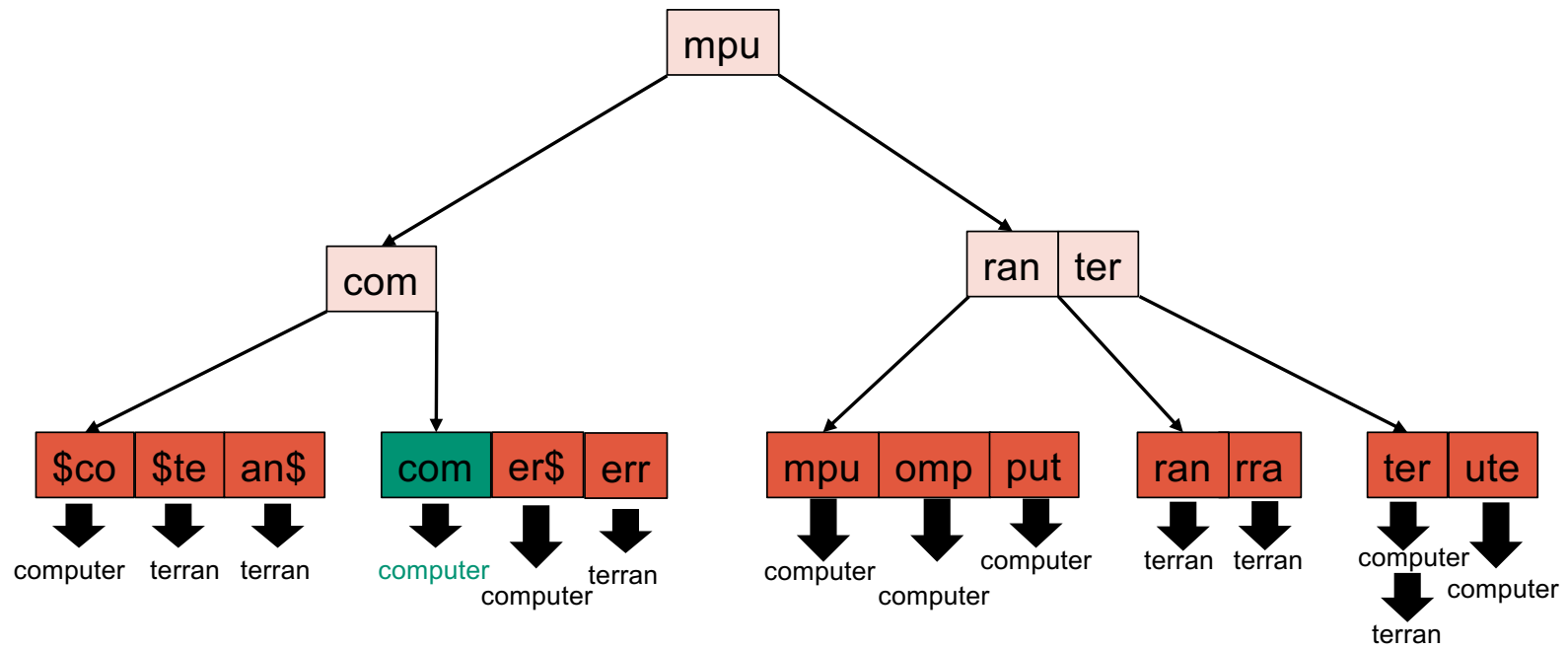
k-gram index

com



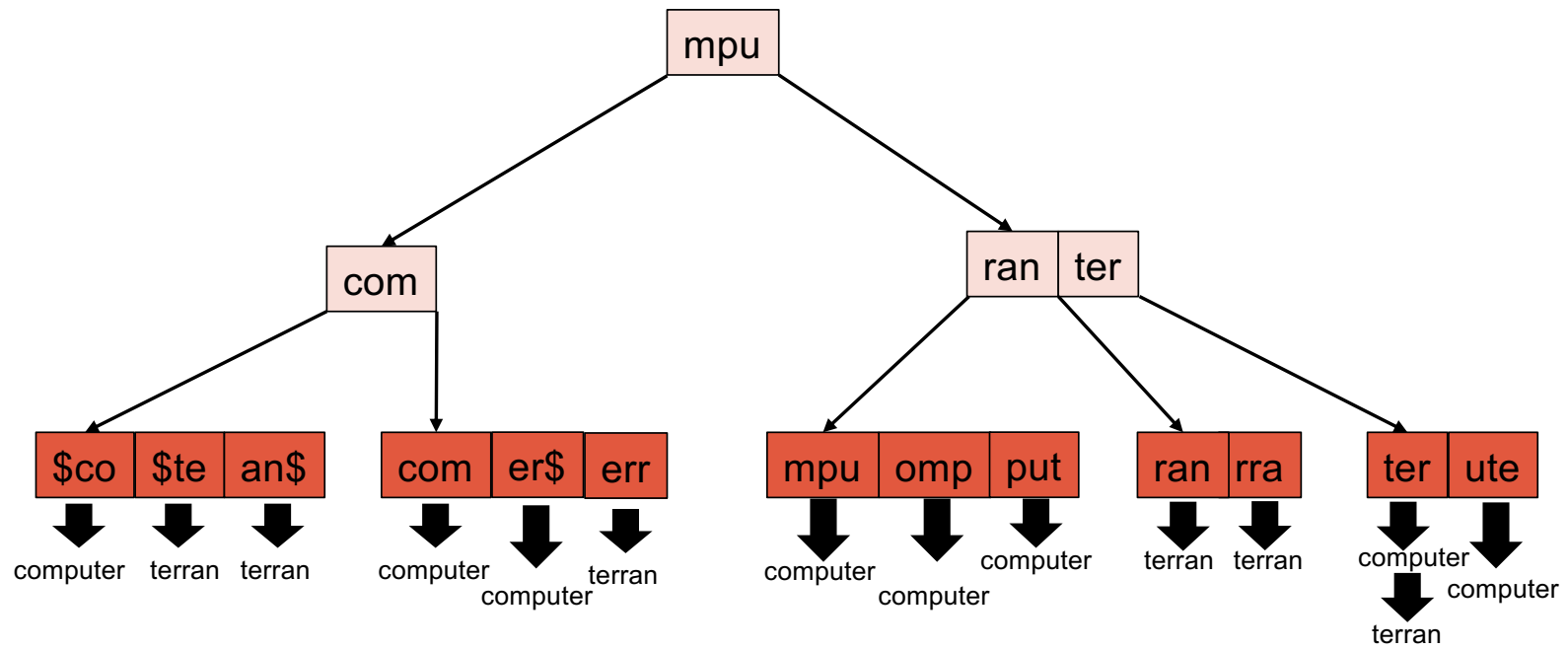
k-gram index

com



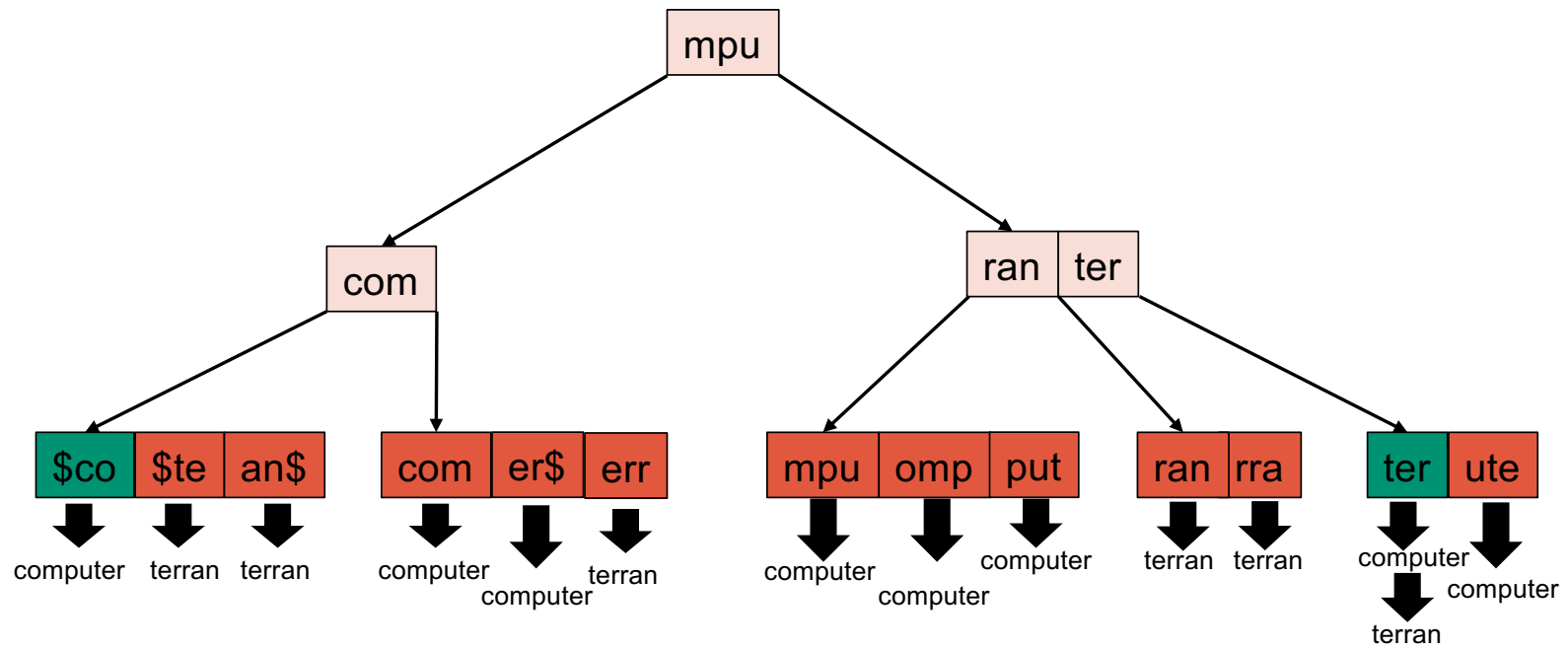
k-gram index

co*ter



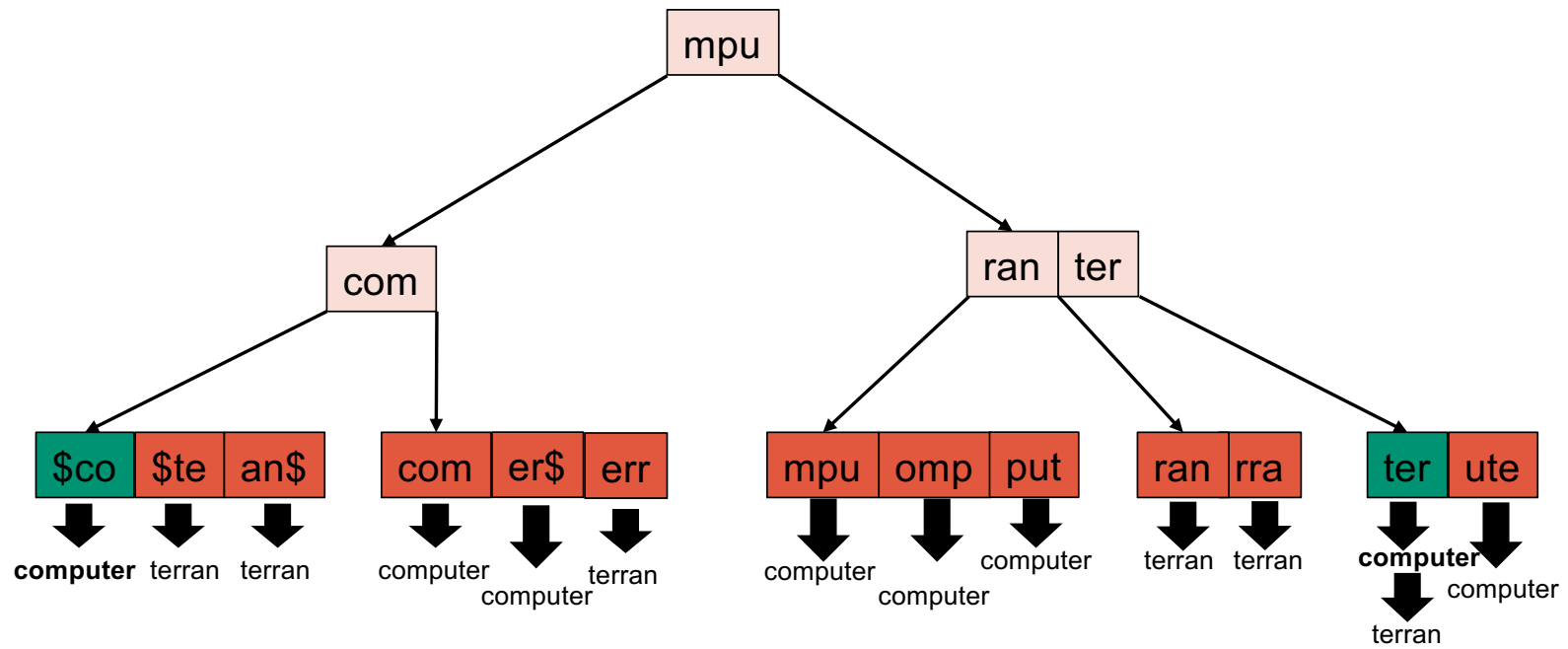
k-gram index

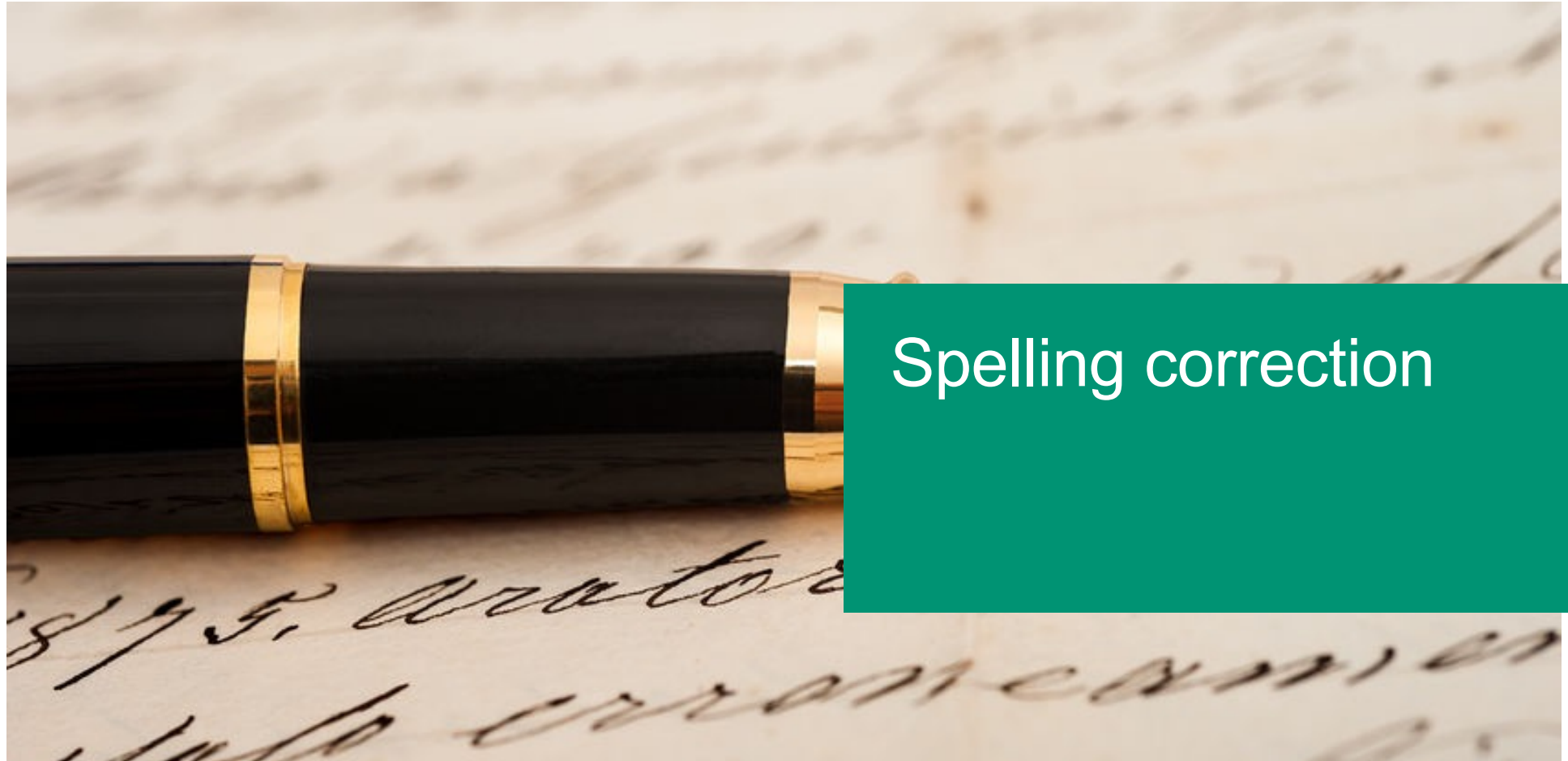
co*ter



k-gram index

co*ter





Spelling correction

Approaches to spelling errors

Approaches to spelling errors

Always query also for corrected terms

Approaches to spelling errors

Only query for corrected terms
if not in the dictionary

Approaches to spelling errors

Only query for corrected terms
if not in the dictionary
and not enough results

Approaches to spelling errors

Making a **spelling suggestion** for corrected terms
if not in the dictionary
and not enough results

Minimum Edit Distance

t o r

t o

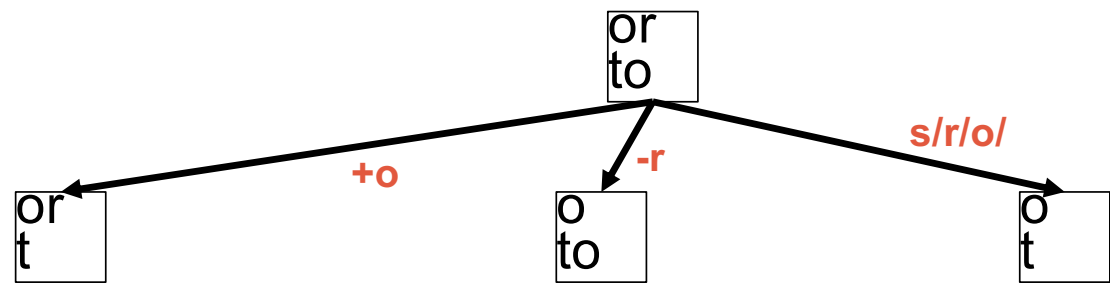
Minimum Edit Distance

o r
t o

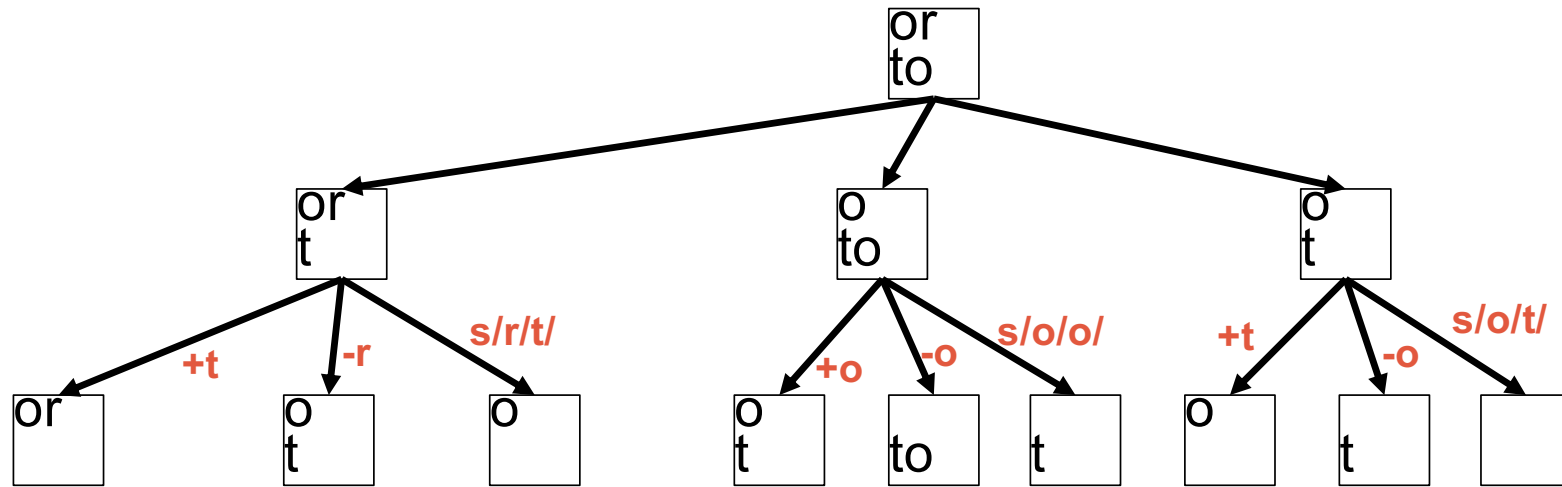
Search Space

or
to

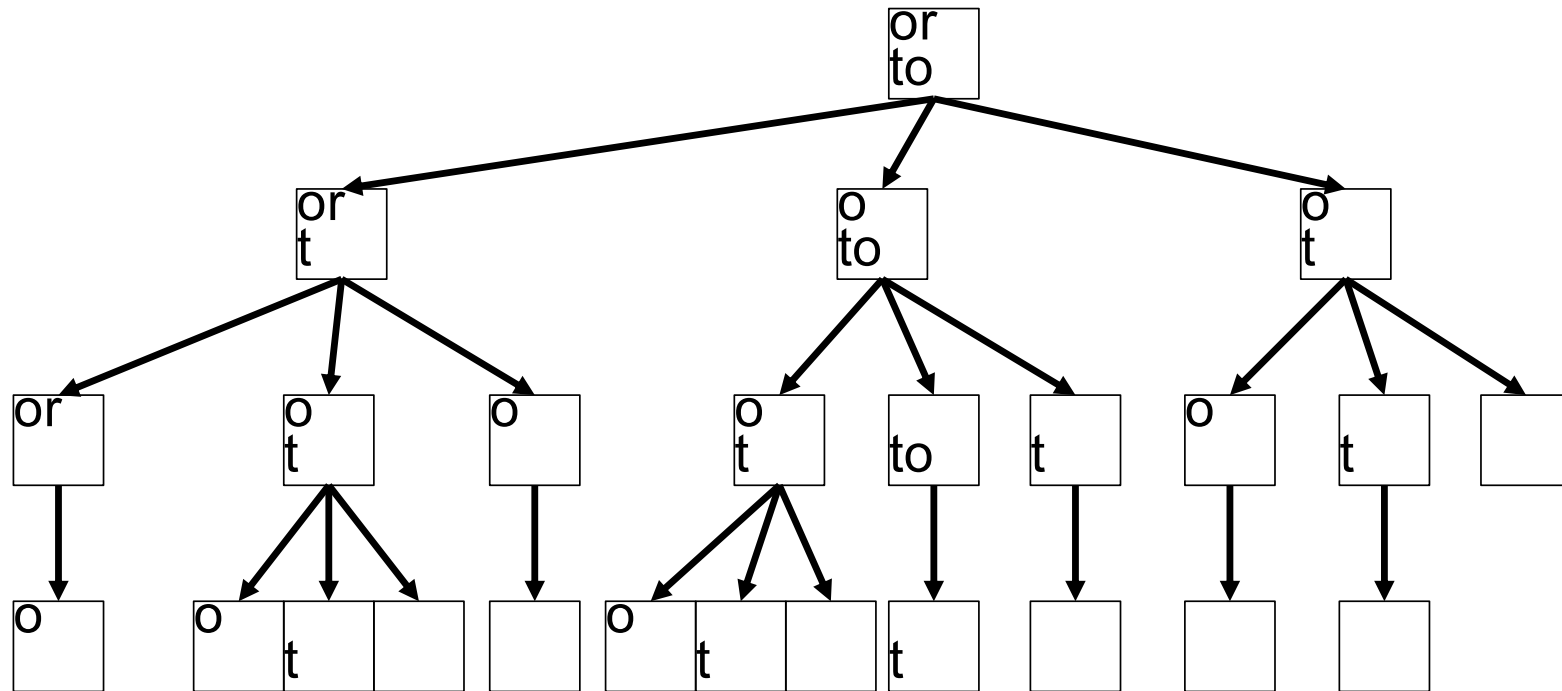
Search Space



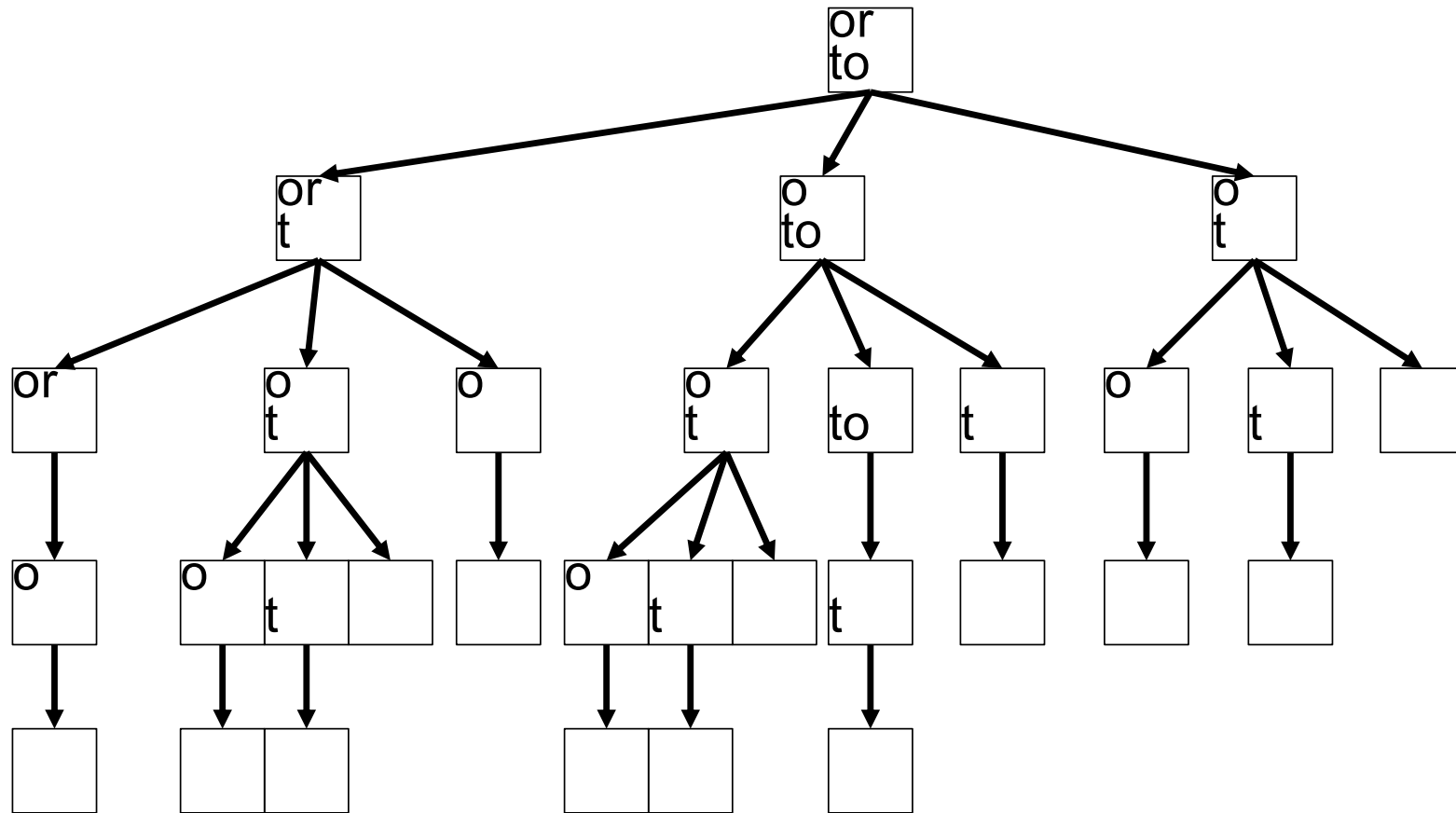
Search Space



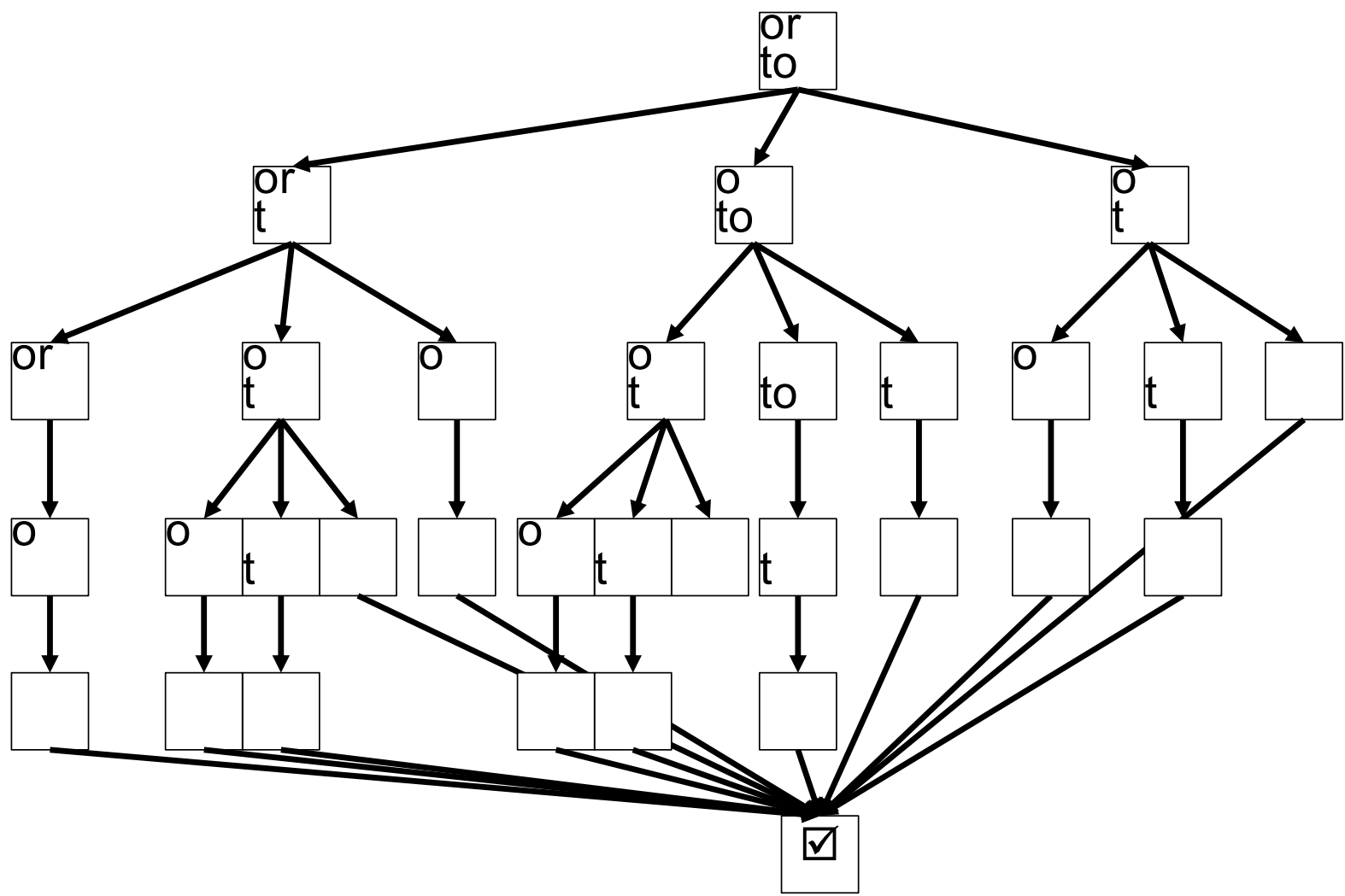
Search Space



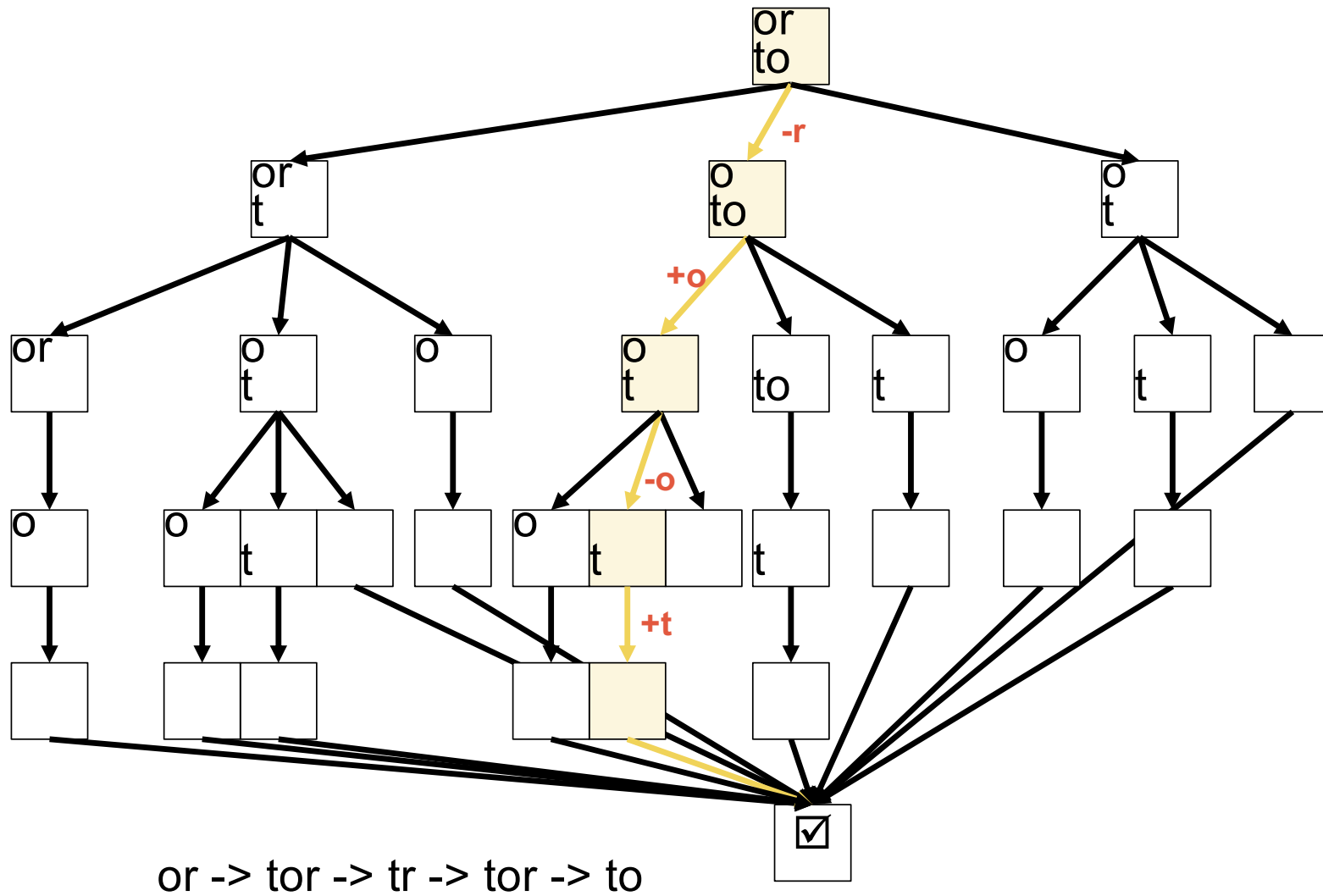
Search Space



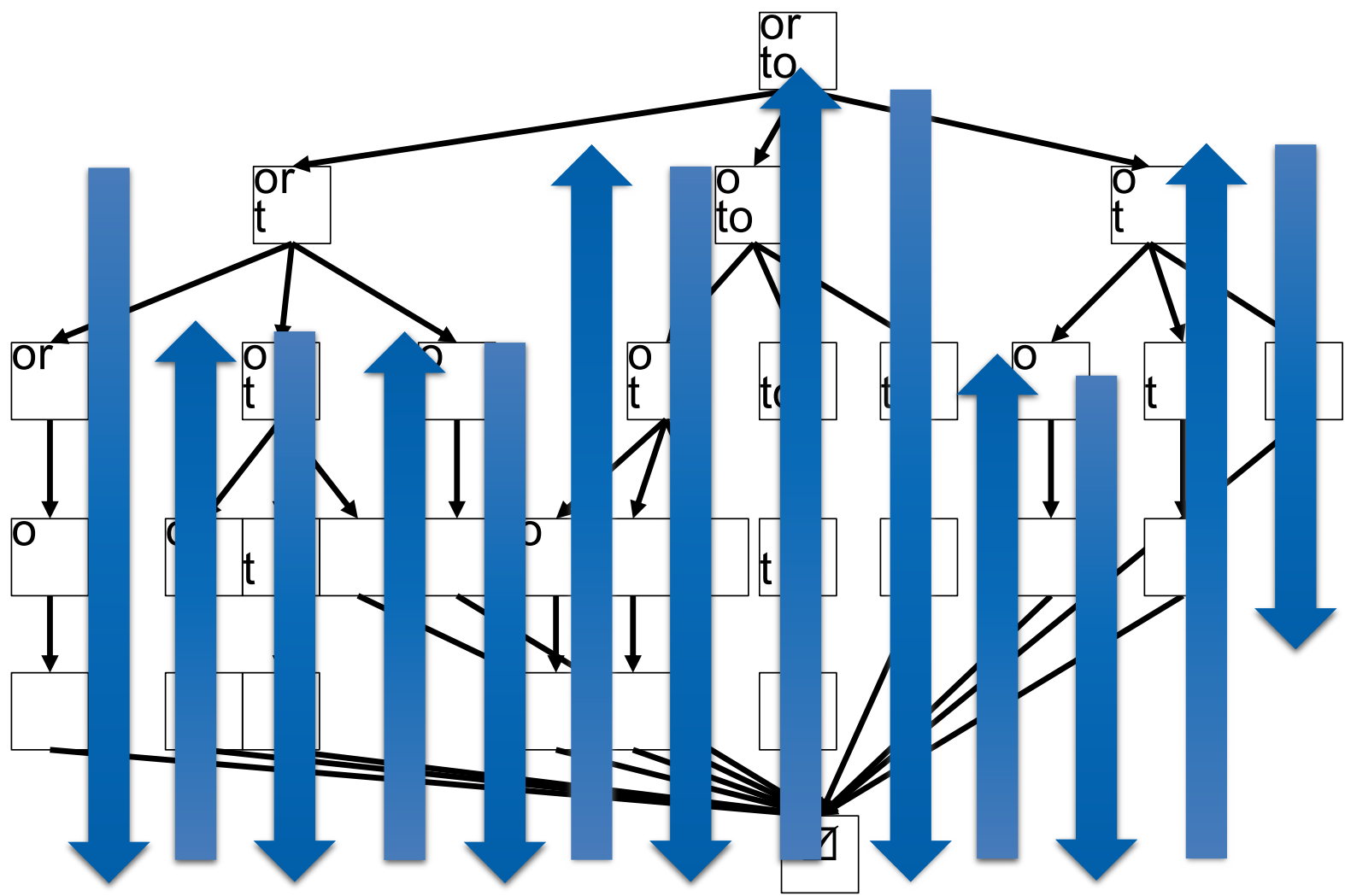
Search Space



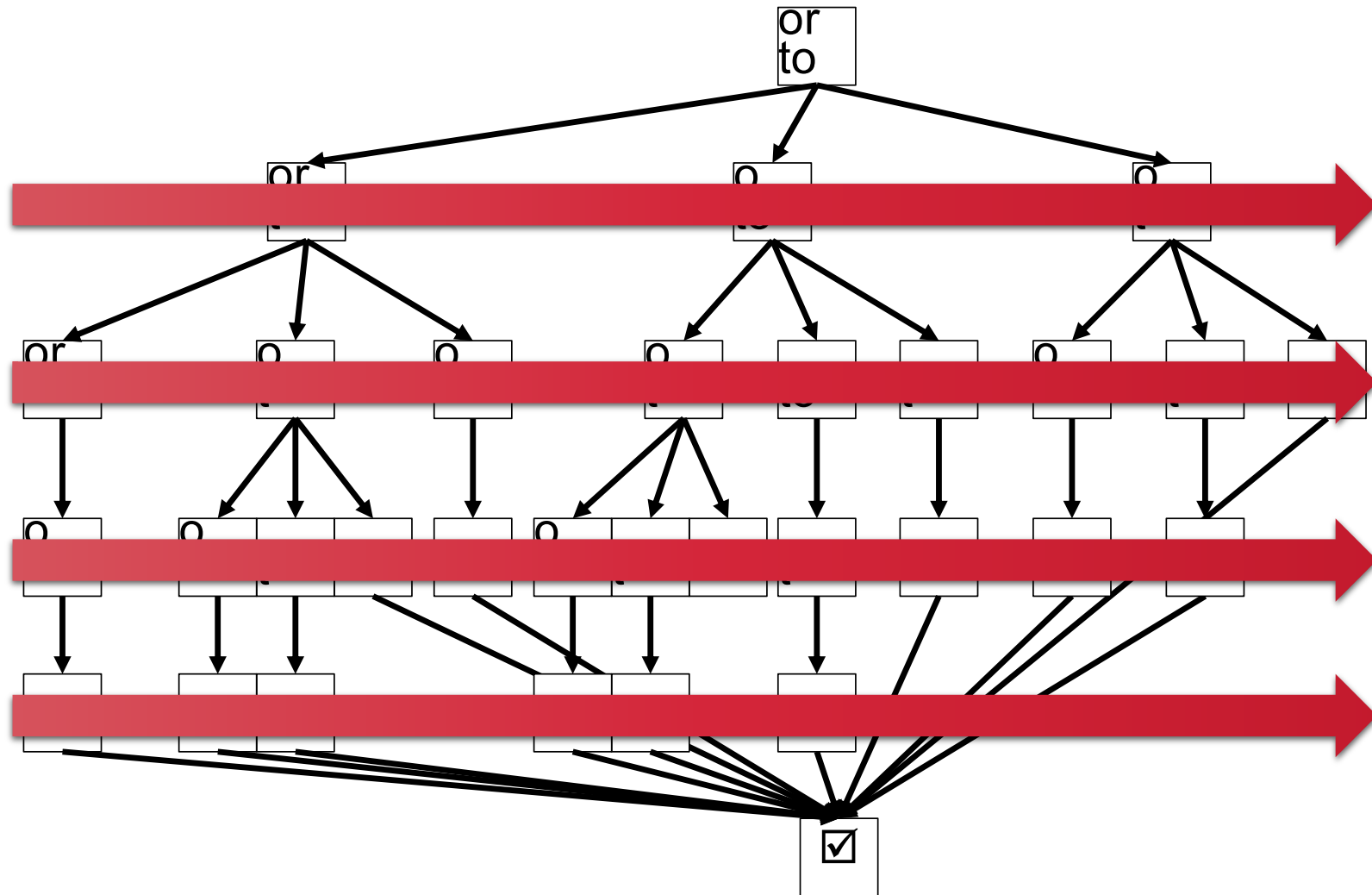
Search Space



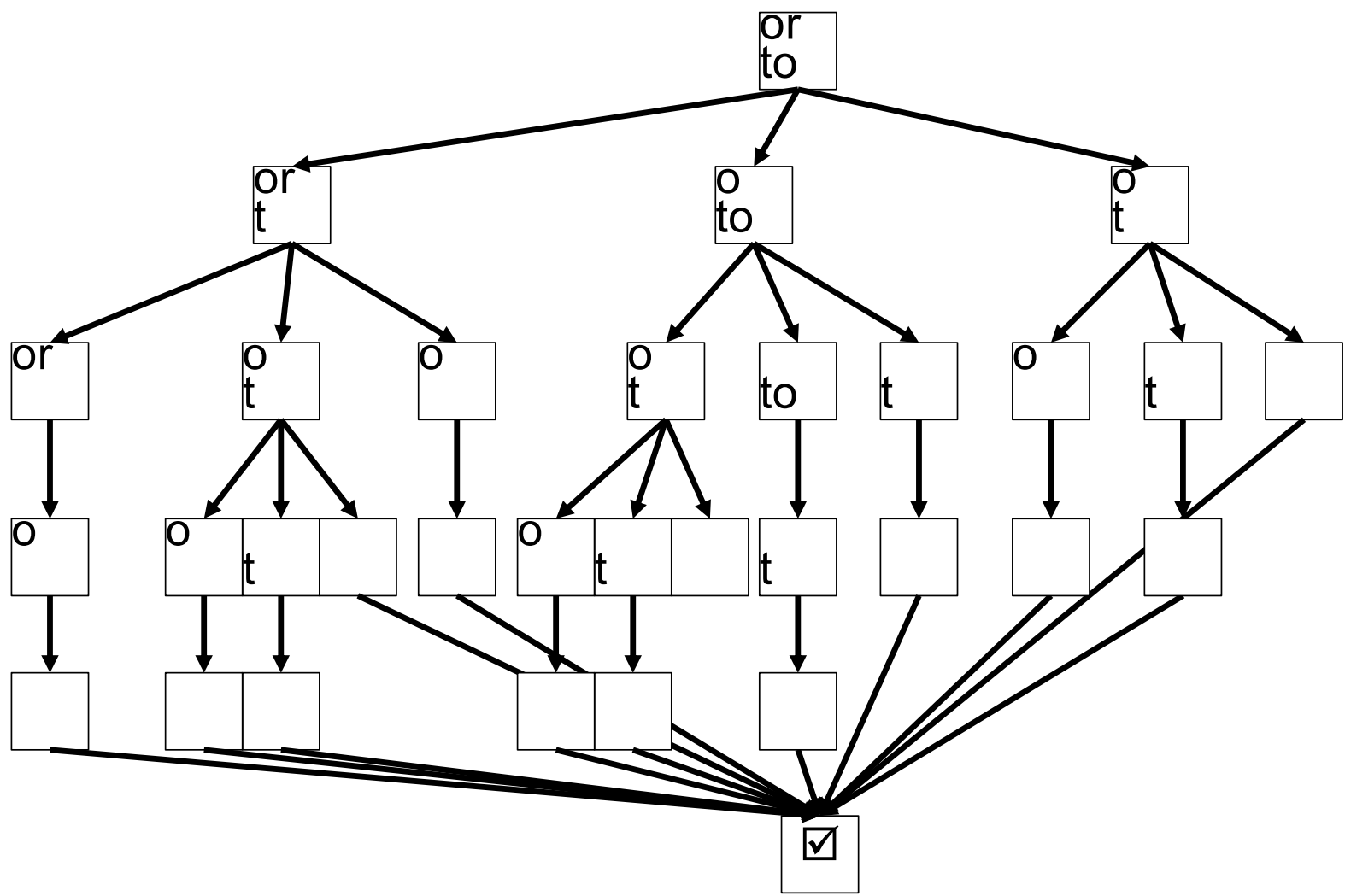
Depth-First-Search



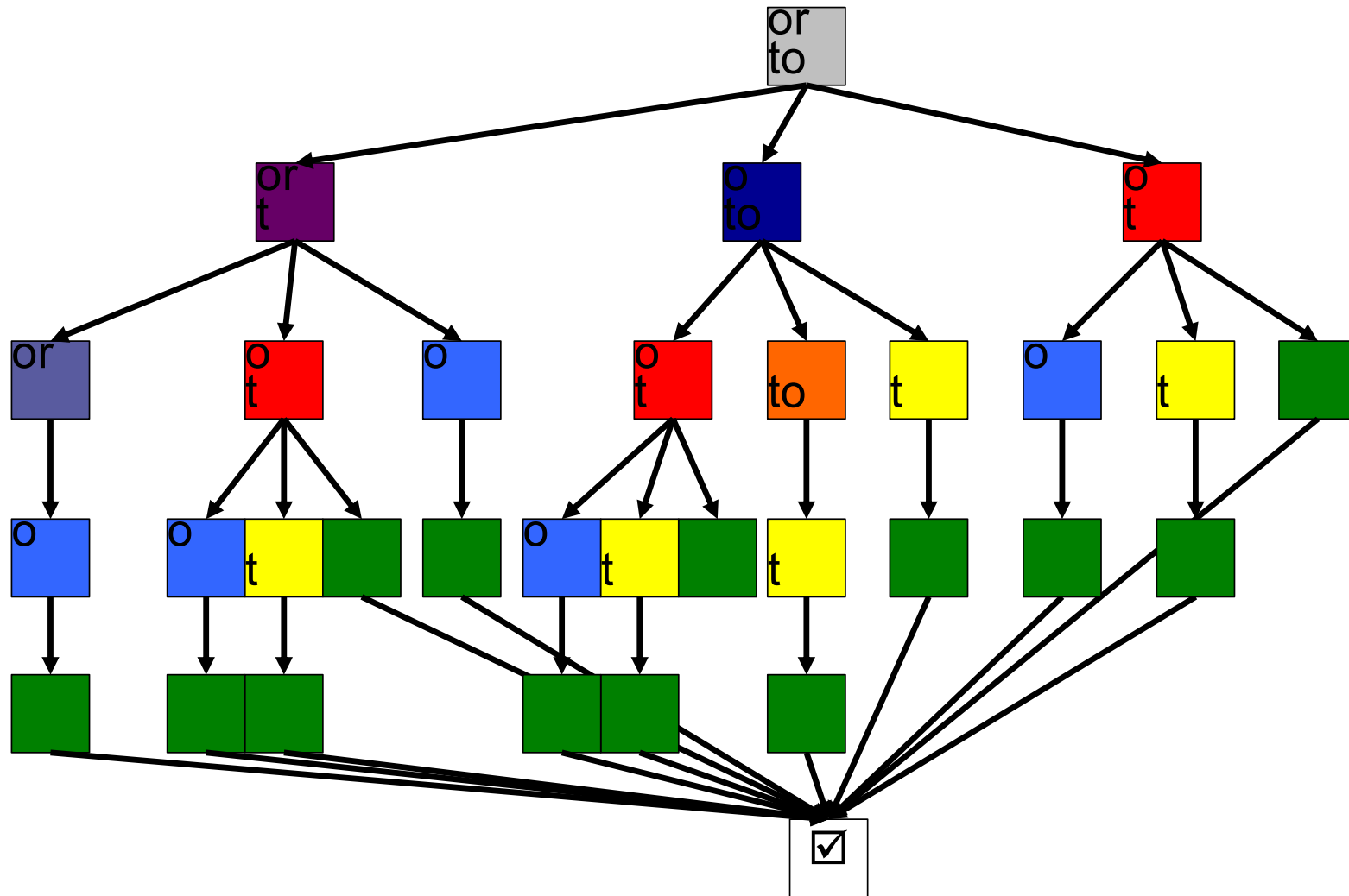
Breadth-First-Search



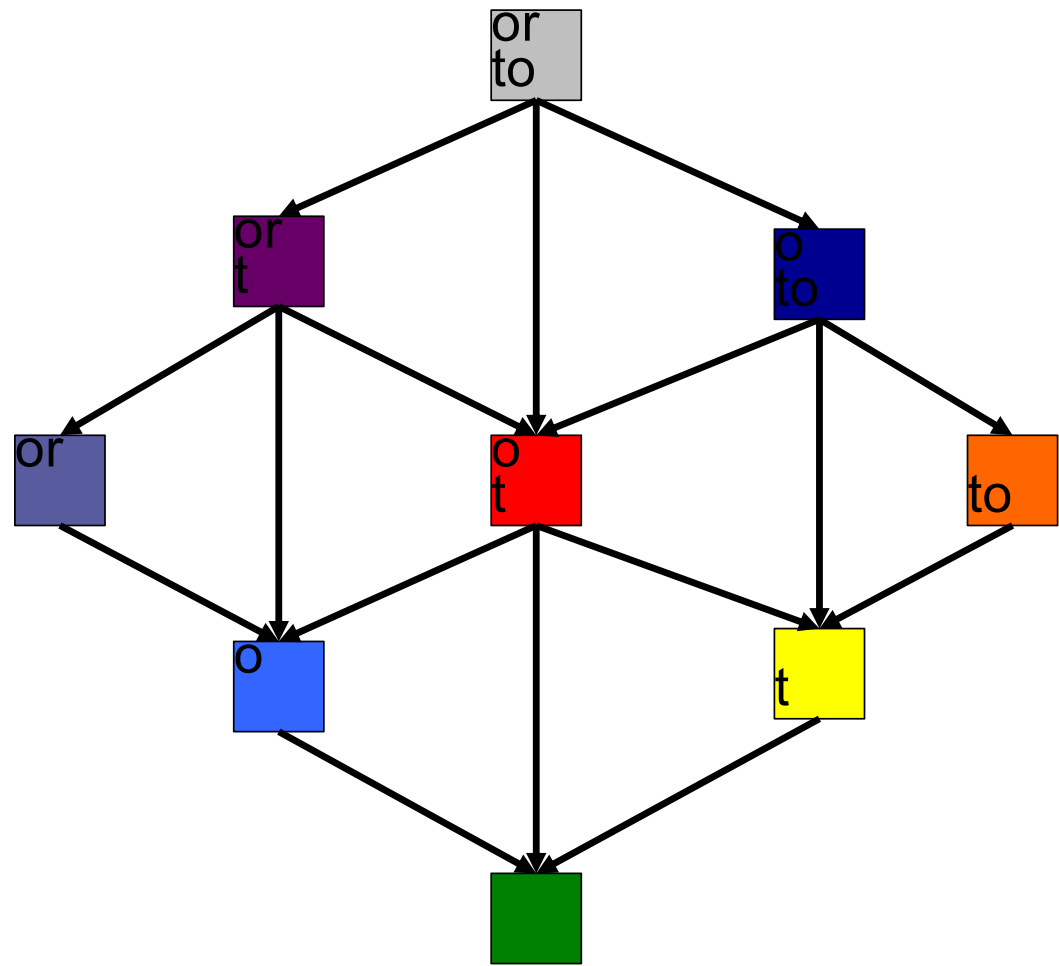
Search Space



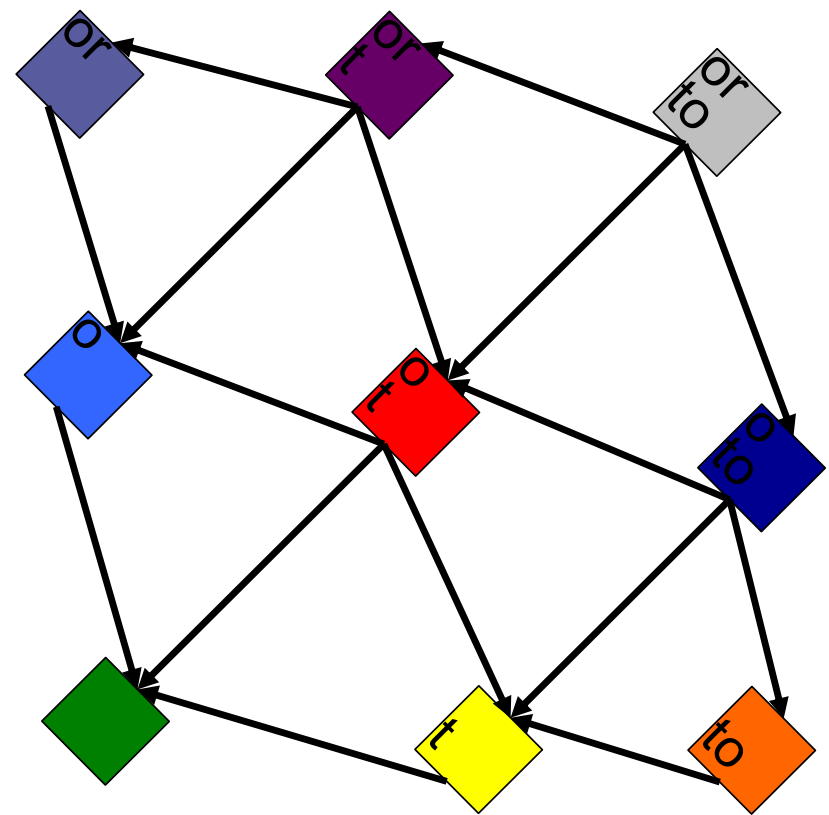
Search Space



Search Space



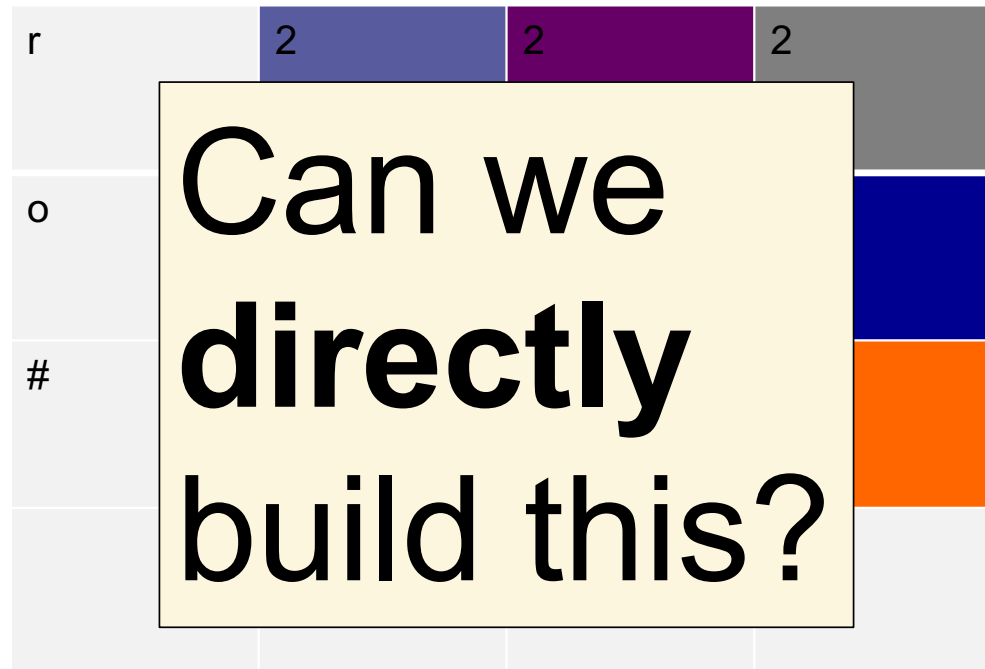
Search Space



Dynamic Programming

r	2	2	2
o	1	1	1
#	0	1	2
	#	t	o

Dynamic Programming



Dynamic Programming

	#	a	t	e
#	0			
c				
a				
t				

Dynamic Programming

	#	a	t	e
#	0	1	2	3
c	1			
a	2			
t	3			

Dynamic Programming

	#	a	t	e
#	0	1	2	3
c	1			
a	2			
t	3			

Diagram illustrating a dynamic programming table for sequence alignment. The table has rows labeled #, c, a, t and columns labeled #, a, t, e. The cell at row #, column a contains the value 1. Three arrows indicate transitions:

- A diagonal arrow from the cell at row #, column a to the cell at row c, column a, labeled "+1 (substitute a for c)".
- A horizontal arrow from the cell at row c, column # to the cell at row c, column a, labeled "+1 (add a)".
- A vertical arrow from the cell at row #, column a to the cell at row #, column t, labeled "+1 (add c)".

Dynamic Programming

	#	a	t	e
#	0	1	2	3
c	1	min(2, 1, 2)		
a	2			
t	3			

Diagram illustrating the dynamic programming table for the edit distance between "c" and "ate".

The table shows the cost of operations (insertion, deletion, substitution) for each character in the strings "c" and "ate".

Operations and their costs:

- +1 (substitute a for c) (from cell (0,1) to (1,2))
- +1 (remove c) (from cell (1,1) to (1,2))
- +1 (add a) (from cell (1,1) to (2,1))

Dynamic Programming

	#	a	t	e
#	0	1	2	3
c	1	1		
a	2			
t	3			

Dynamic Programming

	#	a	t	e
#	0	1	2	3
c	1	1	2	
a	2			
t	3			

Dynamic Programming

	#	a	t	e
#	0	1	2	3
c	1	1	2	3
a	2			
t	3			

Dynamic Programming

	#	a	t	e
#	0	1	2	3
c	1	1	2	3
a	2			
t	3			

The diagram illustrates the dynamic programming table for the word "cater". The table has rows for characters and columns for indices. The current cell being processed is at row 'a', column 2 (value 2). Three arrows indicate possible transitions:

- A diagonal arrow from (c, 1) to (a, 2) labeled "0 (do nothing)".
- A horizontal arrow from (a, 1) to (a, 2) labeled "+1 (add a)".
- A vertical arrow from (c, 2) to (a, 2) labeled "+1 (remove c)".

Dynamic Programming

	#	a	t	e
#	0	1	2	3
c	1	1	2	3
a	2	min(3, 1, 2)		
t	3			

Diagram illustrating the dynamic programming table for the edit distance between "cat" and "ate". The table shows the minimum number of operations (insertions, deletions, or substitutions) required to transform one string into the other.

Annotations for the cell at row 'a', column 'a' (value 2):

- 0 (do nothing) (from row 'c', column 'a')
- +1 (remove c) (from row 'c', column 't')
- +1 (add a) (from row 'a', column '#')

Dynamic Programming

	#	a	t	e
#	0	1	2	3
c	1	1	2	3
a	2	1		
t	3			

Dynamic Programming

	#	a	t	e
#	0	1	2	3
c	1	1	2	3
a	2	1	2	
t	3			

Dynamic Programming

	#	a	t	e
#	0	1	2	3
c	1	1	2	3
a	2	1	2	3
t	3			

Dynamic Programming

	#	a	t	e
#	0	1	2	3
c	1	1	2	3
a	2	1	2	3
t	3	2		

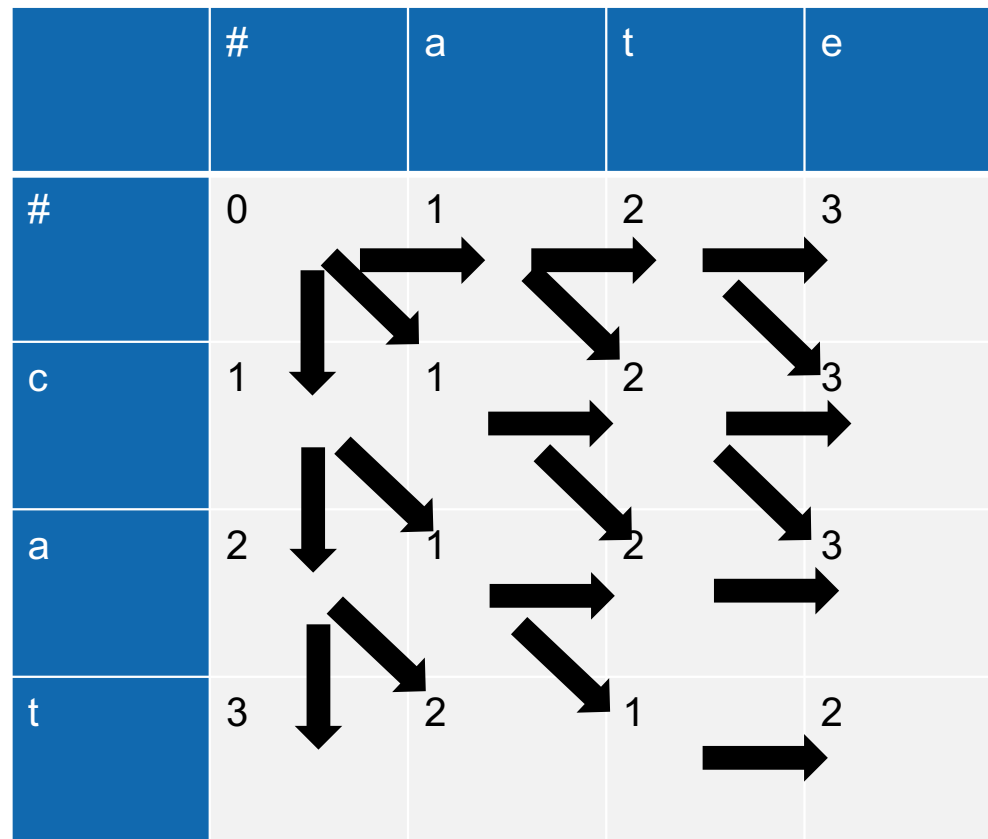
Dynamic Programming

	#	a	t	e
#	0	1	2	3
c	1	1	2	3
a	2	1	2	3
t	3	2	1	

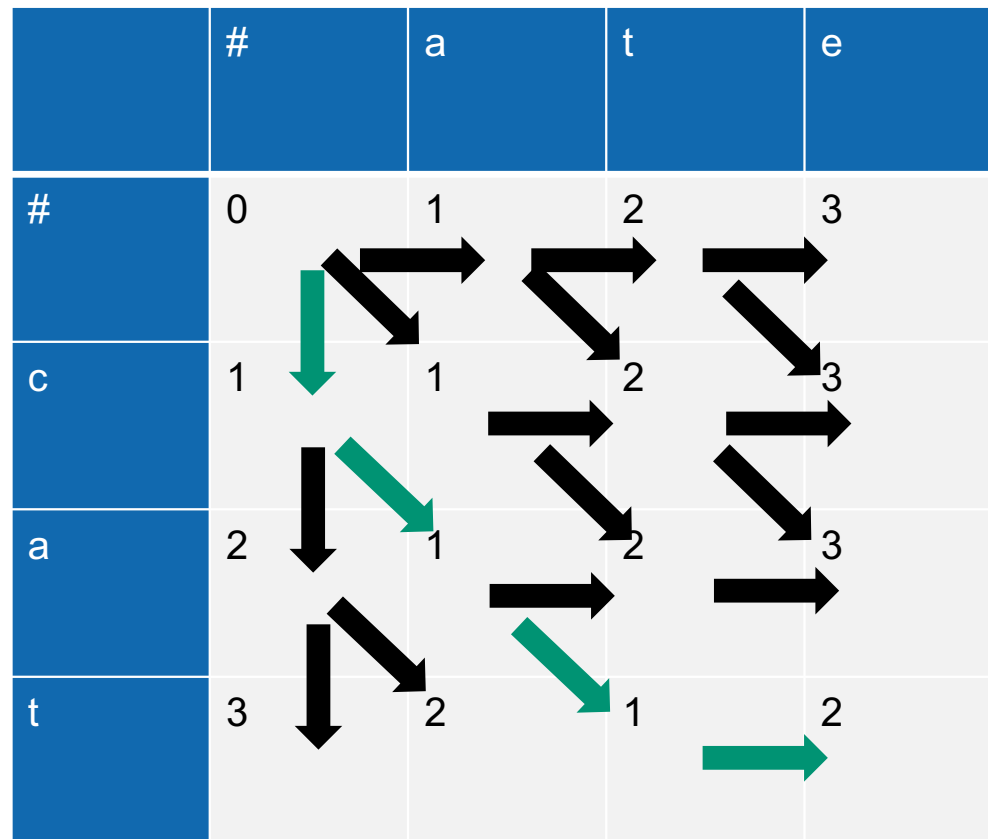
Dynamic Programming

	#	a	t	e
#	0	1	2	3
c	1	1	2	3
a	2	1	2	3
t	3	2	1	2

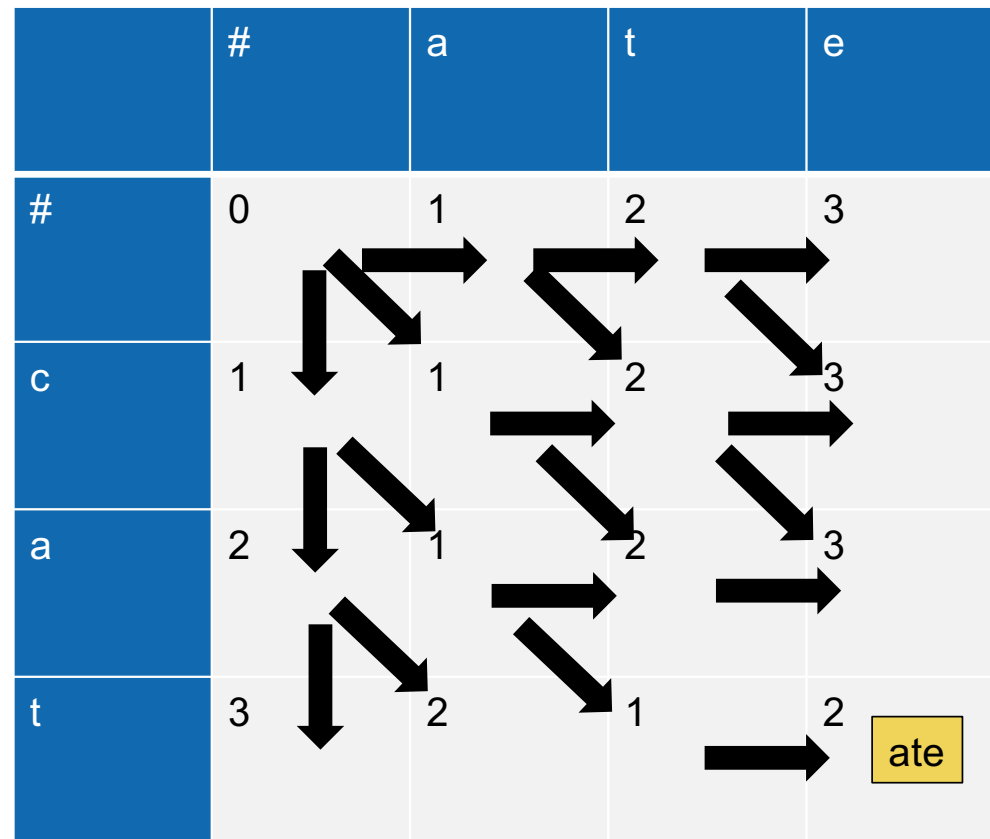
Dynamic Programming



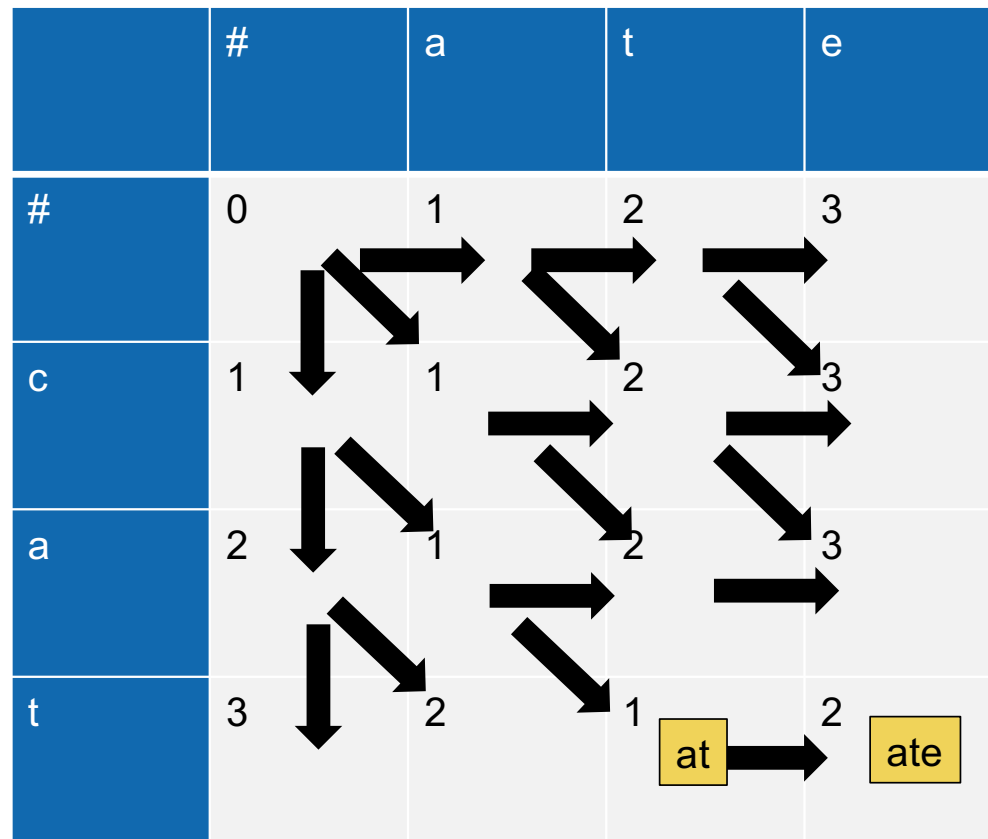
Dynamic Programming



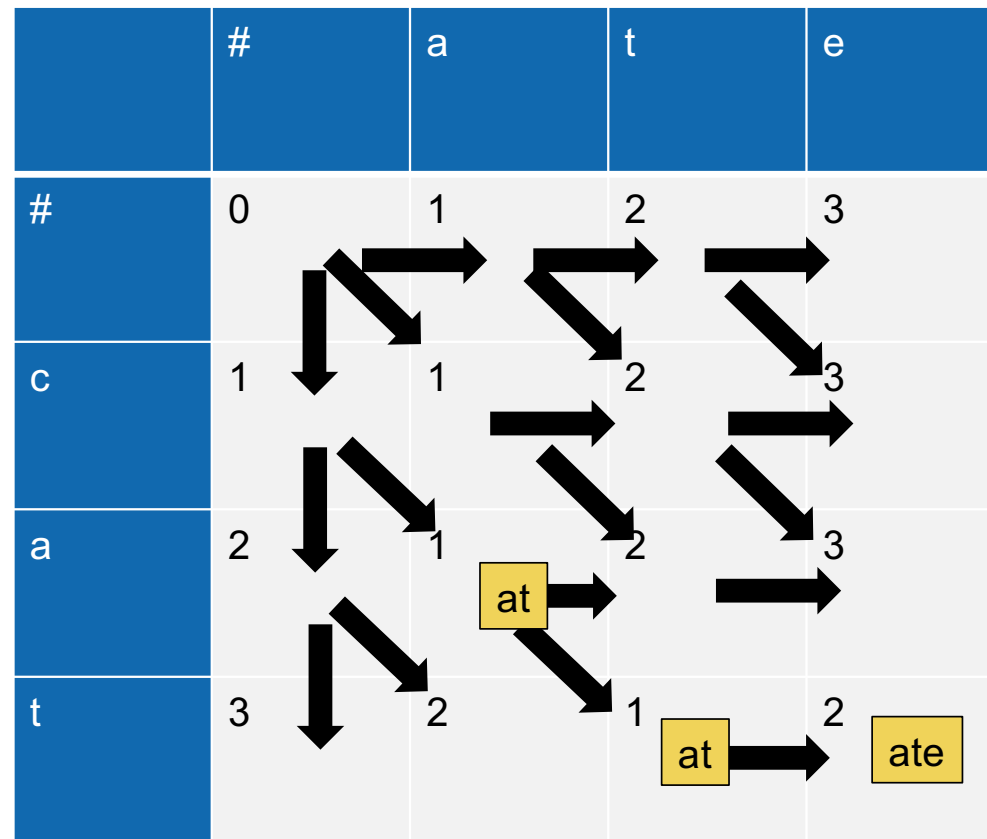
Dynamic Programming



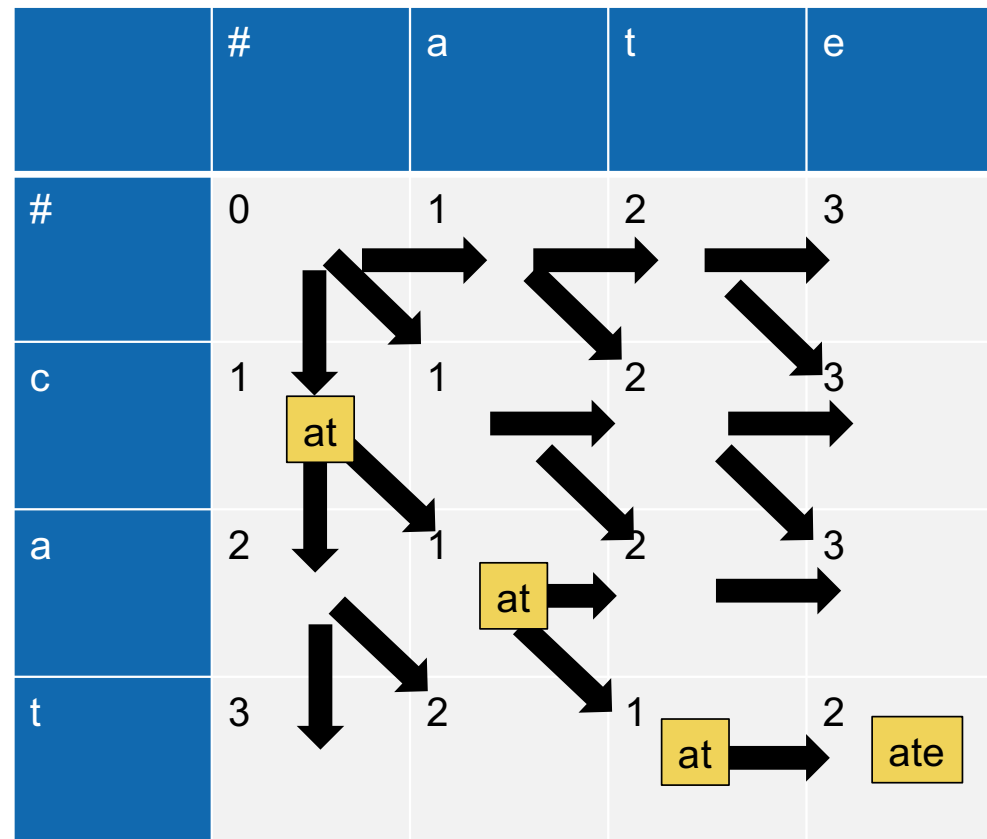
Dynamic Programming



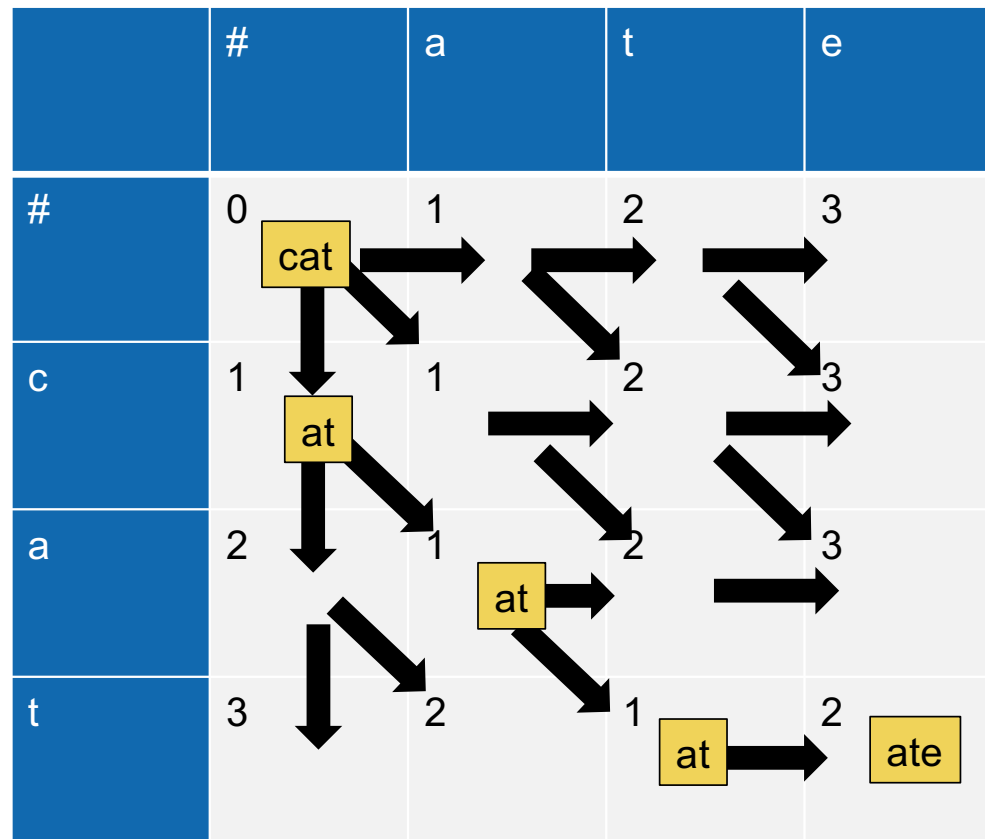
Dynamic Programming



Dynamic Programming



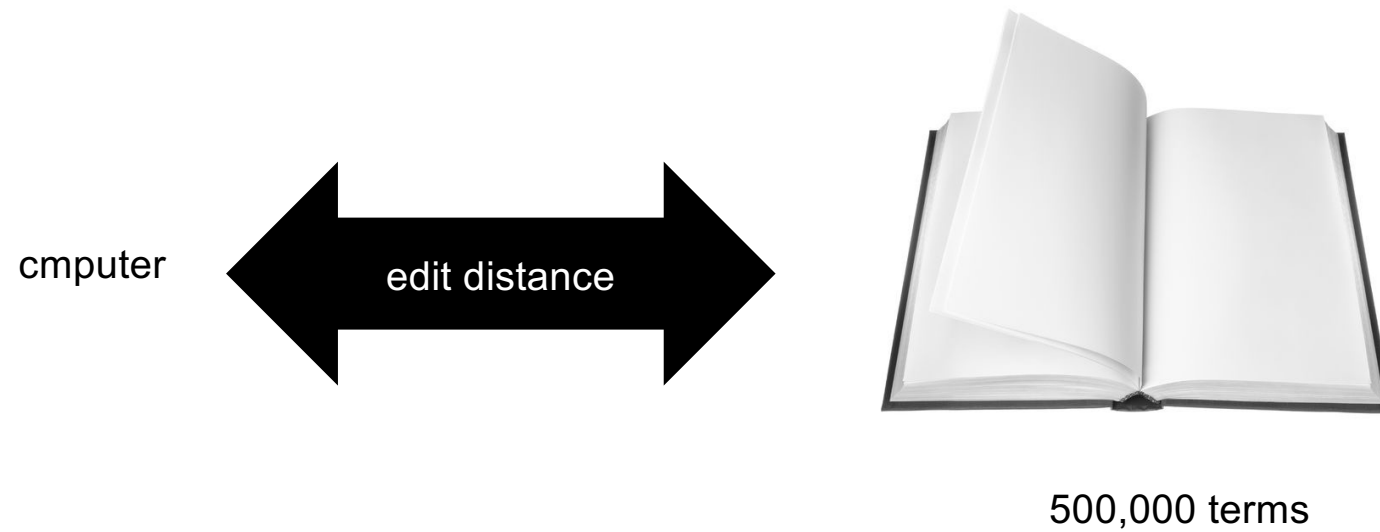
Dynamic Programming



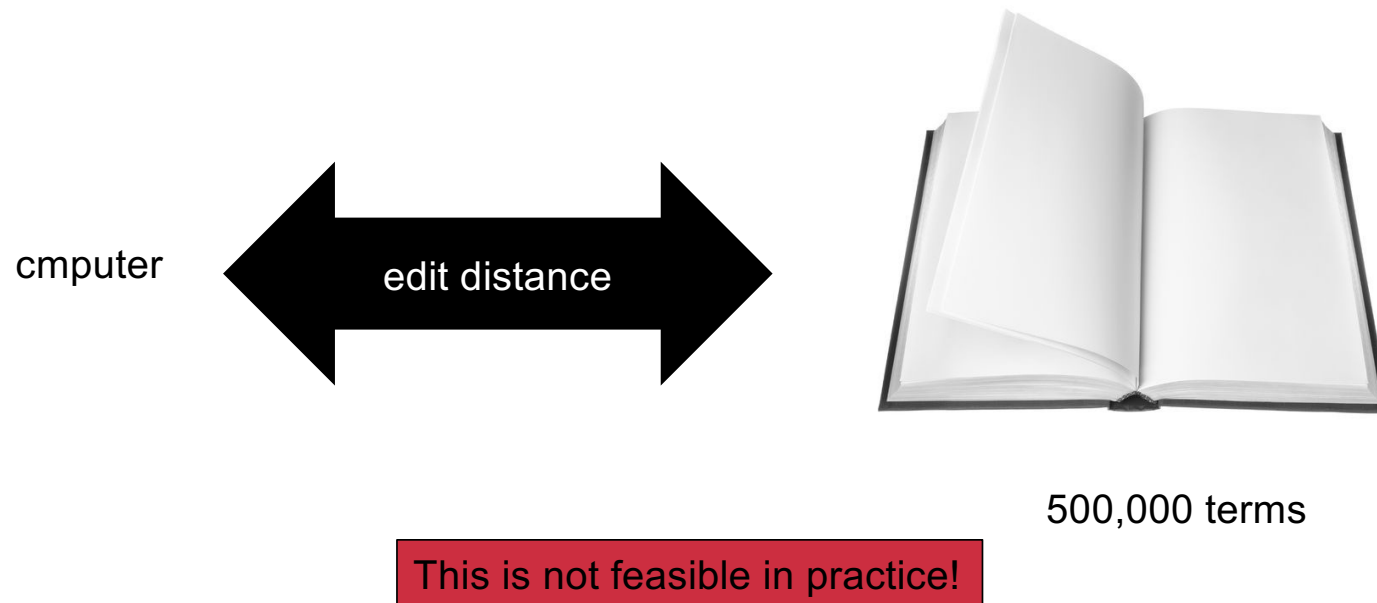
Problem with "raw" edit distance

cmputer

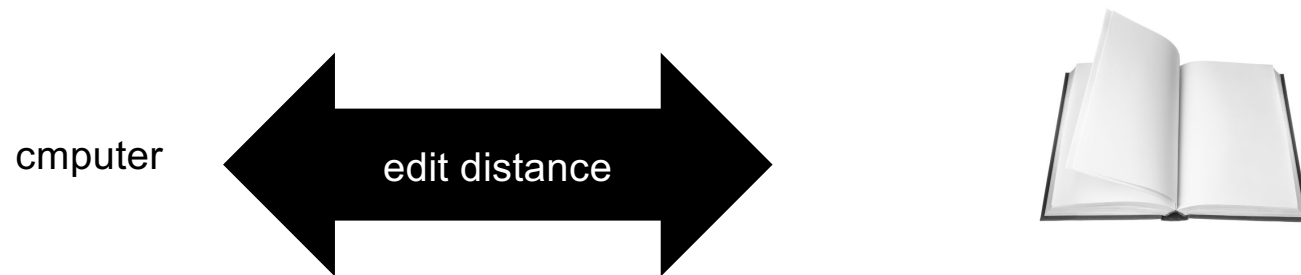
Problem with "raw" edit distance



Problem with "raw" edit distance



Problem with "raw" edit distance



just a few
terms terms

Solution: pre-select a few terms

Spelling and k-grams

Assumption:

Two terms

within a small edit distance

have many k-grams in common

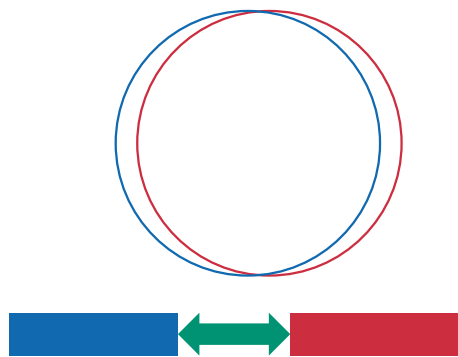
Spelling and k-grams

Assumption:

Two terms

within a small edit distance

have many k-grams in common



Spelling and k-grams

Assumption:

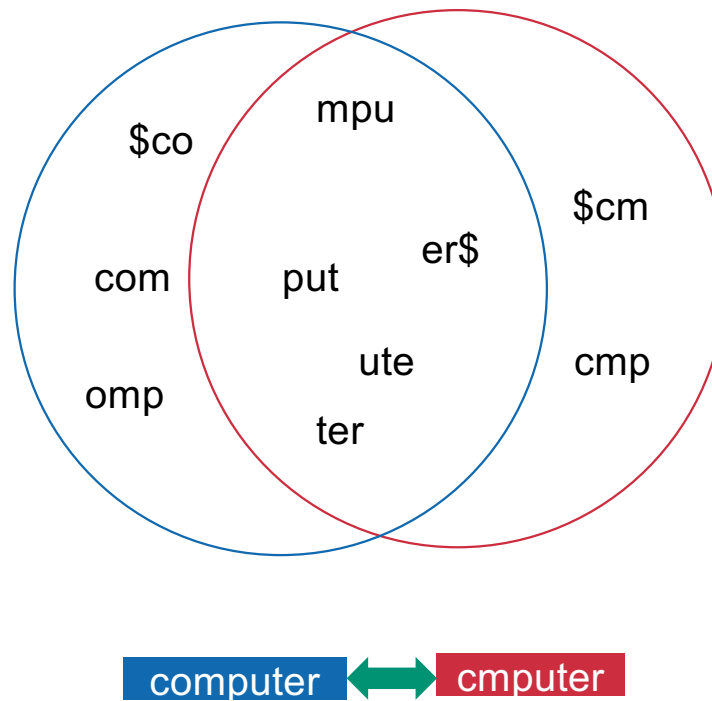
Two terms

within a small edit distance

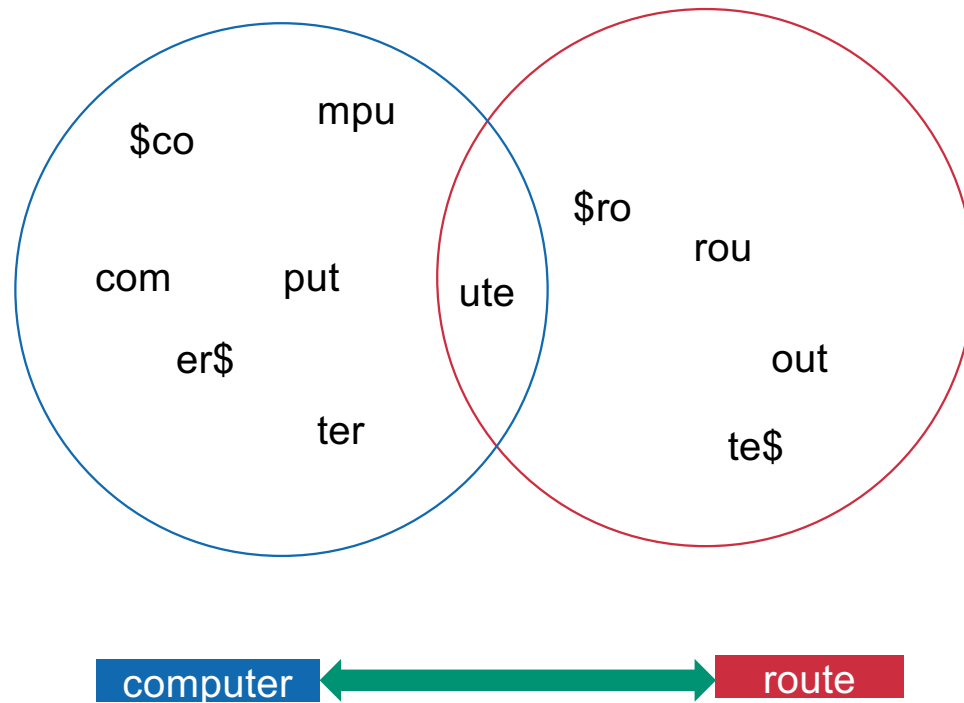
have many k-grams in common



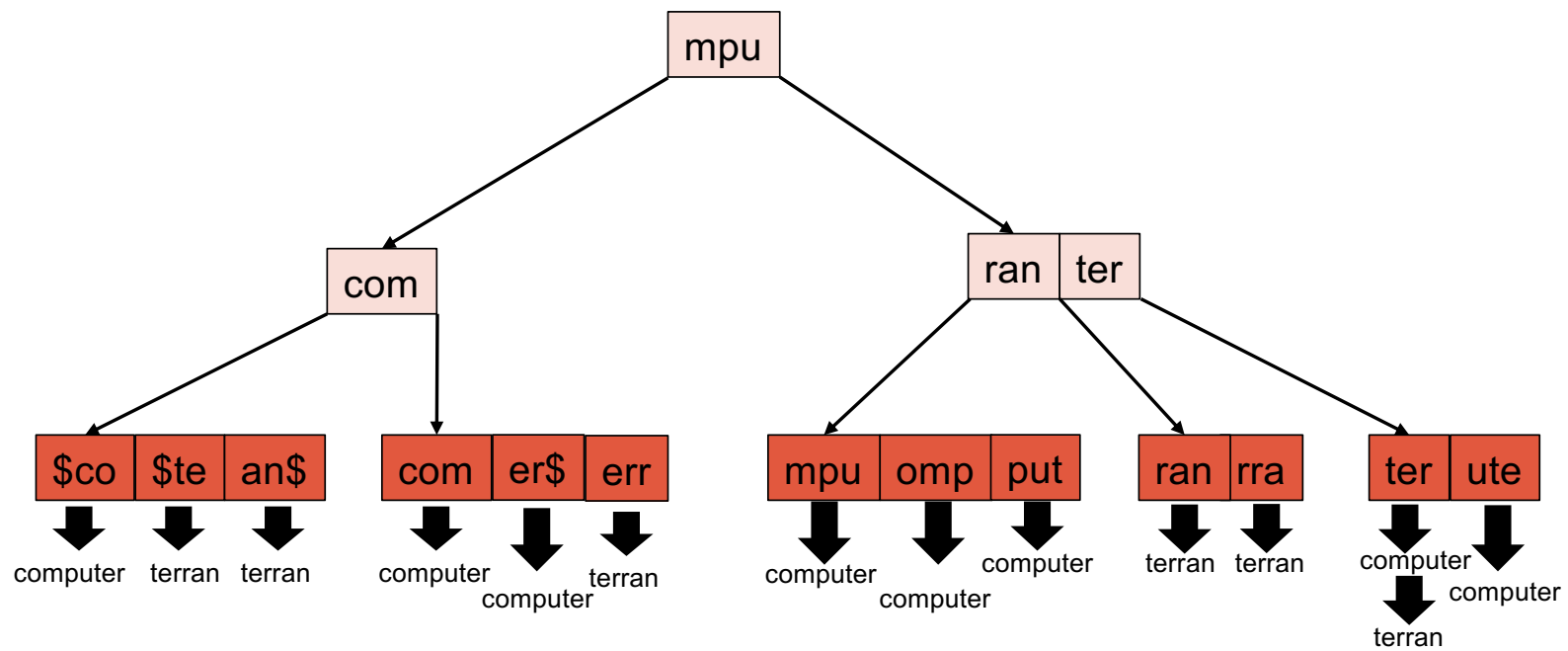
Spelling and k-grams



Spelling and k-grams

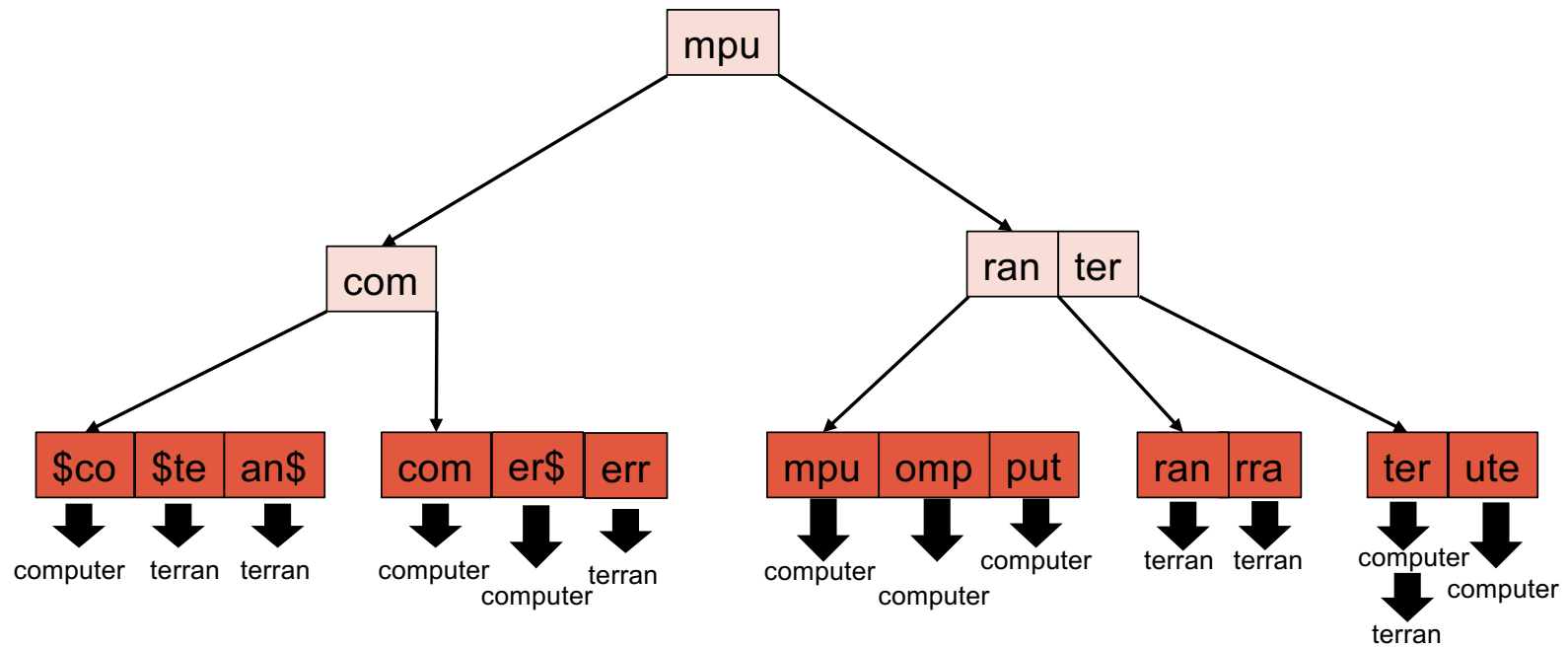


Spelling and k-grams



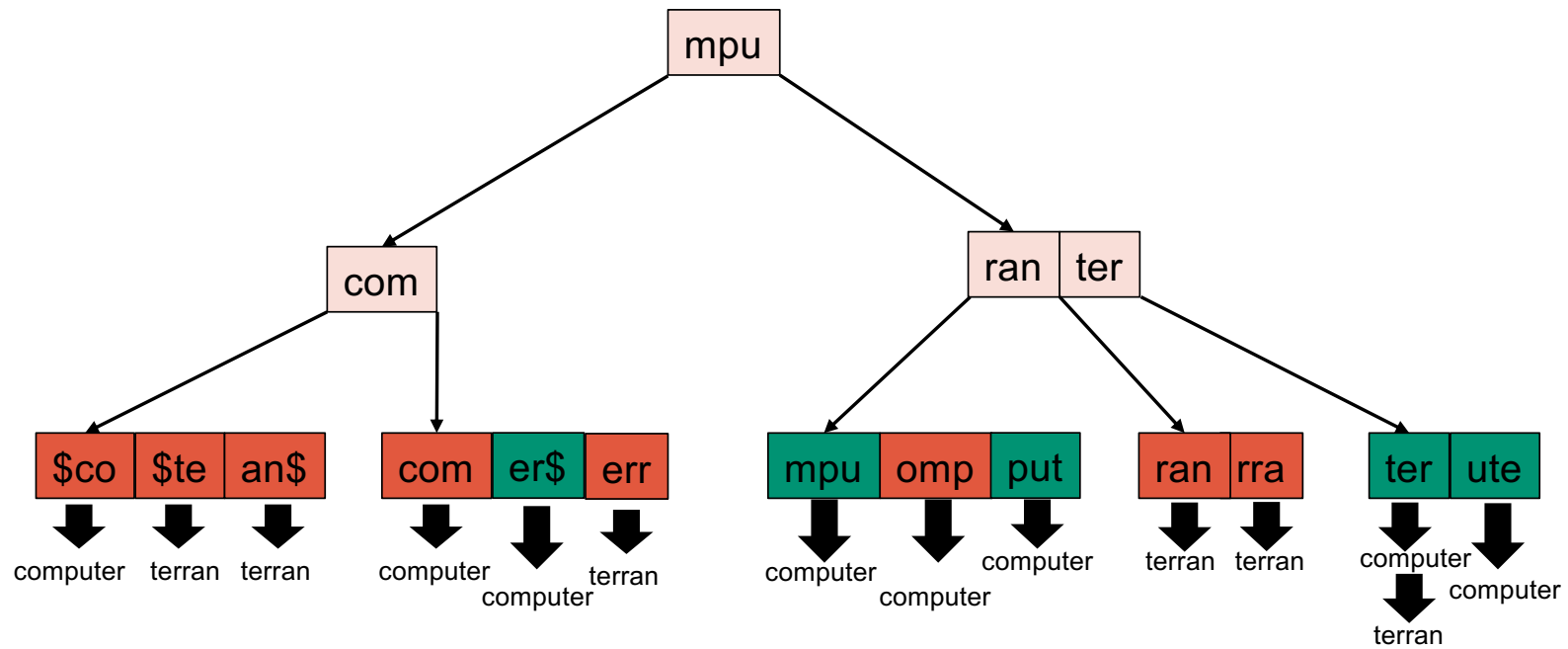
Spelling and k-grams

computer



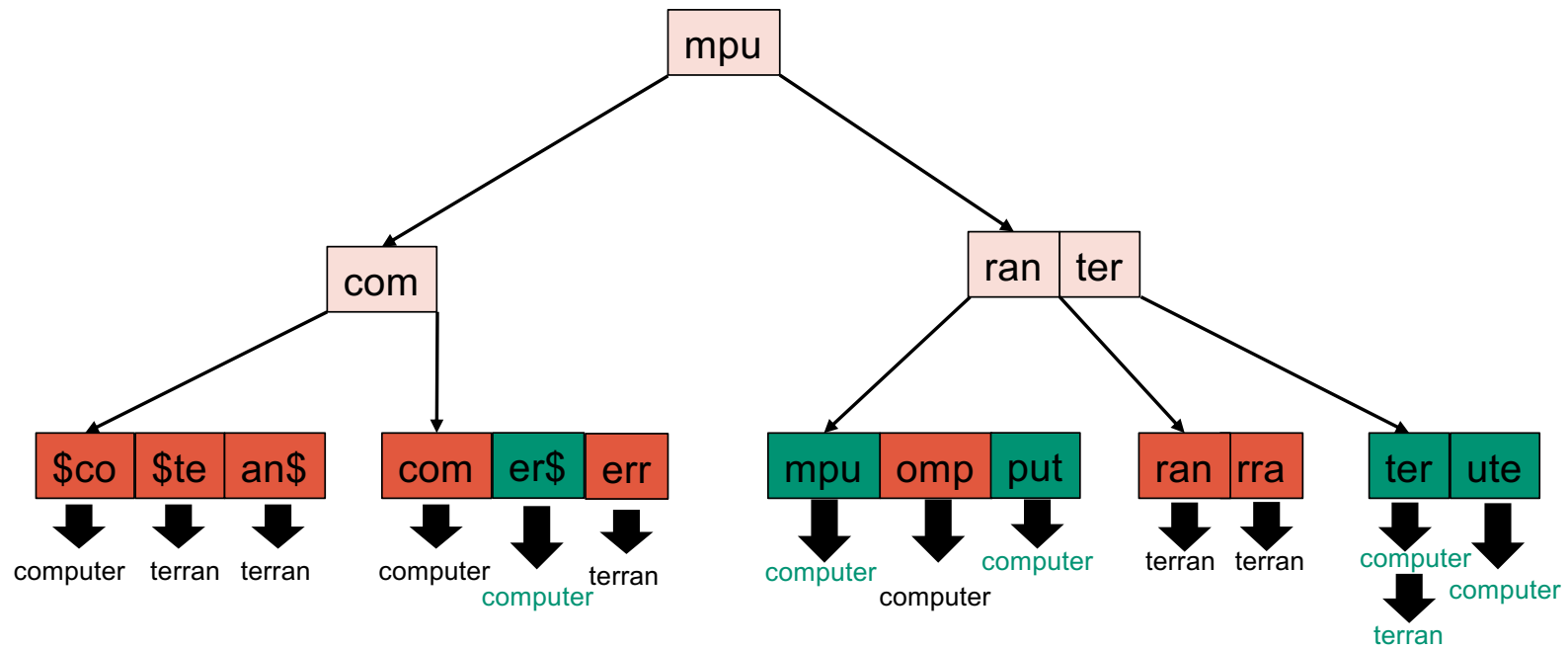
Spelling and k-grams

computer $\longrightarrow$ \$cm, cmp, mpu, put, ute, ter, er\$



Spelling and k-grams

computer $\longrightarrow$ \$cm, cmp, mpu, put, ute, ter, er\$



Spelling and k-grams

cmputer  \$cm, cmp, mpu, put, ute, ter, er\$

computer: five 3-grams in common

terran: one 3-gram in common

Spelling and k-grams

computer  \$cm, cmp, mpu, put, ute, ter, er\$

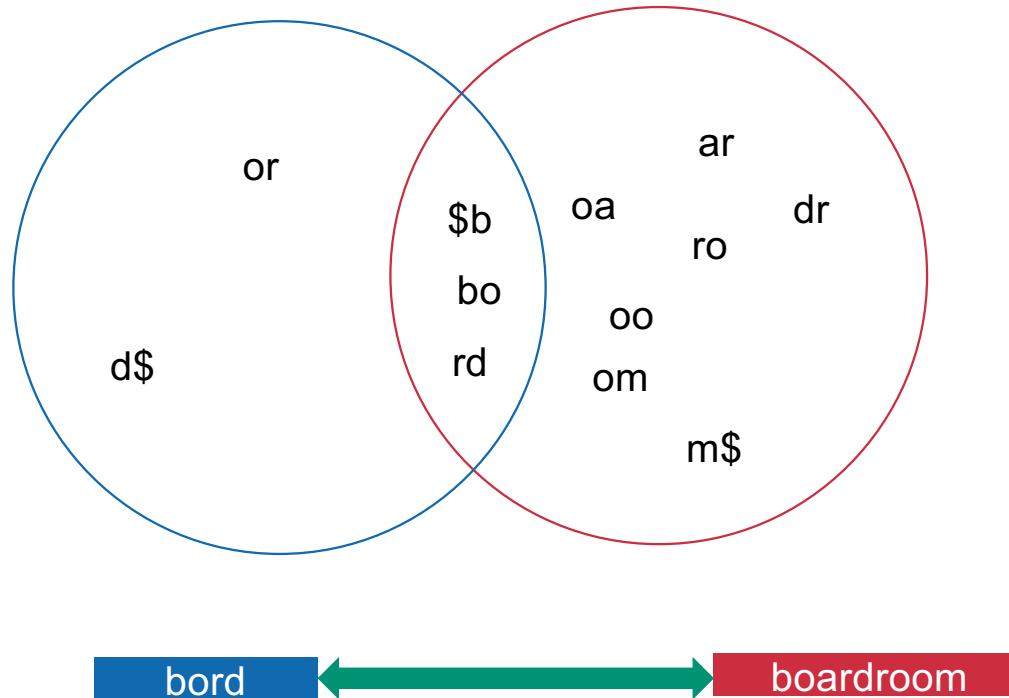
computer: five 3-grams in common

terran: one 3-gram in common

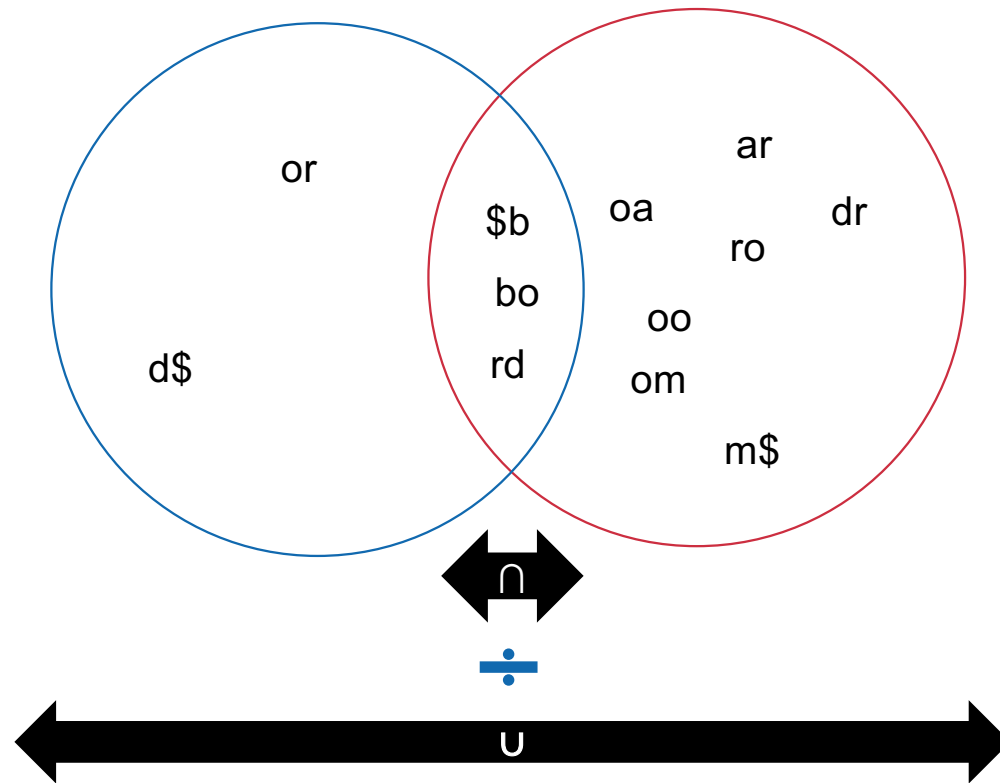
Threshold, e.g., 3

computer

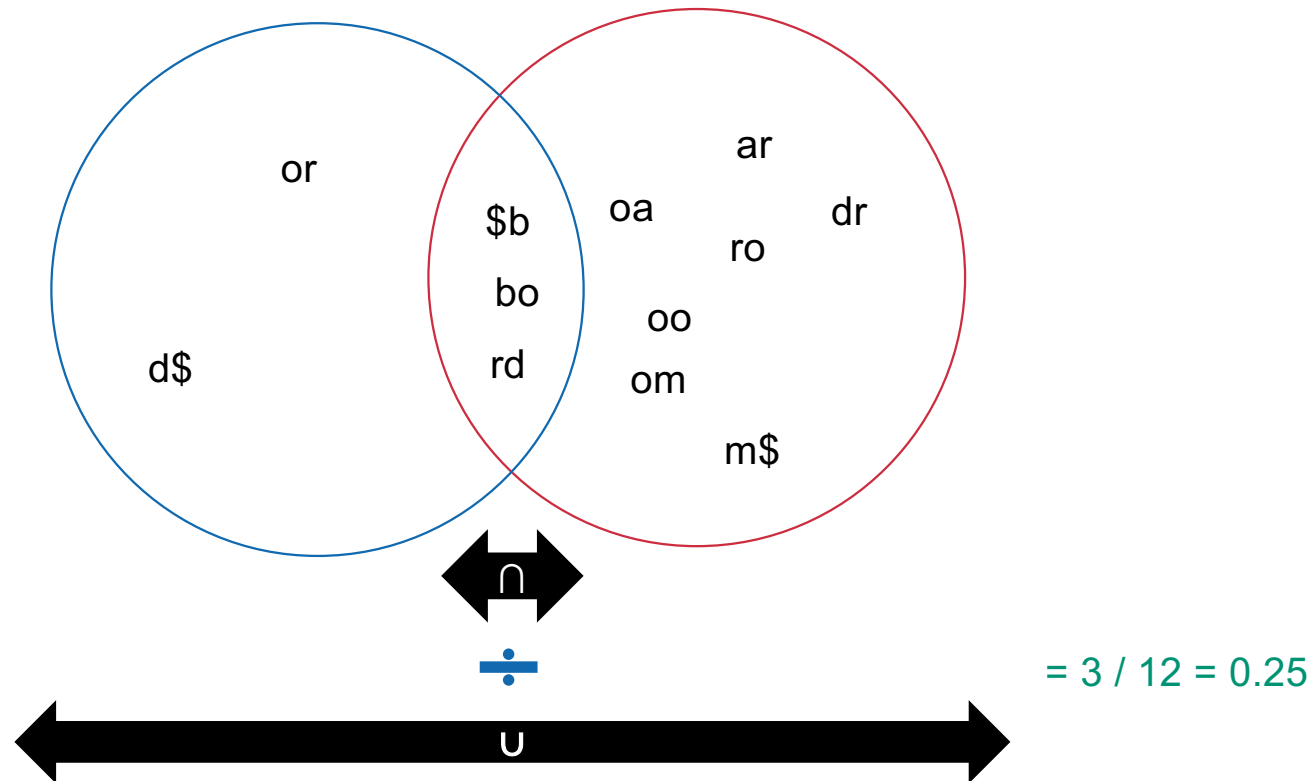
Issue: false "positives"



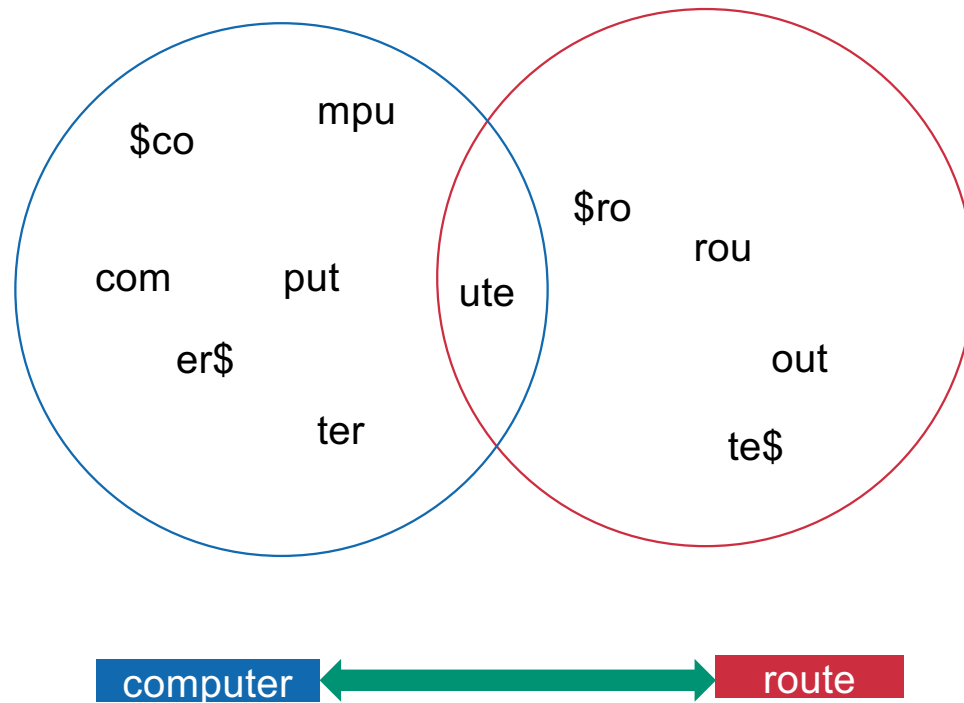
Jaccard coefficient



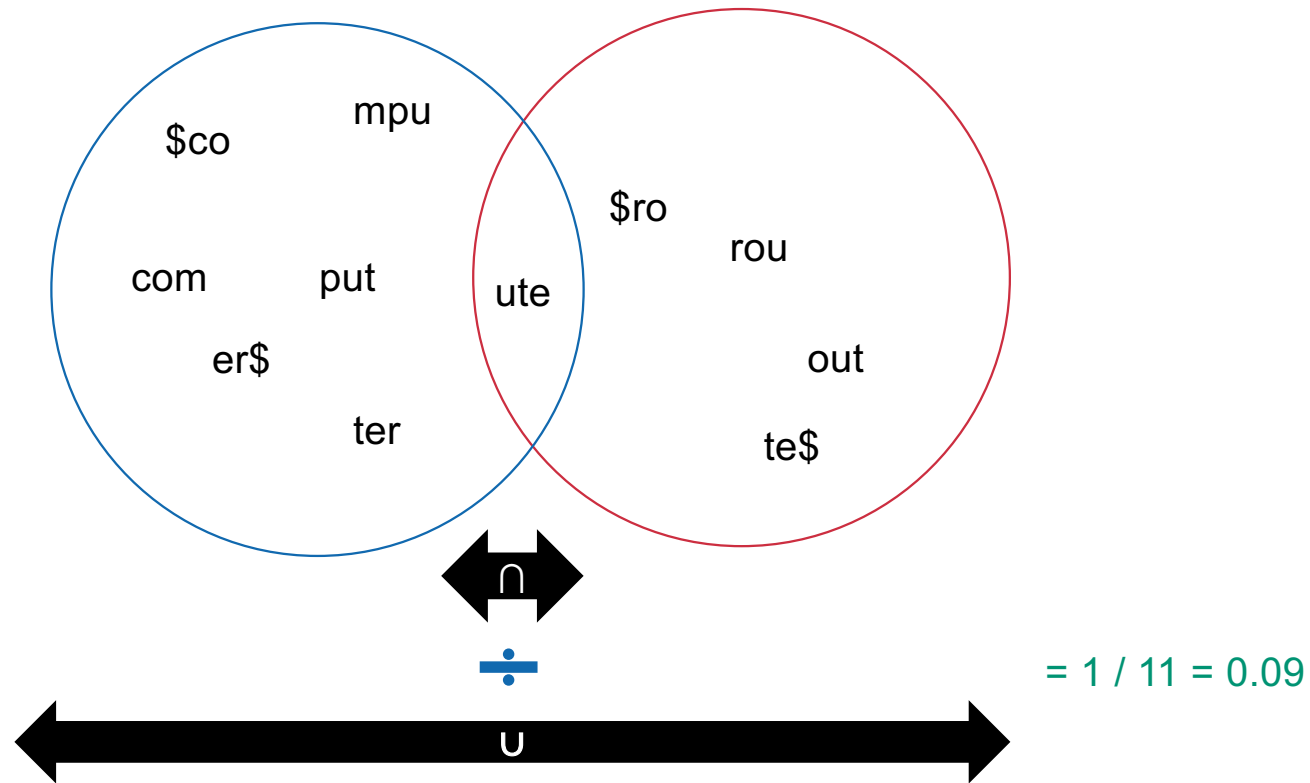
Jaccard coefficient



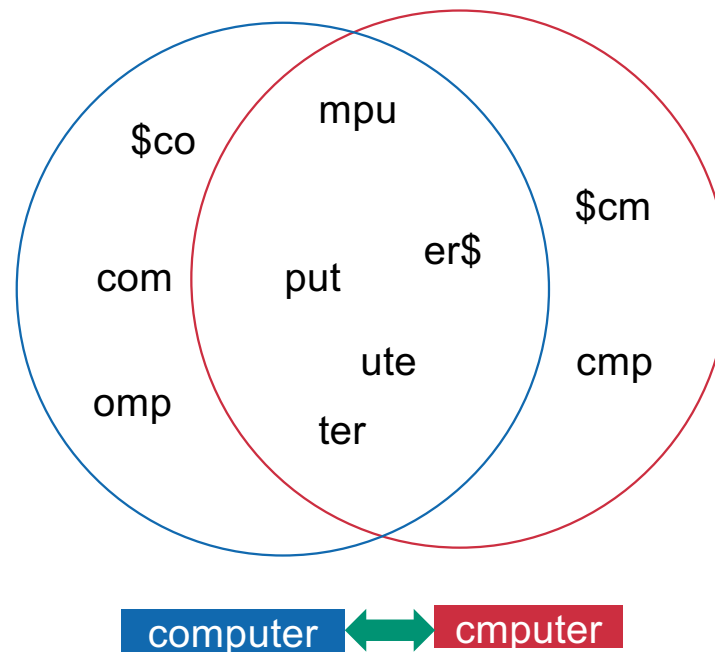
Jaccard coefficient



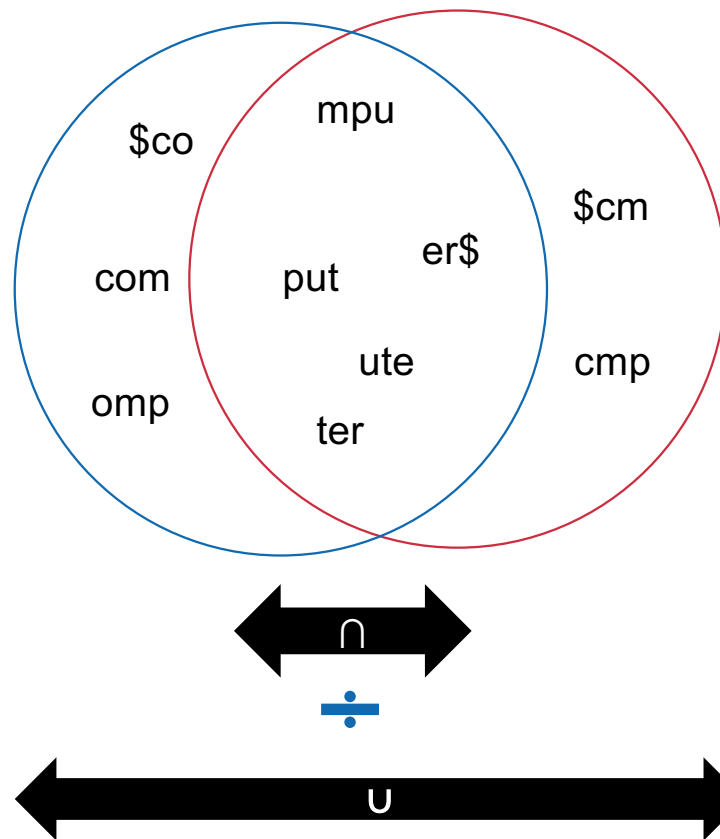
Jaccard coefficient



Jaccard coefficient



Jaccard coefficient



$$= 5 / 10 = 0.5$$

Updated method

Get k-grams from the query term

Updated method

Get k-grams from the query term



Look them up in the k-gram index

Updated method

Get k-grams from the query term



Look them up in the k-gram index



Compute Jaccard coefficients

Updated method

Get k-grams from the query term



Look them up in the k-gram index



Compute Jaccard coefficients



Keep terms within large Jaccard coefficients

Resource issue

We would need to extract k-grams
from **every term** found in the k-gram index,
compute the intersection and union,
and then the Jaccard coefficients

Shortcut

We **already know** the k-grams that the query term and the found term have in common



Shortcut



The denominator can be calculated with just the **found term's length**

Shortcut

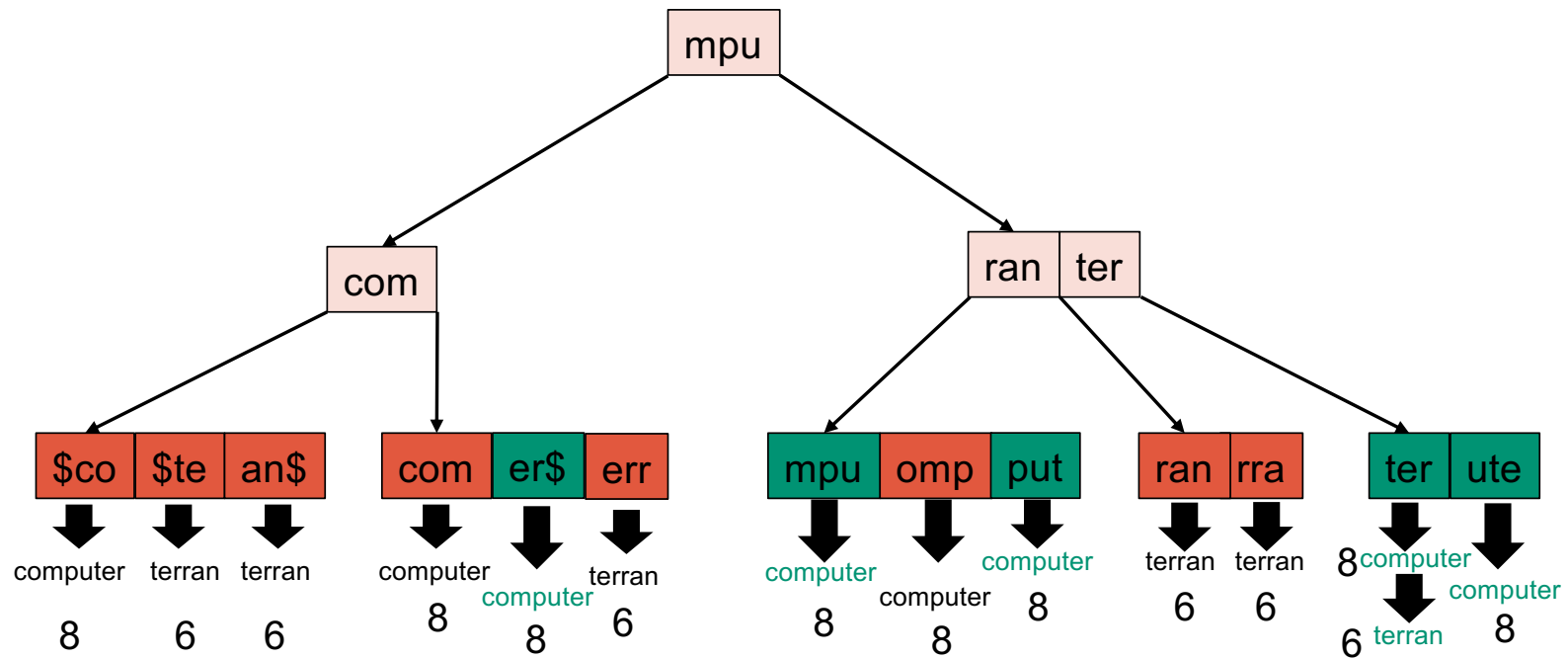


$\# \text{query term's k-grams} + \# \text{found term's k-grams} - \# \text{intersection}$

The denominator can be calculated with just the **found term's number of k-grams**

Spelling and k-grams

computer $\longrightarrow$ \$cm, cmp, mpu, put, ute, ter, er\$



Shortcut

J(computer, cmputer)

Shortcut

$$\begin{array}{r} 5 \\ \hline 7 + 8 - 5 \end{array}$$

Shortcut

5



10

Final method

Get k-grams from the query term



Look them up in the k-gram index



Compute **edit** distances



Keep terms within small **edit** distances

Context-sensitive spelling correction

graduate form ETH

Context-sensitive spelling correction

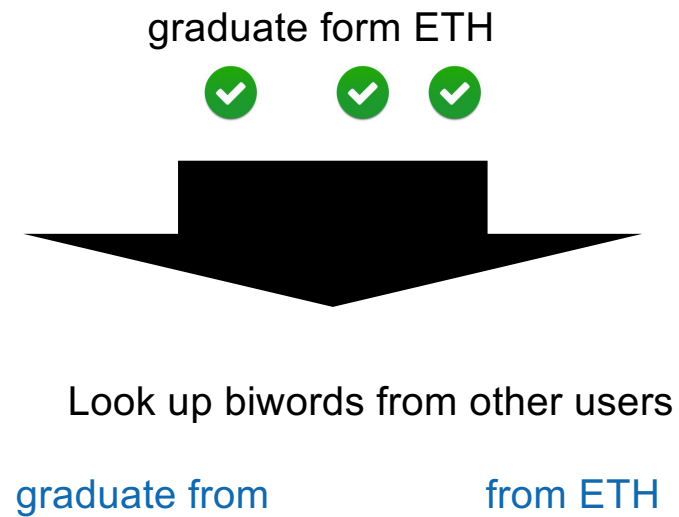
graduate form ETH



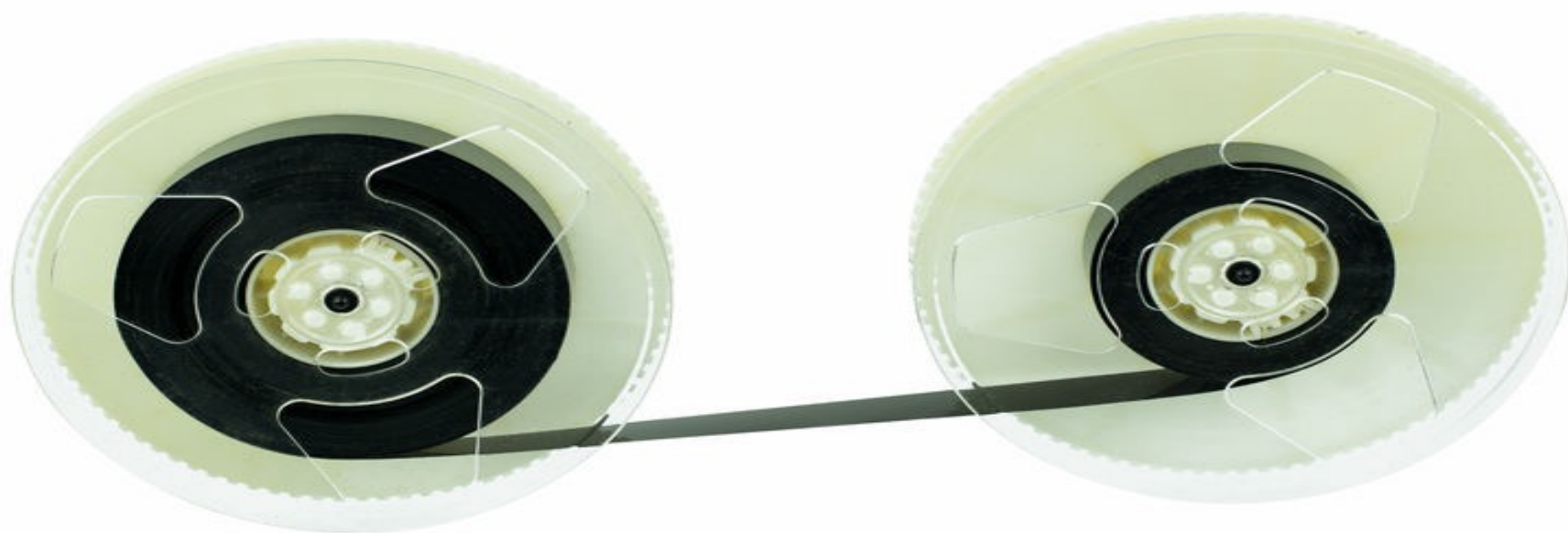
Context-sensitive spelling correction

graduation	ETA
graduated	ETC
graduate form	ETH
from	
foam	

Context-sensitive spelling correction



Phonetic correction



Soundex algorithm

Every word



4 characters fingerprint

Examples

Computer

Cmputer

Zurich

Information

Letter

Retrieval

Soundex algorithm

Change...	To...
A E H I O U W Y	0
B F P V	1
C G J K Q S X Z	2
D T	3
L	4
M N	5
R	6

Examples

Computer

Cmputer

Zurich

Information

Letter

Retrieval

Examples

Computer ➡ C0510306

Cmputer ➡ C510306

Zurich ➡ Z06020

Information ➡ I510650305

Letter ➡ L03306

Retrieval ➡ R0360104

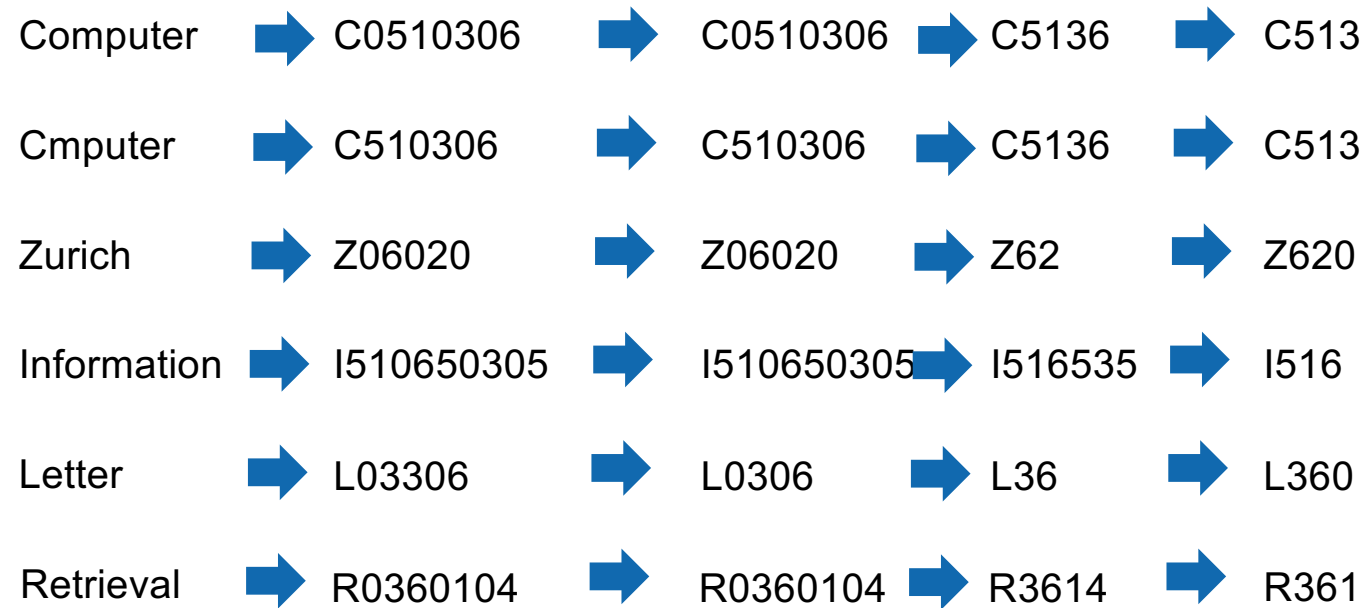
(Repeatedly) remove "duplicates"

Computer	➡	C0510306	➡	C0510306
Cmputer	➡	C510306	➡	C510306
Zurich	➡	Z06020	➡	Z06020
Information	➡	I510650305	➡	I510650305
Letter	➡	L03306	➡	L0306
Retrieval	➡	R0360104	➡	R0360104

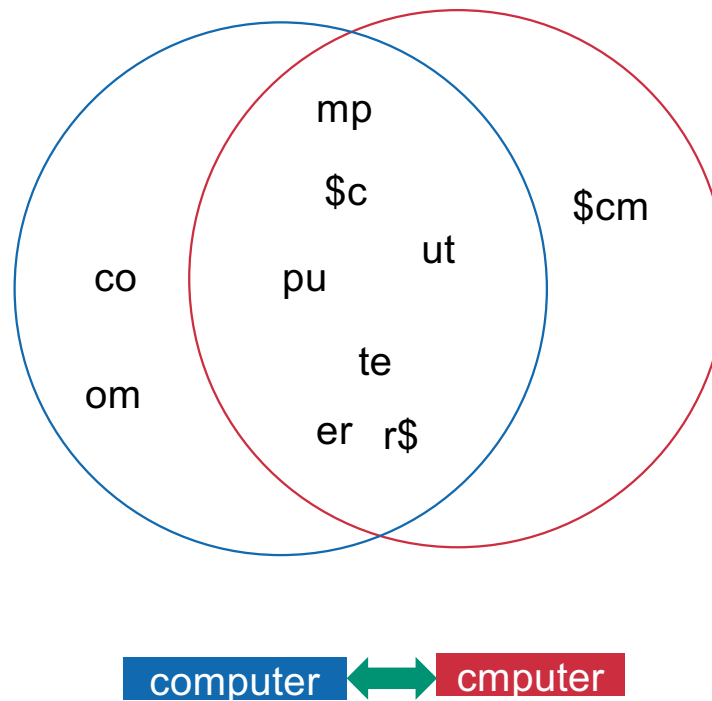
Remove zeros

Computer	➡	C0510306	➡	C0510306	➡	C5136
Cmputer	➡	C510306	➡	C510306	➡	C5136
Zurich	➡	Z06020	➡	Z06020	➡	Z62
Information	➡	I510650305	➡	I510650305	➡	I516535
Letter	➡	L03306	➡	L0306	➡	L36
Retrieval	➡	R0360104	➡	R0360104	➡	R3614

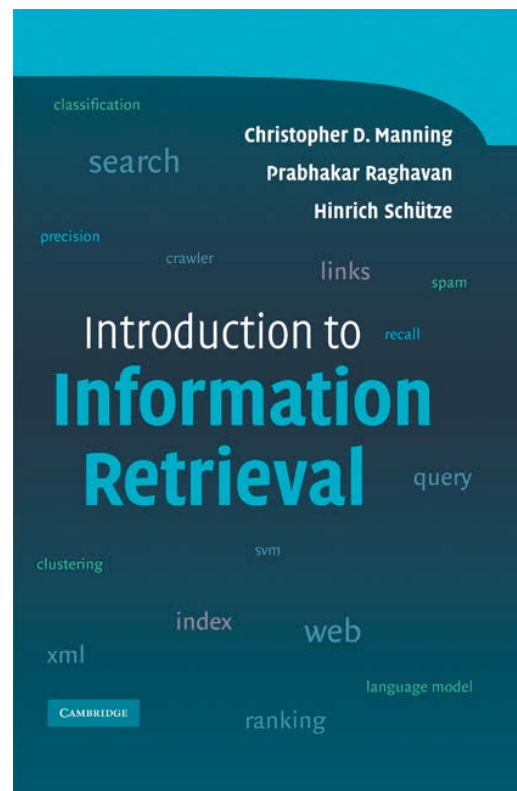
Pad and trim



Spelling and k-grams



This week's reading



Chapter 3

**Dictionaries
and
tolerant retrieval**